AI for Medical Imaging — Course Notes

October 5, 2026

Contents

1 Medical Imaging Modalities and the Role of AI **4**
- 1.1 What is medical imaging? . . . 4
- 1.2 Ultrasound . . . 4
- 1.3 X-rays . . . 5
- 1.4 Computed tomography . . . 5
  - 1.4.1 From a rotating X-ray to a 3D view . . . 5
  - 1.4.2 Tomographic reconstruction and back-projection . . . 6
- 1.5 Magnetic resonance imaging . . . 6
  - 1.5.1 CT versus MRI . . . 6
  - 1.5.2 Inside the scanner . . . 7
  - 1.5.3 Spin orientation and relaxation . . . 7
- 1.6 Beyond the four main modalities . . . 8
- 1.7 Trade-offs between modalities . . . 8
- 1.8 Radiation as a trade-off dimension . . . 9
- 1.9 Anatomical planes and axes . . . 10
- 1.10 Why AI? . . . 10

2 Segmentation: Networks, Preprocessing and Post-processing **11**
- 2.1 From classification to dense labelling . . . 11
- 2.2 Early approaches . . . 11
  - 2.2.1 A bit of history, thresholding and gradients . . . 11
  - 2.2.2 Graph-based methods . . . 12
  - 2.2.3 Limitations . . . 12
- 2.3 The deep learning era . . . 12
  - 2.3.1 Big data and GPUs, and why the shift happened . . . 12
  - 2.3.2 Convolutional networks and the fixed-size problem . . . 12
  - 2.3.3 Maintaining input size: U-Net and fully convolutional networks . . . 12
  - 2.3.4 Tuning the convolutions . . . 13
- 2.4 nnU-Net: the current state of the art . . . 13
  - 2.4.1 The nnU-Net paper: fixed, rule-based and empirical parameters . . . 14
- 2.5 Architectures: 2D, 3D and 2.5D . . . 15
- 2.6 Metadata and preprocessing . . . 15
- 2.7 Data augmentation . . . 16
- 2.8 Overview of training and sub-patching . . . 16
- 2.9 Post-processing . . . 17
  - 2.9.1 Connected-component filtering . . . 17
  - 2.9.2 DenseCRF (i): what it fixes . . . 18
  - 2.9.3 DenseCRF (ii): the energy . . . 18
- 2.10 Ensembling . . . 19
- 2.11 Other approaches . . . 19

2.12 Summary of notation . . . 20
2.12.1 Worked example of the notation . . . 20

3 Segmentation Losses and Their Gradients **21**
3.1 Formal definition and notation . . . 21
3.2 Training as an optimisation problem . . . 22
3.3 Cross-entropy . . . 23
3.3.1 Gradient of the cross-entropy loss . . . 23
3.4 The Sørensen–Dice coefficient and the Dice loss . . . 24
3.5 Derivation of the Dice-loss gradient . . . 25
3.6 Combining the two: the popular mix of losses . . . 25
3.7 Weak supervision: losses for incomplete annotations . . . 26
3.8 Taxonomy of segmentation losses . . . 27

4 Registration **28**
4.1 The registration problem . . . 28
4.2 Similarity and dissimilarity losses . . . 28
4.3 Coordinates and the affine matrix . . . 29
4.4 Elastic (non-rigid) registration . . . 31
4.5 Learned registration . . . 33
4.6 Rigid or non-rigid? Choosing by anatomy, and the trade-offs . . . 34

5 Metrics: Selection, Dice, Reporting **35**
5.1 What to measure depends on the task and on the problem . . . 35
5.2 Problem fingerprints: choosing metrics from the problem . . . 36
5.3 The Sørensen–Dice coefficient . . . 36
5.3.1 Over-segmentation versus under-segmentation . . . 37
5.3.2 What Dice does not capture . . . 37
5.3.3 Corner cases and the ε-modification . . . 38
5.4 Picking your metrics . . . 38
5.5 Reporting your metrics . . . 40
5.6 The limits of metrics and of annotations . . . 40
5.7 Calibration . . . 41

6 Distance-based Metrics and Their Implementation **42**
6.1 Significant numbers and measurement error . . . 42
6.2 The Hausdorff distance . . . 42
6.2.1 Preliminary notation . . . 43
6.2.2 Formal definition, continuous and discrete . . . 43
6.3 Computing the Hausdorff distance . . . 43
6.3.1 The four steps . . . 43
6.3.2 Worked example . . . 44
6.3.3 Boundary extraction . . . 44
6.3.4 Euclidean distance transform . . . 44
6.3.5 Masking . . . 45
6.3.6 Aggregating . . . 46
6.3.7 Runtime . . . 46
6.3.8 DisTorch: a GPU alternative . . . 46
6.4 Other distance-based metrics . . . 46
6.5 Implementation pitfalls across open-source tools . . . 47
6.6 Corner cases and undefined distances . . . 47
6.7 Resolution, units and significant figures . . . 48

7 Ethics, Data Management and Reproducibility **49**
7.1 Data collection and management . . . 50
7.1.1 Regulatory frameworks . . . 50
7.1.2 The GDPR . . . 50
7.1.3 Retrospective studies and consent . . . 50
7.1.4 Ethical committees in the Netherlands . . . 50
7.1.5 Data Management Plans . . . 50
7.1.6 Anonymous versus pseudonymous data . . . 50
7.1.7 The limits of anonymisation: patients are unique . . . 51
7.1.8 Licensing and public challenges . . . 52
7.1.9 Collaboration and data transfer agreements . . . 52
7.1.10 Code storage . . . 52
7.1.11 Data breaches and electronic hygiene . . . 52
7.2 Reproducibility . . . 53
7.2.1 The reproducibility checklist for papers . . . 53
7.2.2 Four meanings of "reproducible" . . . 53
7.2.3 The numerous sources of randomness . . . 53
7.2.4 True randomness versus pseudo-randomness . . . 54
7.2.5 Eliminating some sources of (pseudo-)randomness . . . 54
7.2.6 Deterministic configuration: the nnU-Net pipeline . . . 54
7.2.7 Private data and public code . . . 55
7.3 Inference, deployment and responsibility . . . 55

8 Image Reconstruction **56**
8.1 Motivation: MRI acquisition and under-sampling . . . 56
8.2 Compressed sensing . . . 56
8.3 Learning MRI reconstruction from sub-sampled k-space . . . 57
8.4 Generative imaging: denoising, super-resolution and translation . . . 57
8.5 What the deck does not cover . . . 58

9 Domain Adaptation, Transfer Learning and Federated Learning **58**
9.1 Transfer Learning and Domain Adaptation . . . 59
9.2 Federated Learning . . . 60
9.3 Privacy-preserving Learning: Differential Privacy . . . 61
9.4 Drawbacks: No Free Lunch . . . 61
9.5 Foundation Models: SAM, MedSAM and Beyond . . . 61

10 Classification and Explainability **63**
10.1 Classification among the imaging tasks . . . 63
10.2 What is the question, actually? . . . 63
10.3 Panels, consensus, agreements and disagreements . . . 63
10.4 Multiple-instance learning and very large images . . . 65
10.5 Quantification versus decision . . . 65

A Appendix: Past Exam Questions with Model Answers **66**
A.1 2025 exam . . . 66
A.2 2024 quiz . . . 69
A.3 Coverage gaps . . . 70

1 Medical Imaging Modalities and the Role of AI

1.1 What is medical imaging?

Medical imaging aims at visualising the inside of the body without opening it. The keyword of the field is *modality*: there are many of them, each defined by the physical agent it uses to interrogate the body and by the detector that records the response, and most of them are non-invasive. They are used for *diagnostic* purposes and for *(surgery) planning*, i.e. to decide what is where before and while acting on the patient. This chapter walks through the four modalities that structure the course — ultrasound, X-rays, computed tomography, and magnetic resonance imaging — then through further modalities, their trade-offs, and the reasons why AI is brought into the picture. (Source: `AI4MI.pdf`, slides 8 and 22)

1.2 Ultrasound

Ultrasound (US) acquisition works with a probe held in contact with the patient: the operator places the probe on the skin and moves it while watching the live image; the physical agent is sound waves, which is what makes the modality radiation-free. The deck contrasts the acquisition setting with the resulting image, an embryo at 14 weeks (see `AI4MI.pdf`, slide 9), whose appearance is typical of the modality: a grey-level image in which organ boundaries appear through characteristic speckle rather than as sharp edges. The equipment has shrunk over time: alongside the classic scanner — a console with a screen, a keyboard and a probe (see `AI4MI.pdf`, slide 10) — portable scanners now exist, so the equipment can be moved rather than the patient. (Source: `AI4MI.pdf`, slides 9–11)

Two properties of ultrasound matter for the rest of the course. First, ultrasound is the *operator dependent* modality: because the probe position and angle determine what is imaged, the quality and the field of view depend on who holds the probe, and air and/or bone may prevent visualisation of organs altogether. Second, it involves neither radiation nor sedation, which makes it comparatively cheap and a less invasive option in the trade-off table of Table 1. On the exam-style formulation: of the commonly considered modalities (ultrasound, MRI, CT, X-ray, PET), ultrasound is the operator-dependent one. (Source: `AI4MI.pdf`, slides 9 and 21)



Figure 1: Ultrasound image of an embryo at 14 weeks: organ structures are visible as grey-level contours through the speckle pattern typical of sound-wave imaging. Source: `AI4MI.pdf`, slide 9.

1.3 X-rays

X-rays were discovered by Wilhelm Röntgen; the deck illustrates the discovery with the first radiograph, the *Hand mit Ringen* (1895) — a hand with rings, in which the bones and the metal rings are visible through the soft tissue — and with a modern frontal view of a knee. In a projection radiograph, structures are superimposed along the ray path and contrast comes from the differential attenuation of the beam, so bone, which absorbs strongly, appears bright while soft tissue is comparatively transparent. (Source: `AI4MI.pdf`, slide 12)

The production of X-rays is a two-step physical process (see `AI4MI.pdf`, slide 13). A filament inside a glass envelope is heated and *emits electrons by thermionic emission*; these electrons are then *accelerated by a high voltage* and hit a metal target, where the loss of kinetic energy produces X-rays. A copper rod attached to the target dissipates the heat generated in this collision; the glass envelope keeps the electron path in vacuum. The slide also places the modality on the electromagnetic spectrum: wavelength decreases (and photon energy increases) from visible and ultraviolet light through *soft X-rays* and *hard X-rays* to gamma rays, with photon energies running from about 1 eV at the visible-light end up to the MeV range. Applications are read off that axis: X-ray crystallography sits near 10 keV, mammography at a few tens of keV, medical CT at about 100 keV, and airport security at a few hundred keV. (Source: `AI4MI.pdf`, slide 13)

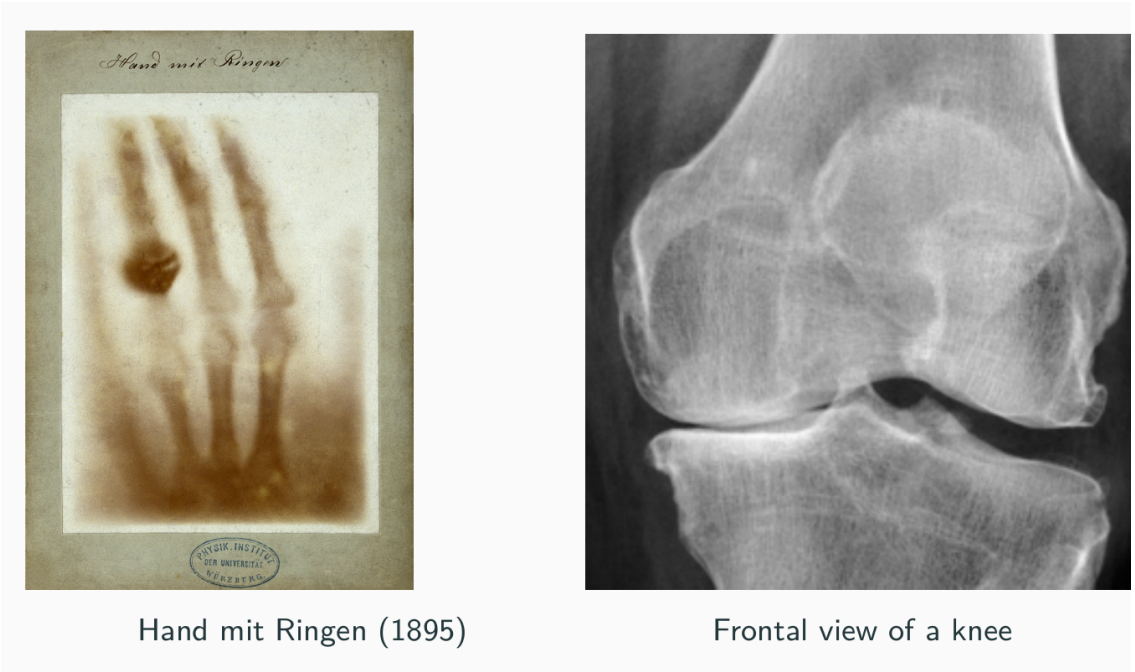


Figure 2: The first radiograph, *Hand mit Ringen* (1895), next to a modern frontal radiograph of a knee. Source: `AI4MI.pdf`, slide 12.

1.4 Computed tomography

1.4.1 From a rotating X-ray to a 3D view

Computed tomography (CT) is presented as the "logical" next step after the plain radiograph: to go from one projection to a three-dimensional view, the *X-ray beam is made to rotate around the patient*. An X-ray source and a detector are mounted on a gantry facing each other, so that at every angle the detector array records the transmitted beam, while a motorised table translates the patient through the gantry; a modern scanner is a large ring-shaped machine with the table running through the bore. The result is a set of projections taken from many angles, which is what makes a volumetric reconstruction possible. CT therefore serves both purposes that the deck attributes to medical imaging: it is used for *diagnosis*, and the volumetric anatomical detail it produces is what

makes it usable for *(surgery) planning.* (Source: `AI4MI.pdf`, slides 8 and 14)

1.4.2 Tomographic reconstruction and back-projection

The reconstruction problem is posed geometrically on the slide (after *Digital Image Processing*, Gonzalez and Woods, §5.11). For a fixed angle θ_k the measurement forms a one-dimensional *projection* $g(\rho, \theta_k)$ indexed by the signed distance ρ of the ray from the origin; the ray associated with that reading is the line

$$\ell(\rho, \theta_k): \quad x\cos\theta_k + y\sin\theta_k = \rho, \tag{1}$$

and $g(\rho, \theta_k)$ measures the integral of the object along that line — the slide labels $g(\rho_j, \theta_k)$ a "point … in the projection" and $g(\rho, \theta_k)$ the "complete projection … for a fixed angle", along the corresponding line $\ell(\rho, \theta_k)$. The *back-projection* method reconstructs the image by taking each projection reading and spreading it back along the line it came from, then summing the contributions of all angles. Writing $\hat{f}$ for the reconstructed image and K for the number of angles,

$$\hat{f}(x, y) \;=\; \frac{\pi}{K}\sum_{k=0}^{K-1} g(x\cos\theta_k + y\sin\theta_k,\ \theta_k)\,, \tag{2}$$

since for the pixel (x, y) the argument $x\cos\theta_k + y\sin\theta_k$ is exactly the distance ρ of the ray through that pixel at angle θ_k, i.e. the pixel accumulates the ray values of all rays passing through it. *No source in this repo for the summation formula itself* (equation from the cited Gonzalez and Woods, §5.11). The deck illustrates the effect of K on the result: "reconstruction for various numbers of projections" shows that with very few projections the object is smeared into streaks, and that the reconstructed shape sharpens as more projections are used. (Source: `AI4MI.pdf`, slide 15; figure reproduced from Gonzalez and Woods, *Digital Image Processing*, 4th ed., §5.11)

Worked example: back-projection on a 2×2 image (exam §1e). The exam gives the image $\left(\begin{smallmatrix} A & B \\ C & D \end{smallmatrix}\right)$ and six ray sums in three directions: the rows ($A + B = 7$, $C + D = 7$), the columns ($A + C = 6$, $B + D = 8$) and the two diagonals ($A + D = 5$, $B + C = 9$). *Back-projection* assigns to each pixel the sum of all ray values passing through it; every pixel lies on exactly one row, one column and one diagonal ray:

$$\begin{aligned} \mathrm{BP}(A) &= 7 + 6 + 5 = 18, & \mathrm{BP}(B) &= 7 + 8 + 9 = 24, \\ \mathrm{BP}(C) &= 7 + 6 + 9 = 22, & \mathrm{BP}(D) &= 7 + 8 + 5 = 20. \end{aligned}$$

This raw back-projection is blurred: e.g. $\mathrm{BP}(A) = (A+B)+(A+C)+(A+D) = 2A+(A+B+C+D)$, i.e. each pixel receives its own value three times plus every other pixel once. The total $A+B+C+D$ is the sum of the readings of any single direction ($7 + 7 = 14$), so subtracting it and dividing by the number of directions minus one ($3 - 1 = 2$) removes the blur:

$$A = \frac{18-14}{2} = 2, \quad B = \frac{24-14}{2} = 5, \quad C = \frac{22-14}{2} = 4, \quad D = \frac{20-14}{2} = 3,$$

which satisfies all six ray sums. The same values follow from solving the linear system directly, e.g. $A = \frac{1}{2}\big[(A + B) + (A + C) - (B + C)\big] = \frac{1}{2}(7 + 6 - 9) = 2$. The subtraction step is the discrete analogue of why plain back-projection alone gives the streaky, blurred reconstructions shown on the slide. (Source: `AI4MI.pdf`, slide 15; the exercise is `exams/exam2025.pdf`, §1e; the arithmetic is a derivation, no model answer is provided)

1.5 Magnetic resonance imaging

1.5.1 CT versus MRI

Placing a CT slice and two MRI slices of the same brain next to each other (see `AI4MI.pdf`, slide 17) shows why MRI is described as having great tissue contrast: in the CT image the skull is the

dominant structure and the interior of the brain has little contrast, whereas both MRI images resolve the internal structure of the brain. The two MRI images, labelled *T1* and *FLAIR*, are acquired with different weightings of the same anatomy, i.e. the same physical slice looks different because the acquisition emphasises different tissue properties; the deck keeps both as an illustration that MRI is not one image but a family of contrasts and that it can image in multiple planes. CT, by contrast, is described as quick and anatomical, and both CT and MRI may need a contrast agent. (Source: `AI4MI.pdf`, slides 17 and 21)

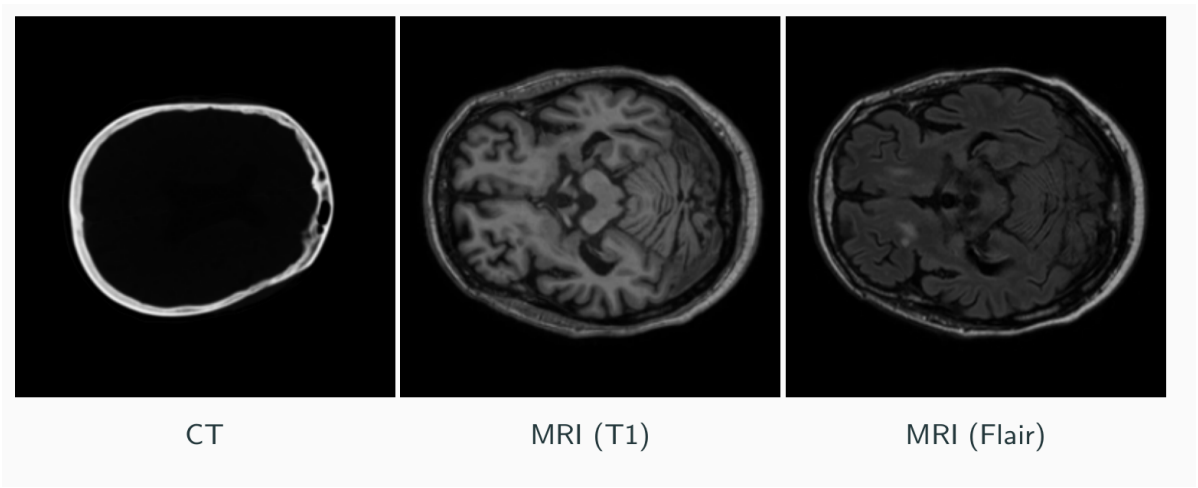


Figure 3: The same brain imaged with CT (left), MRI T1 (middle) and MRI FLAIR (right): the CT slice is dominated by the skull, while the two MRI weightings resolve the internal tissues with different contrast. Source: `AI4MI.pdf`, slide 17.

1.5.2 Inside the scanner

The cross-section of an MRI machine explains the hardware: the patient lies on the scanner table and is moved into the *scanner bore*; around the bore runs the *superconducting primary electromagnet coil*, kept superconducting by a *cryostat filled with liquid helium*, itself wrapped in *thermal insulation*, with *service connections* including an emergency cryostat vent pipe; inside the main magnet sit the *magnetic gradient coils* and the *RF transmit coil*, while a *body-part-specific receive coil* is placed on or around the region of interest and the scanner table carries the patient. The longitudinal section shown next to the cross-section confirms that these elements run along the whole length of the machine rather than only around its middle. (Source: `AI4MI.pdf`, slide 18)

MRI machines are also large, heavy pieces of infrastructure: the deck documents new MRI scanners being hoisted into a hospital building by crane at the EMC (Rotterdam), an illustration of the installation cost and floor-space requirements of the modality. (Source: `AI4MI.pdf`, slide 19)

1.5.3 Spin orientation and relaxation

The basic idea of MRI is simple: tissue contains many tiny magnetic "spins"; we first line them up, then knock them out of line, and finally record the signal they give off while they settle back. The deck shows this as a four-step sequence ("spin orientation during relaxation"), where the vector sum of all the spins is called the *magnetisation*:

1. **No field.** The spins point in random directions, so they cancel each other out and the total magnetisation is zero.
2. **Apply the B_0 field along z.** The strong main magnet of Figure 4 makes the spins line up along the z axis, so the magnetisation now points along z.
3. **90° RF pulse.** A short radio-frequency pulse tips the magnetisation by 90°, from the z axis into the xy plane.

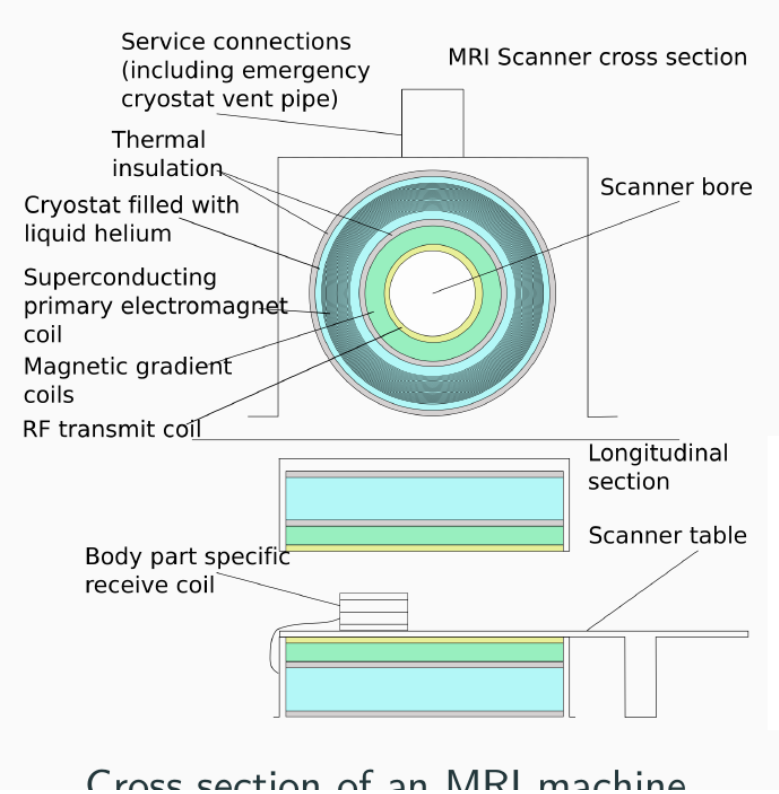


Figure 4: Cross section of an MRI scanner: the patient lies in the scanner bore, inside a superconducting primary electromagnet cooled by a liquid-helium cryostat, with gradient coils and an RF transmit coil around it, plus a body-part-specific receive coil on the table. Source: `AI4MI.pdf`, slide 18.

4. **Relaxation.** After the pulse the magnetisation returns to its equilibrium state, aligned along z again.

The scanner measures what happens during step 4: this return to equilibrium, called *relaxation*, is the signal from which the image is formed. Different tissues relax differently, so we can choose an acquisition that emphasises one kind of relaxation behaviour over another. This is why the same brain looks different in the T1 and FLAIR images of Figure 3: same anatomy, different contrast. (Source: `AI4MI.pdf`, slide 18)

1.6 Beyond the four main modalities

The deck lists further modalities: PET scans, photon-counting CT, scanners used with a contrast agent, and histopathology. A PET scan is shown alongside an MRI with contrast and a histopathology image to make the point that the modality is not only a piece of equipment but also the type of visualisation it produces: PET delivers functional (metabolic) information (it is classified as physiological in the trade-off table, Table 2), MRI with contrast improves the visibility of specific structures, while histopathology reaches the tissue level entirely outside radiology. The list is open-ended — the deck writes "…" after these examples. (Source: `AI4MI.pdf`, slide 20)

1.7 Trade-offs between modalities

Modalities are not ordered but traded off along four axes: *cost*, *capabilities*, *compatibilities* and *patient comfort* (Table 1, after Huda, Slone and Aaland, 2002). The exam-relevant entries: ultrasound is cheap and radiation-free but *operator-dependent*; CT is quick and anatomical but uses radiation; MRI has the best soft-tissue contrast but is *relatively slow and needs sedation in young children*; nuclear imaging uniquely gives physiological information but poor anatomical detail. (Source: `AI4MI.pdf`, slide

21)

Modality	Physical agent	Strength	Weakness	Comment
Plain film	X-ray	Global view of anatomy	Does not reveal fine detail of organs	Least expensive
Ultrasound	Sound waves	Good organ detail, anatomical	Operator-dependent; air and/or bone may prevent visualisation of organs	No radiation, no sedation; less invasive; relatively cheap
CT	X-ray	Good organ detail, quick, anatomical	May need sedation and/or contrast	Moderately expensive, radiation
MR	Electromagnetic waves	Great tissue contrast, multiple planes, anatomical; new inroads to functional imaging	Relatively slow, needs sedation in younger children; may need contrast	Most expensive; no radiation
Nuclear	Gamma rays	Physiological (functional)	Organ-specific; poor on anatomic detail	Moderately expensive

Table 1: Trade-offs between modalities along the axes cost, capabilities, compatibilities and patient comfort. After Huda, Slone and Aaland (2002), as presented in the course. Source: `AI4MI.pdf`, slide 21.

A second table quantifies the comparison (Table 2): soft-tissue contrast and spatial resolution move against each other (X-ray and CT resolve below 0.5 mm with weaker soft-tissue contrast; MRI and nuclear imaging have the best contrast at millimetre-to-centimetre resolution), while cost and acquisition time grow together from radiography to CT, nuclear imaging and MRI. (Source: `AI4MI.pdf`, slide 22)

Modality	Detector	Info. (a/p)	Contrast	Res. (mm)	Time (min)	Cost ($K)
X-ray, screen/film	Screen/film	a	3	< 0.1	10+	0.2
CR / DR	PSP / FPD	a	3	< 0.3	10+	0.2
Mammography	Screen/film	a	3	< 0.1	10+	0.2
Mammography	FPD (digital)	a	3	< 0.2	10+	0.2
Fluoroscopy	II & TV/CCD	a	3	< 0.3	30+	0.4
DSA	FPD, II & CCD	a	1	< 0.3	10+	2.5
CT	X-ray detector array	a	2	< 0.5	10+	1.2
NM	NaI crystal	p	1	< 3.0	30+	1.0
SPECT	NaI crystal	p	1	< 3.0	30+	1.0
PET	Gamma-ray det. array	p	1	< 3.0	40+	1.5
MRI	R.f. coil	a, p	1	10	40+	2.0

Table 2: Typical upper-end characteristics of the principal modalities: information primarily on anatomy (a) or physiology (p), soft-tissue contrast (1 = best), spatial resolution, imaging time and 2004 cost. The tissue attribute imaged is density, effective atomic number and thickness for the X-ray family, radionuclide uptake by organ for NM/SPECT/PET, and proton spin relaxation for MRI. Source: `AI4MI.pdf`, slide 22.

1.8 Radiation as a trade-off dimension

Among the four axes, radiation is the one the deck isolates with an explicit chart: the xkcd radiation dose chart, whose unit is the sievert and whose point is to compare the effective dose of common activities and examinations on one logarithmic scale (Figure 5). The chart makes the orders of magnitude concrete: a dental X-ray is of the order of 5 µSv, a mammogram around 0.4 mSv, a chest X-ray around 20 µSv and a head CT around 2 mSv, while a chest CT reaches about 7 mSv; for reference, the normal yearly background dose received by an average person is about 4 mSv (about 85% of it natural, nearly all of the rest from medical scans), and the maximal yearly dose

permitted for US radiation workers is 50 mSv. Since the chart uses effective dose, it also places non-medical exposures (a flight from New York to LA, living within 50 miles of a coal power plant, sleeping next to someone) on the same axis. This is exactly the asymmetry expressed in the trade-off table: ultrasound and MR carry no radiation, while CT and nuclear imaging do, which is one of the reasons for choosing a modality rather than a "best" modality. (Source: `AI4MI.pdf`, slide 23; chart from `https://xkcd.com/radiation/`)

The course's flashcards add the remaining reference points of the same chart, expressed as "days of background" (the average background dose is about 10 µSv per day): eating one banana 0.1 µSv; an arm X-ray 1 µSv (0.1 day); a dental X-ray 5 µSv (half a day); a chest X-ray 20 µSv (2 days); a flight from New York to Los Angeles 40 µSv (4 days); a mammogram 400 µSv (40 days); a chest CT 7 mSv (700 days). At the other end, 100 mSv is the lowest one-year dose clearly linked to increased cancer risk (25 average yearly background doses), 2 Sv causes severe radiation poisoning, 4 Sv is usually fatal without treatment, and 8 Sv is fatal. A single chest CT is thus far from dangerous, but not negligible. (Source: `UvA__AI4MIv0.1.apkg`; `AI4MI.pdf`, slide 23)

The flashcards also summarise the modalities along the two patient-side axes, radiation and invasiveness (Table 3). (Source: `UvA__AI4MIv0.1.apkg`)

Modality	Radiation?	Invasive?
Ultrasound	No	No
X-ray	Yes	No
CT scan	Yes	No
MRI	No	No
PET scan	Yes	Minimally
Scan with contrast agent	May be	Minimally
Endoscopy	No	Yes
Histopathology	No	Yes

Table 3: Radiation and invasiveness of the modalities. Source: `UvA__AI4MIv0.1.apkg`.

1.9 Anatomical planes and axes

Scans are viewed and sliced along three standard anatomical planes: the *sagittal* plane (dividing left from right; the *median sagittal* plane is the one through the midline), the *frontal* or *coronal* plane (dividing front from back), and the *transversal* (axial) plane (dividing top from bottom). The corresponding axes are the *sagittal* axis (front–back), the *transversal* axis (left–right) and the *longitudinal* axis (head–feet). These names are the vocabulary used whenever a 3D scan is cut into 2D slices (§2). (Source: `UvA__AI4MIv0.1.apkg`)

1.10 Why AI?

The motivation is first quantitative. There are more and more images to process, because resolution keeps increasing while acquisition time keeps going down, and because new modalities keep appearing; the volume of data therefore grows on both counts, per examination and in number of examinations. Second, it is about tasks: anomaly detection, triage, reconstruction and visualisation are listed as jobs that can be assisted by AI, and these span the whole pipeline that this course follows — reconstruction from raw measurements, segmentation and detection in the image, and the metrics used to judge the result. Third, the course states the goal in terms of the division of labour: *quantification versus decision* — AI is well suited to quantifying measurements, while the decision remains a clinical act — and the role of these methods is to *support clinicians, not replace them.* (Source: `AI4MI.pdf`, slides 25–27)

The following chapters develop the corresponding techniques: segmentation (Chapter 2) and its losses (Chapter 3), the metrics (Chapter 5, including the Dice score of §5.3) and the distance-based metrics (Chapter 6), registration, and the ethics and reproducibility issues that medical data brings with them. (Source: `AI4MI.pdf`, slides 25–27)

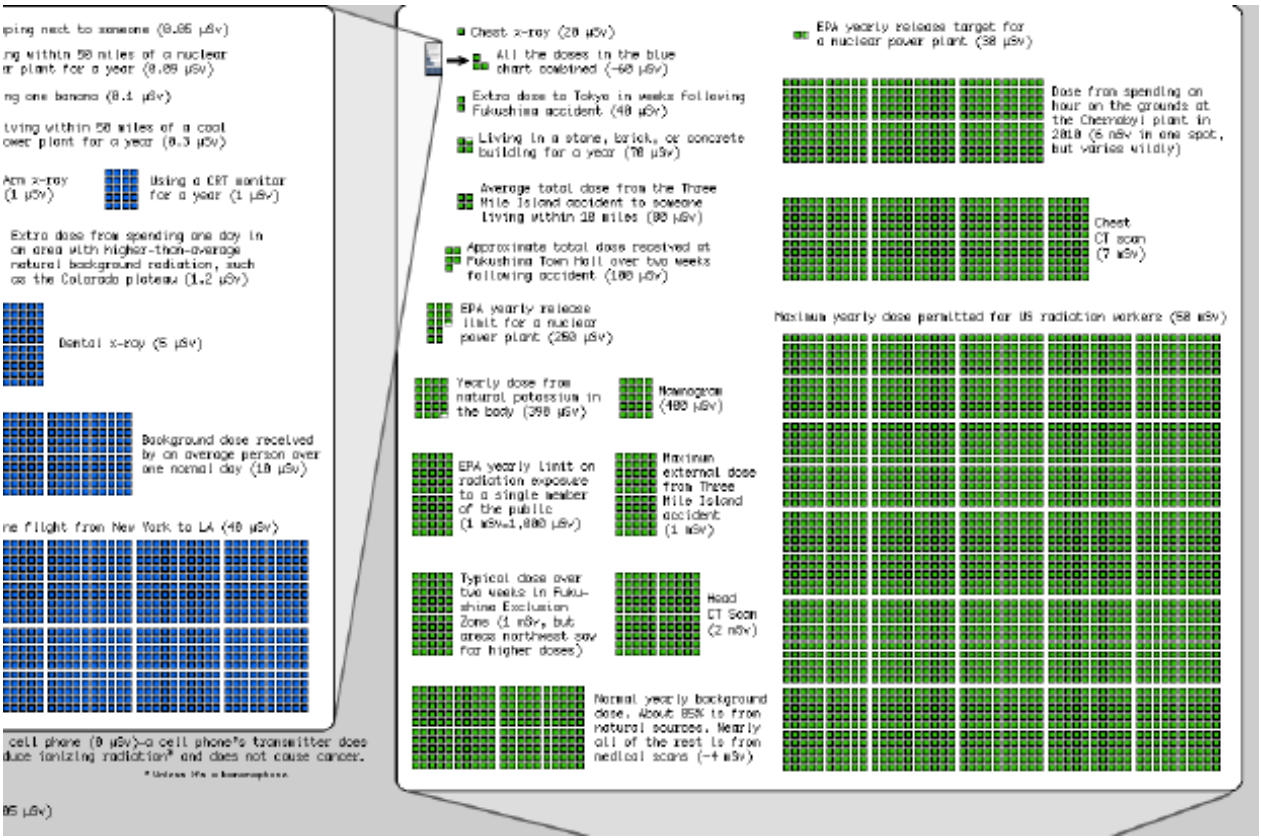


Figure 5: The medical block of the xkcd radiation dose chart, comparing the effective doses (in sieverts) quoted in the text: chest X-ray (20 µSv), dental X-ray (5 µSv), mammogram (0.4 mSv), head CT (2 mSv), chest CT (7 mSv), the normal yearly background dose (4 mSv) and the permitted yearly dose for US radiation workers (50 mSv). Source: `AI4MI.pdf`, slide 23 (chart from `https://xkcd.com/radiation/`).

2 Segmentation: Networks, Preprocessing and Post-processing

2.1 From classification to dense labelling

Medical image analysis is typically posed as one of three tasks. In **classification** the network returns a single label for the whole image; in **object detection** it additionally localises the object with a bounding box; in **image segmentation** it assigns a class to *every* pixel (or voxel), so that the output is a dense map with the same spatial extent as the input. The deck illustrates the three settings on a short-axis view of the heart: the classification output is the tag "left ventricle", the detection output is a box drawn around the left ventricle, and the segmentation output is the full red mask of the ventricle. Because the prediction is a dense map, its quality is judged voxel by voxel: the deck shows some label together with some prediction, and then the same pair zoomed in, which makes the point that segmentation errors are local and structural rather than global. (Source: `20260903_segmentation_01.pdf`, slides 4–5)

2.2 Early approaches

2.2.1 A bit of history, thresholding and gradients

One of the earliest successes of volumetric medical image analysis was the extraction of surfaces rather than labels: the *marching cubes* algorithm of Lorensen and Cline (1987) builds a high-resolution three-dimensional surface out of a volumetric input, so that a CT scan can be converted into a displayable rendering of the bones it contains. This line of work is geometry-driven signal processing: the surface is constructed from the data by a fixed algorithm, not learned from examples. (Source:

20260903_segmentation_01.pdf, slide 6)

The simplest way to obtain a segmentation is **thresholding**: an intensity transform is applied to the input image, and the voxels whose transformed intensity crosses a fixed threshold are declared foreground. Combined with the *gradient*-based variant of the same idea, these are the classical methods of Gonzalez and Woods' *Digital Image Processing*; the deck stresses that this family is mostly signal processing rather than machine learning, since every step is hand-designed and the "knowledge" lives in the operator, not in parameters fitted to data. (Source: 20260903_segmentation_01.pdf, slide 7)

2.2.2 Graph-based methods

A second classical family formulates segmentation as energy minimisation on a graph: the image is turned into a graph whose nodes are voxels, and the segmentation is the labelling that minimises an energy made of unary terms (how well a voxel fits its label) and pairwise terms (how likely two neighbouring voxels are to share a label). The seminal paper of this family is Boykov, Veksler and Zabih's *Fast approximate energy minimization via graph cuts*, which showed how such energies can be optimised efficiently by min-cut/max-flow algorithms. (Source: 20260903_segmentation_01.pdf, slide 8)

2.2.3 Limitations

These classical pipelines share two structural weaknesses: they are *sensitive to parameters and heuristics*, and they are *difficult to parallelise or accelerate with dedicated hardware*. The pipeline the deck shows is a chain of such decisions — input image, then an intensity transform, then a raw segmentation, then a final segmentation — in which each arrow hides a hand-tuned choice: change the threshold, the smoothing width or the connectivity, and the final mask changes. None of these steps has the regular, data-parallel structure that GPUs are good at, so scaling them with hardware is not straightforward. (Source: 20260903_segmentation_01.pdf, slide 9)

2.3 The deep learning era

2.3.1 Big data and GPUs, and why the shift happened

The deep learning era changed both halves of the problem at once. First, "large" datasets make it possible to *learn* the features instead of defining them by hand: the discriminative operators are no longer chosen by the engineer but fitted to the training data. Second, GPUs and dedicated hardware make the corresponding computations fast, notably because they rely on parallel/SIMD operations — exactly the structure that convolutional networks have and that the hand-designed pipelines of the previous section lack. (Source: 20260903_segmentation_01.pdf, slide 11)

2.3.2 Convolutional networks and the fixed-size problem

Convolutional neural networks go back to LeCun et al.'s 1989 *Backpropagation applied to handwritten zip code recognition*, which alternated convolution layers with fully connected layers. This is the template of the classic network that the deck revisits for segmentation, and its crucial property for our purpose is stated on the slide as a caveat: **the input and the output are of fixed size**. A network whose classifier head is a fully connected layer can only consume images of the exact resolution it was built for, and it produces a single vector of scores — neither is what dense segmentation needs. (Source: 20260903_segmentation_01.pdf, slide 12)

2.3.3 Maintaining input size: U-Net and fully convolutional networks

The fix is to *use only convolutions or size-preserving operations*. The archetype is the U-Net of Ronneberger, Fischer and Brox, whose diagram the deck reproduces: an input image tile of 572×572 pixels is pushed through a contracting path (repeated `conv 3x3, ReLU` followed by `max pool 2x2`), an expanding path (repeated `up-conv 2x2` concatenated with the *copy and crop* skip connections coming from the contracting path), and a final `conv 1x1` that maps the feature maps to the desired

number of classes; the network outputs a segmentation map of 388×388 pixels, i.e. one class score per pixel. The skip connections restore the fine spatial detail lost to pooling, and because every operation is either a convolution or a size-preserving one, the network can in principle be applied to inputs of different sizes with the same weights — the property that makes the family *fully convolutional*. (Source: `20260903_segmentation_01.pdf`, slide 13)

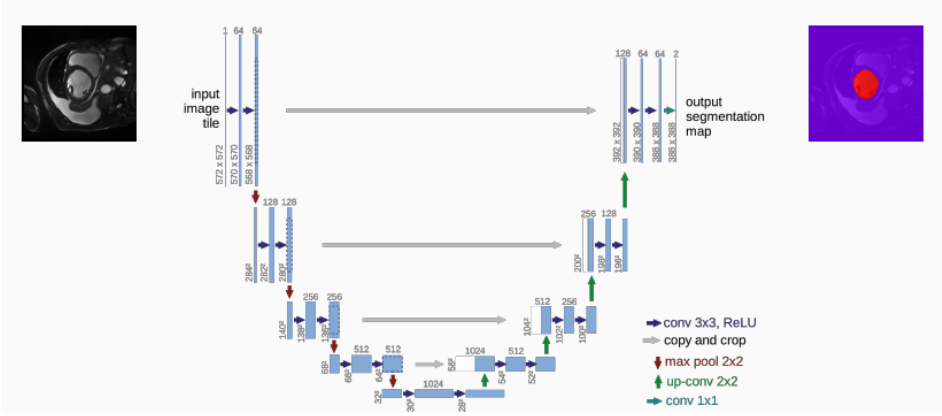


Figure 6: The U-Net architecture: contracting path with `conv 3x3, ReLU` and `max pool 2x2`, expanding path with `up-conv 2x2` and `copy and crop` skip connections, final `conv 1x1`; a 572×572 input tile yields a 388×388 output segmentation map. Source: `20260903_segmentation_01.pdf`, slide 13.

Several architectures exploit the same idea. **FCN** (Long, Shelhamer and Darrell, 2015) introduced fully convolutional networks for semantic segmentation; **DeepLab** (Chen et al., 2017) combined atrous (dilated) convolution with fully connected conditional random fields; **ENet** (Paszke et al., 2016) targeted real-time semantic segmentation by trimming the architecture. (Source: `20260903_segmentation_01.pdf`, slide 14)

2.3.4 Tuning the convolutions

The deck makes the parameter argument explicit by contrasting a LeNet-style classifier with the U-Net. The classifier takes 256 input units, then $12 \times 12 \times 64 = 768$ hidden units in layer H1, $12 \times 16 = 192$ in H2 and 30 in H3, before 10 output units; because the layers are fully connected, H1 already costs about 20 000 links coming from 12 kernels of size 5×5, H2 costs about 40 000 links from another 12 kernels of size $5 \times 5 \times 8$, and the remaining fully connected stages about 6000 and 300 links. The U-Net, by contrast, is built exclusively from `conv 3x3, ReLU`, `max pool 2x2`, `up-conv 2x2` and `conv 1x1`: one small kernel is shared over the whole image, so the number of parameters no longer explodes with the image size, and the spatial layout of the input is preserved throughout instead of being flattened. "Tuning the convolutions" is precisely the act of choosing these architectural levers — kernel size, number of channels, where to pool, where to up-convolve — rather than the width of a dense layer. (Source: `20260910_lecture04_segmentation2.pdf`, slide 10)

2.4 nnU-Net: the current state of the art

nnU-Net (Isensee et al., *Nature Methods* 18(2), 2021) is presented as the current state of the art in biomedical segmentation. Its defining property is that it is a deep learning-based segmentation method that *automatically configures itself*, including preprocessing, network architecture, training and post-processing, for any new task. The key design choices of that process are modelled as a set of *fixed parameters*, *interdependent rules* and *empirical decisions*; the reference implementation is public.[1] (Source: `20260903_segmentation_01.pdf`, slide 15)

[1] `https://github.com/MIC-DKFZ/nnUNet`

The high-level pipeline has four stages, which the course maps onto the components of the teaching code: (1) *pre-processing*, about `slice_segthor.py` (Assignment 1); (2) *data loading*, about `dataset.py`; (3) *forward and backward propagation*, about `main.py` and `losses.py` (Project); and (4) *post-processing* (optional), about `stitching.py` (Assignment 3). The losses used in stage 3 are the subject of Section 3. The dataset statistics that drive the self-configuration (the dataset "fingerprint") are detailed in §2.4.1 below; why the resulting configuration is deterministic is discussed in §7.2.6. The name stands for "*no new* U-Net", and its loss is a fixed parameter of the pipeline, $L = L_{CE} + \lambda L_{DSC}$ (both terms are derived in §3). (Source: `20260903_segmentation_01.pdf`, slide 16; `UvA__AI4MIv0.1.apkg`)

2.4.1 The nnU-Net paper: fixed, rule-based and empirical parameters

Motivation. Configuring a segmentation pipeline depends heavily on dataset properties (modality, size, anisotropic spacing, class ratio) and on hardware, and good configurations rarely transfer between datasets; in practice this is manual trial and error, while AutoML search is far too expensive. nnU-Net's recipe: fix what need not change (*fixed parameters*), derive as much as possible from the dataset fingerprint with heuristic rules (*rule-based parameters*), and learn only the rest from data (*empirical parameters*). (Source: `nnunet paper.pdf`, §1)

Dataset fingerprint. nnU-Net first crops every training case to its non-zero region (which substantially shrinks brain datasets). On the cropped data it computes a fingerprint containing: the *image size* (number of voxels per spatial dimension) before and after cropping; the *image spacing* (physical size of the voxels); the *modalities*, read from the metadata; the *number of classes*; the total *number of training cases*; and the *intensity distribution* of the foreground voxels (voxels belonging to any class), namely its mean, standard deviation, and 0.5 and 99.5 percentiles, computed over all training cases. These are the statistics asked for in quiz Q10. (Source: `nnunet paper.pdf`, Methods)

Rule-based parameters. Heuristic rules map the fingerprint (and a *predefined* GPU memory budget) to the pipeline:

- *Normalisation:* z-score per image for every modality except CT; for CT (quantitative intensities) clip to the foreground 0.5/99.5 percentiles and normalise with the global foreground mean and s.d.
- *Resampling:* to a common target spacing, the per-axis *median* spacing (tenth percentile on the out-of-plane axis for strongly anisotropic data, max/min spacing > 3); images with third-order spline, labels as one-hot channels then arg max.
- *Patch size, topology, batch size:* "initialize the patch size to median image shape and iteratively reduce it while adapting the network topology accordingly until the network can be trained with a batch size of at least two given GPU memory constraints". Memory use is *estimated*, so no GPU is needed to configure.
- *3D cascade:* only if the full-resolution patch covers less than 12.5% of the median image; a low-resolution U-Net's output then guides a full-resolution one.

(Source: `nnunet paper.pdf`, Methods)

Fixed parameters. One plain U-Net template (no residual, dense or attention blocks: "a well-configured plain U-Net is hard to beat"); instance normalisation instead of batch normalisation (batches are tiny), leaky ReLU, deep supervision; SGD with Nesterov momentum 0.99, learning rate 0.01 with poly decay, 1000 epochs of 250 mini-batches; loss = CE + Dice; one third of the patches forced to contain foreground (class imbalance); online augmentation (rotation, scaling, noise, blur, brightness, contrast, gamma, mirroring); sliding-window inference with mirroring test-time augmentation. (Source: `nnunet paper.pdf`, Methods)

Empirical parameters. Decided by five-fold cross-validation: which configuration (2D, 3D full-resolution, 3D cascade) or ensemble of two to use, and whether keeping only the largest connected component improves the foreground Dice. (Source: `nnunet paper.pdf`, Methods)

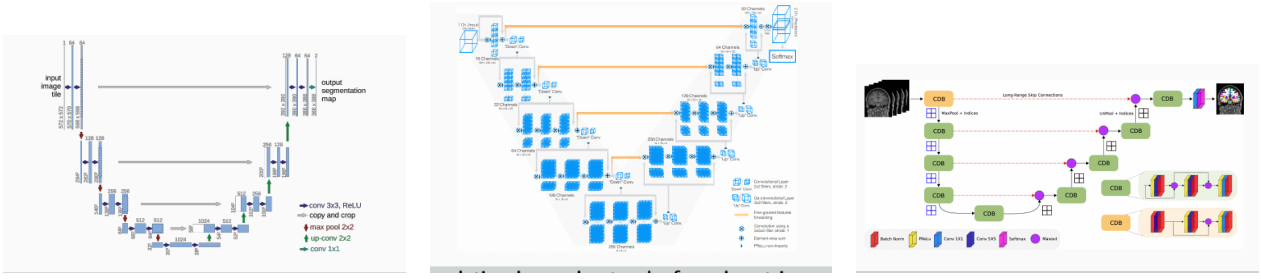
Results and the main lesson. Applied without manual intervention to 11 international challenges (23 datasets, 53 segmentation tasks, covering CT, MRI, electron and fluorescence microscopy), nnU-Net sets a new state of the art on 33 of the 53 target structures and is otherwise on par with or close to the top entries. An analysis of the KiTS 2019 leaderboard (over 100 teams) shows that the top 15 methods were all derived from the 3D U-Net, that methods using the same architecture are spread over the whole leaderboard, and that none of the popular architectural modifications was necessary for good performance: *details of the method configuration matter more than architectural variations.* (Source: `nnunet paper.pdf`, Results)

2.5 Architectures: 2D, 3D and 2.5D

2D. The 2D U-Net processes one slice at a time: a contracting path alternates `conv 3x3, ReLU` blocks with `max pool 2x2`, an expanding path alternates `up-conv 2x2` with `copy and crop` concatenations from the contracting path, and a final `conv 1x1` produces the per-pixel class scores of that slice. Its cost is low and it can be trained on 2D slices, but it cannot use the information contained in neighbouring slices. (Source: `20260910_lecture04_segmentation2.pdf`, slide 7)

3D. The V-Net (Milletari, Navab and Ahmadi, 3DV 2016) is a fully convolutional network for volumetric segmentation: convolutions and upsampling are three-dimensional, so a volume is consumed as a whole and the through-plane context is available. Its blocks are *residual* modules with PReLU activations, downsampling is done by *strided convolution* and upsampling by *deconvolution*, and the network ends in a softmax over classes; the diagram shows the feature maps shrinking from full volume resolution down to a coarse level ($8 \times 8 \times 4$) and being brought back up through the skip connections. The price is memory and computation: a full 3D forward pass is much heavier than its 2D counterpart. (Source: `20260910_lecture04_segmentation2.pdf`, slide 8)

2.5D. FastSurfer (Henschel et al., *NeuroImage* 219, 2020) is a fast and accurate deep learning-based neuroimaging pipeline that takes a middle road between the 2D and the 3D extremes: instead of full three-dimensional convolutions it stacks 2D convolutional blocks — hence the "2.5D" label — with batch normalisation, PReLU and 1×1 convolutions inside the blocks. This keeps the network about as cheap as a 2D network while still being applied to volumetric input. (Source: `20260910_lecture04_segmentation2.pdf`, slide 9)



(a) 2D U-Net (b) 3D V-Net (c) 2.5D FastSurfer

Figure 7: The three architecture families. Source: `20260910_lecture04_segmentation2.pdf`, slides 7, 8 and 9.

2.6 Metadata and preprocessing

A single dataset is rarely uniform: the images of the same dataset can have diverse "spacings"/resolutions, so the physical size of a voxel varies from sample to sample. Such data might require or benefit from *resampling* — upsampling or downsampling — before the network sees it. This is also the

reasoning behind the exam question on variable resolution: if one uses a batch size of 1 and no sub-patching is needed (the GPU can hold a whole sample), one does *not* need to resize all samples to be able to use a fully convolutional network, because there is no fixed-size layer that would impose a common resolution; resampling is then a modelling choice (uniform voxel size, more consistent batching, easier augmentation), not a structural requirement of the architecture. (Source: `20260910_lecture04_segmentation2.pdf`, slide 11; `exams/quiz2024.pdf`, Q2)

2.7 Data augmentation

Training data is augmented by applying *random modifications of the input images*. Formally the augmentation is a non-deterministic function A applied jointly to the image and its label,

$$A(x, y) = (x_a, y_a), \tag{3}$$

whose behaviour is selected by a random draw r over a set of operations:

$$A(x,y) = \begin{cases} \text{flip}(x,y) & \text{if } r = \text{flip} \\ (\text{noise}(x),\ y) & \text{if } r = \text{noise} \\ \text{rotation}(x,y) & \text{if } r = \text{rotation} \\ (x,y) & \text{if } r = \text{id} \end{cases} \qquad r \sim \mathcal{U}\{\text{id}, \text{flip}, \text{noise}, \text{rotation}, \dots\} \tag{4}$$

The cases show which transforms must act on the label too — a flip or a rotation moves the anatomy as well, so it must be applied to the image and the mask together, with the same parameters — whereas noise is applied to the image only, since it modifies the intensities and leaves the mask valid. The identity case leaves the sample untouched. (Source: `20260910_lecture04_segmentation2.pdf`, slide 12)

Augmentation can be applied *offline* or *online*. Given the dataset $D = \{(x_n, y_n)\}_{n=1}^{N}$, the offline variant materialises an enlarged dataset

$$D_a = \bigcup_A \big\{\{A(x_n, y_n)\}_{n=1}^{N}\big\} = \bigcup_A \{A(D)\}, \qquad |D| = N, \quad |D_a| = A \cdot N, \tag{5}$$

so the number of available samples is multiplied by the number of augmentation functions A used, at the cost of storing all of them. The online variant applies the augmentation at each iteration instead: the batch loop becomes $(x_a, y_a) = A(x_n, y_n)$, $s_\theta = \text{softmax}(N(x_a; \theta))$, $L(y_a, s_\theta)$. The advantage is that the network never sees twice the same image, since a fresh random draw is made every epoch; the cost is that the augmentation is recomputed at every iteration rather than once. (Source: `20260910_lecture04_segmentation2.pdf`, slide 14)

A more recent direction replaces generic geometric noise with augmentations that follow the *physics* of the acquisition. VORTEX (*Physics-Driven Data Augmentations Using Consistency Training for Robust Accelerated MRI Reconstruction*, Desai et al., MIDL 2022) augments accelerated MRI data with physics-based perturbations of the acquisition, and trains the network with a *consistency* constraint so that the prediction is stable under those perturbations. The diagram distinguishes a supervised branch — fully sampled k-space, IFFT, network f_θ, reconstruction — from a consistency branch in which an *augmentation operator* is applied and the network outputs at the original and augmented inputs are forced to agree; the lower part of the figure contrasts augmentation performed in image space with augmentation performed in k-space, each paired with the corresponding supervised or unsupervised reconstruction loss. The reconstruction and consistency losses themselves are derived in §8.3, (77). (Source: `20260910_lecture04_segmentation2.pdf`, slide 13)

2.8 Overview of training and sub-patching

In compact notation — derived and used in §3.2 — training solves

$$\min_\theta \ L(y, s_\theta), \tag{6}$$

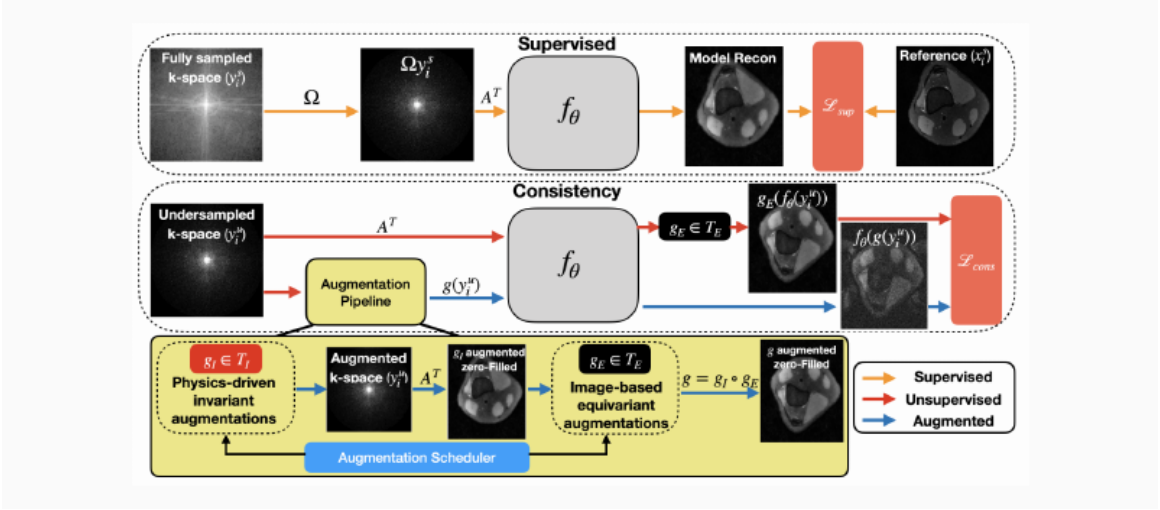


Figure 8: Physics-driven augmentation with consistency training (VORTEX): a supervised branch on fully sampled k-space and a consistency branch where the augmentation operator is applied, with image-space and k-space augmentation variants. Source: `20260910_lecture04_segmentation2.pdf`, slide 13.

where s_θ is the softmax output of the network $N(\cdot;\theta)$ and L is a loss such as the cross-entropy L_{CE} or the Dice loss L_{DSC} (both are treated in Section 3). In practice, with $D = \{(x_n, y_n)\}_{n=1}^{N}$, each step samples a batch $B \sim \mathcal{U}(D) = \{(\breve{x}_b, \breve{y}_b)\}_{b=1}^{B}$ whose elements are *sub-patches* of the samples, $\breve{x}_b \subset x_b$ and $\breve{y}_b \subset y_b$, and accumulates their losses,

$$L = \sum_{b \in B} L(\breve{y}_b, \breve{s}_{\theta,b}), \qquad L_{CE} = \sum_{b \in B} \sum_{k \in \mathcal{K}} \sum_{i \in \breve{\Omega}_b} - y_b^{(i,k)} \log\left(s_{\theta,b}^{(i,k)}\right), \tag{7}$$

before taking an optimisation step

$$\theta_{t+1} = \text{optimizer}\left(\theta_t, \frac{\partial L}{\partial \theta}\right). \tag{8}$$

The essential consequence, stated on the slide, is that *the network never sees the whole picture*: it is trained on small windows of the volume. (Source: `20260910_lecture04_segmentation2.pdf`, slide 15)

Sub-patching is what makes aggressive 3D training feasible, but it has a specific failure mode: many patches can (and will) end up *empty* for some classes — a patch drawn from a region that contains none of the voxels of class k has $\breve{\Omega}_y^{(k)} = \emptyset$ — and one must watch out for this corner case in the losses, whose behaviour on an empty label or prediction is not the one of a generic average. The loss-side treatment of these corner cases is in Section 3. (Source: `20260910_lecture04_segmentation2.pdf`, slide 16)

2.9 Post-processing

2.9.1 Connected-component filtering

The predicted mask can be cleaned up after the network by looking at its topology: labelling the connected components of the prediction (in `scikit-image`, `skimage.measure.label`, possibly per colour/class) gives a set of candidate objects. Depending on the task one then keeps only the components that make sense — for instance keeping the biggest one and setting all the others to background, which removes small spurious blobs. In the deck this step is named non-maxima suppression: it is a *component* filtering step, applied to the already-thresholded map. (Source: `20260910_lecture04_segmentation2.pdf`, slide 17)

2.9.2 DenseCRF (i): what it fixes

A conditional random field is attached to the *softmax* output of the network and used to regularise it with the image itself. The reference is Krähenbühl and Koltun's *Efficient inference in fully connected CRFs with Gaussian edge potentials* (NeurIPS 2011). The slide shows why this is worth doing on a colour photograph: the raw *unary classifiers* (b) produce fragmented, salt-and-pepper masks, whereas a *fully connected CRF* resolves them into the coherent objects of the *image* (a); a robust variant with MCMC inference took 36 hours on that example, while the fully connected CRF of the paper with its efficient (mean-field) inference takes 0.2 seconds. The point for segmentation is the same: the CRF sharpens object boundaries and removes isolated errors left by the per-voxel classifier. (Source: `20260910_lecture04_segmentation2.pdf`, slide 18)

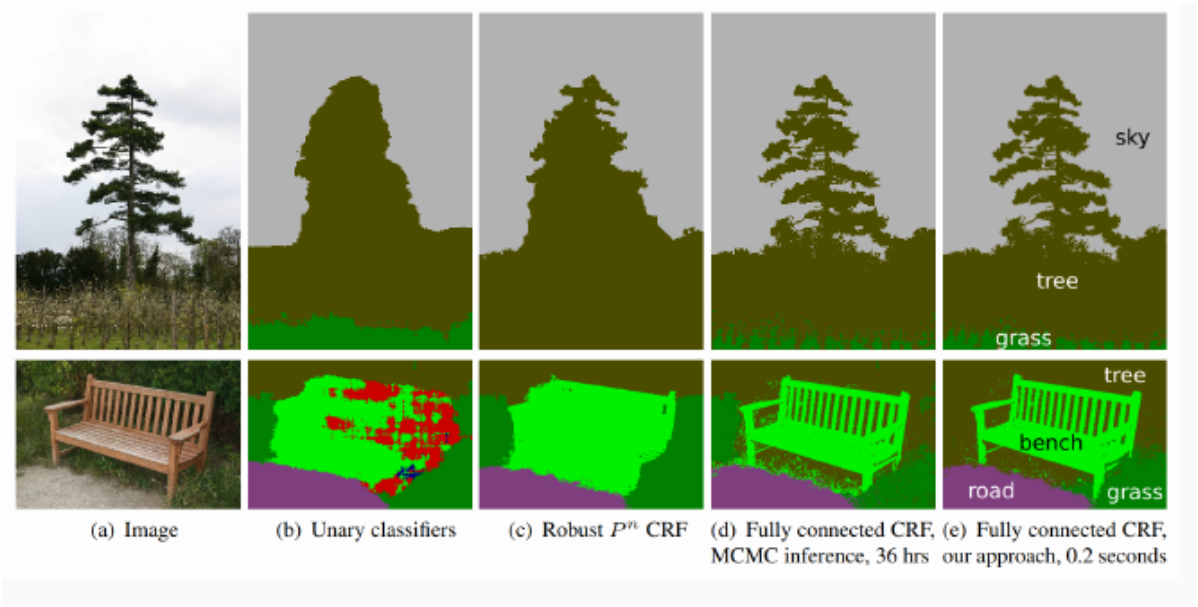


Figure 9: DenseCRF post-processing: (a) image, (b) unary classifiers, (c) robust P^n CRF, (d) fully connected CRF with MCMC inference (36 hrs), (e) fully connected CRF with the efficient approach (0.2 s). Source: `20260910_lecture04_segmentation2.pdf`, slide 18.

2.9.3 DenseCRF (ii): the energy

The regularised labelling is obtained by minimising an energy over the whole labelling S,

$$\min_{S} \sum_{i \in \Omega} \phi_{\text{unary}}\big(S^{(i)}; s_\theta\big) \; + \sum_{i,j \in \Omega^2} \phi_{\text{pairwise}}\big(S^{(i)}, S^{(j)}; X\big), \tag{9}$$

whose unary term penalises disagreement with the network,

$$\phi_{\text{unary}}\big(S^{(i)}; s_\theta\big) := -\log\Big(s_\theta^{\left(p, S^{(i)}\right)}\Big), \tag{10}$$

and whose pairwise term couples *all* pairs of pixels (the CRF is fully connected), weighted by two Gaussian kernels,

$$\phi_{\text{pairwise}}\big(S^{(i)}, S^{(j)}; X\big) := \mathbf{1}_{\left[S^{(i)} = S^{(j)}\right]} \sum_{m=1}^{2} w_m \kappa_m(i,j), \tag{11}$$

with

$$\kappa_1(i,j) := \exp\left(-\frac{|\Omega_i - \Omega_j|^2}{2\omega_\alpha^2} - \frac{|x(i) - x(j)|^2}{2\omega_\beta^2}\right), \qquad \kappa_2(i,j) := \exp\left(-\frac{|\Omega_i - \Omega_j|^2}{2\omega_\gamma^2}\right). \tag{12}$$

Here Ω_i are the pixel coordinates, $x(p)$ the pixel intensity and $w_m \in \mathbb{R}$ are the kernel weights. The first kernel is the *appearance* kernel for local consistency: it is large both when two pixels are spatially close and when they have similar intensities, so it enforces agreement between nearby pixels of similar appearance. The second is the *smoothness* kernel, which depends on position only and removes noise by smoothing the labelling locally. (Source: `20260910_lecture04_segmentation2.pdf`, slide 19)

2.10 Ensembling

Ensembling improves performance and reliability and is popular in challenges: the same network (or variants of it) is trained several times, and the resulting predictions are averaged or otherwise combined. The downside is stated plainly on the slide — it is very costly, since the training and the inference cost are multiplied by the number of members of the ensemble. (Source: `20260910_lecture04_segmentation2.pdf`, slide 20)

2.11 Other approaches

Robot's pathfinding. Segmentation can be recast as a sequential decision problem: Yin et al. (*Left Ventricle Contouring in Cardiac Images Based on Deep Reinforcement Learning*, MIDL 2022) learn a policy that moves along the image and traces the left-ventricle contour, i.e. the contouring task is treated as a robot's pathfinding problem on the image grid. (Source: `20260910_lecture04_segmentation2.pdf`, slide 29)

Boundary loss. Instead of comparing region overlaps, one can compare *boundaries*: the boundary loss (Kervadec et al., *Medical Image Analysis* 2020) minimises the distance between the label boundary $\partial\Omega_y$ and the predicted boundary $\partial\Omega_s$ through the signed distance map ϕ of the label,

$$L_B(y, s_\theta) = \sum_{k \in \mathcal{K}} \sum_i s_\theta^{(i,k)} \, \phi_{\partial\Omega_y}^{(i,k)}, \qquad \frac{\partial L_B}{\partial s_\theta^{(i,k)}} = \phi_{\partial\Omega_y^{(k)}}^{(i,k)}, \tag{13}$$

so that the loss weights each voxel by its signed distance to the true boundary, and its gradient with respect to the predicted probabilities is simply the distance map itself. This is attractive for highly unbalanced segmentation, where region-based losses such as Dice are dominated by the large background class. (Source: `20260910_lecture04_segmentation2.pdf`, slide 30)

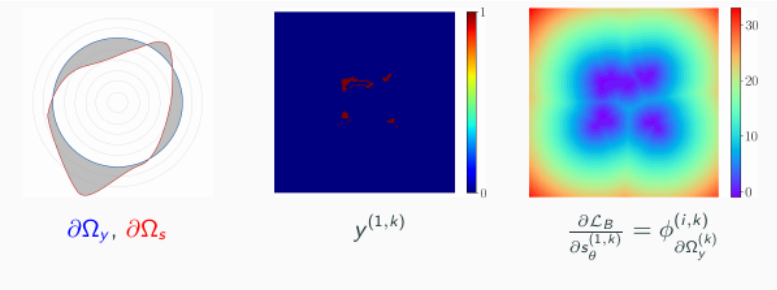


Figure 10: The boundary loss: the label and prediction boundaries $\partial\Omega_y$, $\partial\Omega_s$, the one-hot prediction $y^{(1,k)}$ and the signed distance map $\phi_{\partial\Omega_y}^{(i,k)}$ that weights the loss. Source: `20260910_lecture04_segmentation2.pdf`, slide 30.

Tightness prior for bounding boxes. With weak labels one can exploit structural prior knowledge of the annotation. A bounding box is *tight* around its object by construction, and this tightness can be encoded as a series of losses that compensate for the lack of accurate annotations (Kervadec, Dolz, Wang, Granger and Ben Ayed, *Bounding boxes for weakly supervised segmentation: Global constraints get close to full supervision*, MIDL 2020). (Source: `20260910_lecture04_segmentation2.pdf`, slide 31)

Extreme points with geodesic distances. An even weaker annotation consists of a few extreme points of the object; Dorent et al. (*Inter extreme points geodesics for end-to-end weakly supervised*

image segmentation, MICCAI 2021) connect these extreme points by geodesic paths in the image and use those paths as an end-to-end supervision signal. (Source: `20260910_lecture04_segmentation2.pdf`, slide 32)

2.12 Summary of notation

Throughout the two segmentation lectures the following notation is used, grouped as the deck groups it.

Image and labels. An image element is a *pixel* (picture element) in 2D and a *voxel* (volume element) in 3D, and segmentation is essentially voxel-wise classification. $\Omega \subset \mathbb{R}^D$ is the image space of dimension D; $x \in \mathbb{R}^{M \times \Omega}$ is the input image with M modalities; $\mathcal{K} = \{0 : \text{background}, 1 : \text{class-}1, \ldots, K : \text{class-}K\}$ is the set of $K+1$ classes to predict; $Y \in \mathcal{K}^\Omega$ is the ground truth and $S \in \mathcal{K}^\Omega$ the prediction; θ are the network parameters to "learn". (Source: `20260903_segmentation_01.pdf`, slide 32)

Network output and simplex. $N(x;\theta) \in \mathbb{R}^{(K+1)\times\Omega}$ is the raw output of the neural network; Δ_K denotes the K-simplex (so $K+1$ probabilities drawn in a K-dimensional space) and $\hat{\Delta}_K$ the K-simplex restricted to its vertices, i.e. the one-hot vectors. The three representations of the label and of the prediction are then related by

$$Y \in \mathcal{K}^\Omega \xrightarrow{\text{one-hot}} y \in (\hat{\Delta}_K)^\Omega \subset \{0,1\}^{(K+1)\times\Omega}, \tag{14}$$

$$N(x;\theta) \xrightarrow{\text{softmax layer}} s_\theta \in (\Delta_K)^\Omega \subset [0,1]^{(K+1)\times\Omega} \xrightarrow{\text{argmax}} S \in \mathcal{K}^\Omega \xrightarrow{\text{one-hot}} s \in (\hat{\Delta}_K)^\Omega, \tag{15}$$

where the one-hot representation of the prediction is useful for writing indicator functions. $L(s_\theta, y)$ is some loss, with L_{CE} the cross-entropy loss and L_{DSC} the Dice loss, and $\nabla_\theta = \frac{\partial L(s_\theta, y)}{\partial \theta}$ is the gradient used by the optimiser. (Source: `20260903_segmentation_01.pdf`, slides 33–34; `20260910_lecture04_segmentation2.pdf`, slides 34–35)

Training. γ is the learning rate; $A(x,y) = (x_a, y_a)$ is the (non-deterministic) data augmentation function; $D = \{(x_n, y_n)\}_{n=1}^N$ is the dataset with all annotated samples; $D_a = \cup\{\{A(x_n, y_n)\}_{n=1}^N\}_A$ is the "offline" augmented dataset; $B \sim \mathcal{U}(D) = \{(\breve{x}_b, \breve{y}_b)\}_{b=1}^B$ with $\breve{x}_b \subset x_b$, $\breve{y}_b \subset y_b$ is a batch — a subset of the dataset made of *sub-patches* of samples — and $\breve{\Omega}_b \subset \Omega_b$ is the image-space subset of the sub-patched sample (a 2D slice or a 3D patch). (Source: `20260910_lecture04_segmentation2.pdf`, slide 36)

Weak labels and regularisation. $R(s_\theta)$ denotes an (unsupervised) regulariser and $\text{CRF}(s_\theta; x)$ a CRF conditioned on the input image x; finally, $L(\tilde{y}, \tilde{s}_\theta)$ denotes a loss designed for a weak label $\tilde{y}$. (Source: `20260910_lecture04_segmentation2.pdf`, slide 37)

2.12.1 Worked example of the notation

The handwritten notation sheet runs the whole chain on a toy 2D image, and is the template for exam questions such as "write down the predicted segmentation". (Source: `segmentation_notation.png`)

Image space and indexing. A 2D image of 3×4 pixels has Ω of shape 3×4 (axis 1 runs down the rows, axis 2 along the columns); a 3D scan of $3 \times 4 \times 2$ voxels has $\Omega \subset \mathbb{R}^3$. Elements are accessed with a superscript in parentheses: $s_\theta^{(2)} \in [0,1]^\Omega$ is the prediction map for class 2, and for the pixel $i = (1,2)$, $s_\theta^{(1,i)} = s_\theta^{(1,1,2)} \in [0,1]$ is the predicted probability of class 1 at that pixel. The parentheses matter: s_θ^2 would mean the softmax squared, $s_\theta^{1,1,2}$ is ambiguous, and subscripts such as $s_{\theta,1,1,2}$ are long and messy to combine (e.g. $s_{\theta,1,1,2}\, y_{\theta,1,1,2}$). (Source: `segmentation_notation.png`)

The simplex. For three classes, Δ^2 is the triangle of vectors (a,b,c) with $a,b,c \in [0,1]$ and $a+b+c=1$, for instance its centre $(\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$ or the edge point $(0.5, 0.5, 0)$; its vertices $\hat{\Delta}^2 = \{(1,0,0),(0,1,0),(0,0,1)\}$ are exactly the one-hot vectors. (The sheet writes Δ^K where the slides write Δ_K; they denote the same object.) (Source: `segmentation_notation.png`)

From input to segmentation. The input is a single-modality image ($M = 1$), $x \in \mathbb{R}^{1\times 3\times 4}$, and the classes are $\mathcal{K} = \{0 : \text{background}, 1 : \text{lung}, 2 : \text{trachea}\}$, so $K = 2$ and $|\mathcal{K}| = K + 1 = 3$. The U-Net $\mathcal{N}$ with parameters θ produces the softmax prediction $s_\theta = \mathcal{N}(x;\theta) \in (\Delta^K)^\Omega \subset [0,1]^{(K+1)\times\Omega}$, here a $3 \times 3 \times 4$ tensor (one 3×4 map per class, each pixel's three values summing to one), and the arg max over the class axis gives the predicted segmentation $S \in \mathcal{K}^\Omega$:

$$x = \begin{bmatrix} 0 & 38 & 200 & 233 \\ 255 & 100 & 187 & 221 \\ 21 & 231 & 256 & 0 \end{bmatrix},$$

$$s_\theta = \left[\begin{bmatrix} 1 & 0.1 & 0.8 & 1 \\ 0.9 & 0.1 & 0.3 & 1 \\ 1 & 0.9 & 0.9 & 1 \end{bmatrix}, \begin{bmatrix} 0 & 0.2 & 0.1 & 0 \\ 0.1 & 0.8 & 0.7 & 0 \\ 0 & 0.1 & 0.1 & 0 \end{bmatrix}, \begin{bmatrix} 0 & 0.7 & 0.1 & 0 \\ 0 & 0.1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \right]$$

$$\xmapsto{\arg\max} \quad S = \begin{bmatrix} 0 & 2 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

For example, pixel $(0, 1)$ has probabilities $(0.1, 0.2, 0.7)$, so $S^{(0,1)} = 2$ (trachea), and pixel $(1, 2)$ has $(0.3, 0.7, 0)$, so $S^{(1,2)} = 1$ (lung). (Source: `segmentation_notation.png`)

Label and loss. Here the label $Y \in \mathcal{K}^\Omega$ equals S; its one-hot transform $y \in (\hat{\Delta}^K)^\Omega \subset \{0,1\}^{(K+1)\times\Omega}$ is again a $3 \times 3 \times 4$ tensor:

$$Y = \begin{bmatrix} 0 & 2 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \xmapsto{\text{one-hot}} y = \left[\begin{bmatrix} 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}, \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \right].$$

The loss $L(s_\theta, y)$ then compares s_θ and y. In implementation form the sheet normalises the cross-entropy by $|\Omega| = 12$ and $|\mathcal{K}| = 3$ and adds a small $\varepsilon \approx 10^{-10}$ inside the logarithm, while the Dice term adds ε to numerator and denominator to avoid $0/0$:

$$L_{CE} = \frac{1}{|\Omega|}\frac{1}{|\mathcal{K}|}\sum_{i\in\Omega}\sum_{k\in\mathcal{K}} y^{(k,i)}\big(-\log(s_\theta^{(k,i)} + \varepsilon)\big), \qquad \frac{1}{|\mathcal{K}|}\sum_{k\in\mathcal{K}} \frac{\sum_{i\in\Omega} 2\, s_\theta^{(k,i)} y^{(k,i)} + \varepsilon}{\sum_{i\in\Omega}\big[s_\theta^{(k,i)} + y^{(k,i)}\big] + \varepsilon},$$

the second expression being the mean soft Dice coefficient, of which the Dice loss (24) is the complement $1 - (\cdot)$. Note that the sheet indexes class first, (k, i), while the slides write (i, k). (Source: `segmentation_notation.png`)

3 Segmentation Losses and Their Gradients

This chapter covers the second half of the segmentation material: how a segmentation problem is written down formally, how a network is trained, and what the loss functions used for that training actually are. Its centre of gravity is the pair of losses the course uses throughout — cross-entropy (CE) and the Dice loss — together with the full derivations of their gradients, because those gradients are what the optimiser sees and because the two supervise the network in visibly different ways. It closes with the loss taxonomy of the *Loss odyssey* paper and with the losses used when the annotations are weak.

3.1 Formal definition and notation

Segmentation is the task of predicting a label *for every element of the image domain*, in contrast with classification, which returns one label for the whole image, and with object detection, which returns a set of localised boxes. The domain is $\Omega \subset \mathbb{R}^D$ (dimension $D = 2$ for a 2D slice, $D = 3$ for a volume), the input is $x \in \mathbb{R}^{M\times\Omega}$ with M modalities, the set of classes is $K = \{0 : \text{background}, 1 :$

class-$1, \ldots, K$: class-K}, i.e. $K+1$ classes to predict, and the ground truth and the prediction are $Y \in K^\Omega$ and $S \in K^\Omega$. Here and below Ω denotes both the domain and its (finite) set of voxels, so a sum $i \in \Omega$ runs over all voxels of the volume. (Source: `20260903_segmentation_01.pdf`, slide 18)

The parameter vector of the network is θ and its raw output is $N(x;\theta) \in \mathbb{R}^{(K+1)\times\Omega}$. The ground-truth map Y and the predicted map S are *difficult to work with* directly: they are discrete, and a gradient can neither be taken with respect to the annotation nor through an arg max. The fix is to represent both as vectors of per-class numbers. The annotation becomes a *one-hot* vector at each voxel (a 1 for its class, 0 for the others), and the network output is passed through a *softmax*, which turns the raw scores into probabilities that are non-negative and sum to 1. Geometrically, the set of such probability vectors is called the *simplex*: the one-hot labels sit at its corners (vertices) and the softmax output lies inside it. Only at the very end is the arg max used to get back a discrete label map:

$$\begin{aligned} Y \in K^\Omega &\xrightarrow{\text{one-hot}} y \in (\hat{\Delta}_K)^\Omega \subset \{0,1\}^{(K+1)\times\Omega}, \\ N(x;\theta) &\xrightarrow{\text{softmax}} s_\theta \in (\Delta_K)^\Omega \subset [0,1]^{(K+1)\times\Omega}, \\ s_\theta &\xrightarrow{\arg\max} S \in K^\Omega, \end{aligned} \tag{16}$$

where the *K-simplex* is $\Delta_K = \{z \in [0,1]^{K+1} : \sum_{k\in K} z_k = 1\}$ (the slide draws the 2-simplex for three classes) and $\hat{\Delta}_K = \Delta_K \cap \{0,1\}^{K+1}$ is its set of vertices, one per class. The notation is θ: network parameters to "learn"; $y^{(i,k)} \in \{0,1\}$: label; $s_\theta^{(i,k)} \in [0,1]$: network output (a softmax); $i \in \Omega$: image space; $k \in K$: predicted classes. (Source: `20260903_segmentation_01.pdf`, slides 19 and 22)

3.2 Training as an optimisation problem

Training a segmentation network means repeatedly showing annotated examples to the network and adjusting θ so that the prediction approaches the annotation. Formally this is the optimisation problem

$$\min_\theta \ \mathcal{L}(s_\theta, y), \qquad \begin{aligned} &\theta : \text{network parameters;} \\ &s_\theta : \text{network output;} \\ &y : \text{annotated mask.} \end{aligned} \tag{17}$$

The mask enters the objective in its one-hot form y, and the network through its softmax output s_θ, exactly as in (16). (Source: `20260903_segmentation_01.pdf`, slide 20)

The cost is minimised by back-propagation, which applies the chain rule along the computation graph to obtain the gradient of the loss with respect to every parameter,

$$\nabla_\theta \ = \ \frac{\partial \mathcal{L}(s_\theta, y)}{\partial \theta} \ = \ \frac{\partial \mathcal{L}(s_\theta, y)}{\partial s_\theta} \frac{\partial s_\theta}{\partial l_L} \frac{\partial l_L}{\partial l_{L-1}} \cdots \frac{\partial l_1}{\partial l_0}, \tag{18}$$

where $l_0, \ldots, l_L$ are the successive layers of the network, after which the parameters are updated by

$$\theta_{t+1} = \theta_t - \gamma \nabla_{\theta_t}, \qquad \gamma : \text{ learning rate}, \tag{19}$$

with variants of this rule being stochastic gradient descent (with or without momentum), Adam, and others. (Source: `20260903_segmentation_01.pdf`, slide 21)

Because (18) contains the factor $\partial\mathcal{L}/\partial s_\theta$, a training loss must be *real-valued* (a scalar the optimiser can decrease) and *differentiable with respect to the network output s_θ*; it need not be differentiable with respect to the annotation (the annotation is a constant of the training set, not a variable), and nothing requires the loss values themselves to lie in $[0,1]$ or to be positive. This is the condition asked for by the exam question "what are the condition(s) for a loss function to be usable for training?". (Source: `exams/exam2025.pdf`, §5a)

A loss, then, is simply *a formal way to define our objective*: it is the scalar the optimiser drives down, and its choice dictates how the network is supposed to trade off its mistakes. (Source: `20260903_segmentation_01.pdf`, slide 22)

3.3 Cross-entropy

The first loss is cross-entropy. With a softmax output s_θ and a one-hot annotation y, the deck defines

$$\min_\theta \; \mathcal{L}_{CE}(s_\theta, y) = \sum_{i \in \Omega} \sum_{k \in K} -y^{(i,k)} \log s_\theta^{(i,k)}, \tag{20}$$

so that the loss is a sum of one term per (voxel, class) pair, and only the terms whose class is the annotated one survive the factor $y^{(i,k)} \in \{0, 1\}$: the loss is the negative log-probability the network assigns to the true class of each voxel, i.e. a per-voxel negative log-likelihood. (Source: `20260903_segmentation_01.pdf`, slide 22)

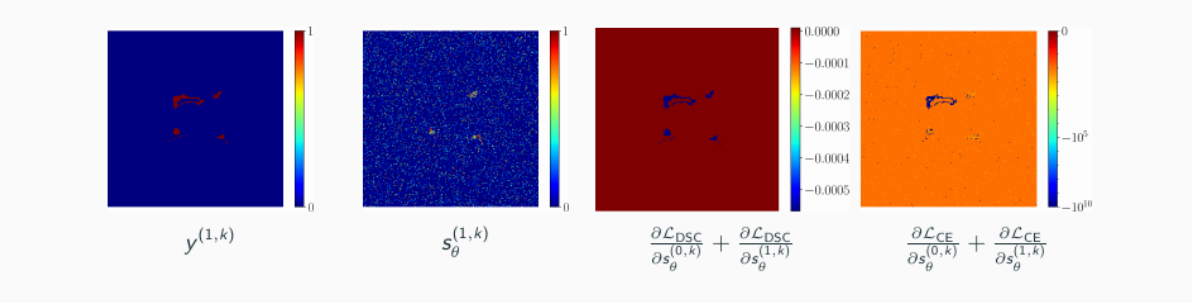


Figure 11: Left to right: the one-hot annotation $y^{(1,k)}$, the predicted probability $s_\theta^{(1,k)}$, and the resulting gradient maps summed over the background and foreground channels, for the Dice loss and for the cross-entropy loss. The Dice gradient is concentrated on the object boundary and vanishes elsewhere, whereas the cross-entropy gradient is non-zero on essentially every voxel of the image. Source: `20260903_segmentation_01.pdf`, slide 29 (same slide in `20260910_lecture04_segmentation2.pdf`, slide 5).

The paper accompanying this material derives the same loss from the Kullback–Leibler divergence between a target distribution P and a predicted distribution Q,

$$D_{KL}(P \| Q) = \sum_i p_i \log \frac{p_i}{q_i} = -\sum_i p_i \log q_i + \sum_i p_i \log p_i = H(P, Q) - H(P), \tag{21}$$

where $H(P, Q) = -\sum_i p_i \log q_i$ is the cross-entropy and $H(P) = -\sum_i p_i \log p_i$ the entropy of P. Since in supervised learning the data distribution P is given by the training set, $H(P)$ is a constant, so minimising the KL divergence is equivalent to minimising the cross-entropy $H(P, Q)$ — which is precisely the quantity in (20). The paper writes the same loss with an explicit normalisation over the N voxels, which only rescales it by $1/N$ (the exam states the loss as $L_{CE}(s_\theta, y) \propto \sum_{i \in \Omega} \sum_{k \in K} y^{(i,k)}(-\log s_\theta^{(i,k)})$, i.e. up to exactly such a constant factor). (Source: `1-s2.0-S1361841521000815-main.pdf`, §2.1.1)

The name of the quantity being minimised comes with a story the course quotes verbatim: *"You should call it entropy, for two reasons. In the first place your uncertainty function has been used in statistical mechanics under that name, so it already has a name. In the second place, and more important, no one really knows what entropy really is, so in a debate you will always have the advantage."* — John von Neumann to Claude Shannon on his new uncertainty function. The reason von Neumann gave for the name is therefore, in his own words, twofold: the function had already been called entropy in statistical mechanics, and nobody really knows what entropy is, which makes it a name one can always defend in a debate. (Source: `20260903_segmentation_01.pdf`, slide 23; `exams/quiz2024.pdf`, Q11)

3.3.1 Gradient of the cross-entropy loss

The partial derivative of (20) with respect to the predicted probability of one voxel and one class is obtained by differentiating term by term. The double sum contains exactly one term that depends

on $s_\theta^{(i,k)}$, namely the term with the same indices (i,k); all other terms are constants with respect to that variable. Hence

$$\frac{\partial \mathcal{L}_{CE}}{\partial s_\theta^{(i,k)}} = -y^{(i,k)} \frac{1}{s_\theta^{(i,k)}} = \begin{cases} -\dfrac{1}{s_\theta^{(i,k)}} & \text{if } y^{(i,k)} = 1, \\ 0 & \text{otherwise.} \end{cases} \tag{22}$$

Two things are worth reading off (22). First, the gradient of a voxel/class pair is *local*: it depends only on $y^{(i,k)}$ and $s_\theta^{(i,k)}$, not on the prediction at any other voxel. Second, it is non-zero only for the annotated class (the term $-1/s_\theta^{(i,k)}$) and *zero for every other class*: cross-entropy trains on all voxels, but each voxel only pushes up the probability of its own class. (Source: `20260910_lecture04_segmentation2.pdf`, slide 5; `exams/exam2025.pdf`, §5b)

The gradient can go towards infinity on the branch $y^{(i,k)} = 1$: as the predicted probability of the *correct* class tends to zero, $s_\theta^{(i,k)} \to 0$, the factor $1/s_\theta^{(i,k)}$ diverges, so $-1/s_\theta^{(i,k)} \to -\infty$; equivalently the loss term $-\log s_\theta^{(i,k)} \to +\infty$. In other words, the gradient blows up when the network places probability zero on a class that the annotation says is present — a confident mistake — and this is exactly the reason the "raw" logarithm is unusable at the extreme: it is the same division by a vanishing quantity that the smooth term ε absorbs in the losses below. (Source: `exams/exam2025.pdf`, §5c)

3.4 The Sørensen–Dice coefficient and the Dice loss

The second loss is built from the overlap of the two masks. For a given class k, let $\Omega_y^{(k)} \subseteq \Omega$ be the subset labelled as k and $\Omega_s^{(k)} \subseteq \Omega$ the subset predicted as k; the Sørensen–Dice coefficient is

$$\mathrm{DSC}(y, s; k) := \frac{2\left|\Omega_y^{(k)} \cap \Omega_s^{(k)}\right|}{\left|\Omega_y^{(k)}\right| + \left|\Omega_s^{(k)}\right|} = \frac{2\sum_{i\in\Omega} y^{(i,k)} s^{(i,k)}}{\sum_{i\in\Omega}\left[y^{(i,k)} + s^{(i,k)}\right]}, \tag{23}$$

where the left form is the discrete-set definition and the right form the indicator-function form; for a probability field $s_\theta \in [0,1]^{(K+1)\times\Omega}$ the same expression is used with s replaced by s_θ. The metric itself — its value range, its biases, and the corner cases of an empty label or an empty prediction — is the subject of the metrics chapter, see §5.3 and, for the over-segmentation bias, §5.3.1. (Source: `20260903_segmentation_01.pdf`, slides 24–25)

To turn a similarity into a loss one subtracts it from one and averages over classes,

$$\mathcal{L}_{DSC}(y, s_\theta) := \frac{1}{|K|} \sum_{k\in K} \left(1 - \frac{2\sum_{i\in\Omega} y^{(i,k)} s_\theta^{(i,k)}}{\sum_{i\in\Omega}\left[y^{(i,k)} + s_\theta^{(i,k)}\right]}\right). \tag{24}$$

Because the Dice loss is defined directly on the predicted *probabilities* s_θ (and not on a thresholded mask), it is differentiable end-to-end and can be averaged over the batch like any other loss. (Source: `20260903_segmentation_01.pdf`, slide 25)

One practical pitfall matters here because the derivation below divides by the denominator. If a class is absent from *both* the annotation and the prediction for a sample — for instance an empty batch entry $y = 0, s = 0$ — then numerator and denominator are both zero and the loss evaluates to $0/0 = \text{nan}$; since the implementation reduces over the batch with a mean, a single such entry makes the loss of the whole batch `nan`. The fix is to add a smoothing constant to the fraction rather than to special-case it,

```
num   = 2 * einsum("bk...,bk...->bk", y, s)
denom = einsum("bk...->bk", y + s) + eps
return (1 - (num + eps) / denom).mean()
```

i.e. a term $\varepsilon > 0$ in the numerator and denominator, which is a smooth operation: no `if`/`else` on the emptiness of the mask is needed, and the loss stays finite and differentiable in the degenerate case. This is the same ε that appears in the Dice term of the exponential logarithmic loss below. (Source: `exams/exam2025.pdf`, §5e)

3.5 Derivation of the Dice-loss gradient

The derivation is written for one class k, since the average over classes in (24) contributes only the constant factor $1/|K|$ to the gradient. Abbreviate the two sums appearing in the k-th term by

$$I^{(k)} = \sum_{i \in \Omega} y^{(i,k)} s_\theta^{(i,k)}, \qquad U^{(k)} = \sum_{i \in \Omega} \left[y^{(i,k)} + s_\theta^{(i,k)} \right], \tag{25}$$

so that the k-th term of (24) is $1 - 2I^{(k)}/U^{(k)}$, whose derivative is the derivative of the quotient $(-2I^{(k)}/U^{(k)})$,

$$\frac{\partial \mathcal{L}_{DSC}}{\partial s_\theta^{(i,k)}} = \frac{\partial}{\partial s_\theta^{(i,k)}} \left(-\frac{2I^{(k)}}{U^{(k)}} \right) = -2 \, \frac{\dfrac{\partial I^{(k)}}{\partial s_\theta^{(i,k)}} U^{(k)} - I^{(k)} \dfrac{\partial U^{(k)}}{\partial s_\theta^{(i,k)}}}{\left(U^{(k)} \right)^2}. \tag{26}$$

(Source: 20260903_segmentation_01.pdf, slide 26)

The two inner derivatives are immediate. Since $\partial s_\theta^{(j,k)} / \partial s_\theta^{(i,k)} = 1$ if $j = i$ and 0 otherwise (the probabilities are independent variables as far as this partial derivative is concerned),

$$\frac{\partial I^{(k)}}{\partial s_\theta^{(i,k)}} = \sum_{j \in \Omega} y^{(j,k)} \frac{\partial s_\theta^{(j,k)}}{\partial s_\theta^{(i,k)}} = y^{(i,k)}, \qquad \frac{\partial U^{(k)}}{\partial s_\theta^{(i,k)}} = \sum_{j \in \Omega} \frac{\partial s_\theta^{(j,k)}}{\partial s_\theta^{(i,k)}} = 1, \tag{27}$$

so that substitution into (26) leaves

$$\frac{\partial \mathcal{L}_{DSC}}{\partial s_\theta^{(i,k)}} = \frac{-2 \left(y^{(i,k)} U^{(k)} - I^{(k)} \right)}{\left(U^{(k)} \right)^2}. \tag{28}$$

(Source: 20260903_segmentation_01.pdf, slide 27)

Finally, the label $y^{(i,k)}$ is binary, so the expression splits into the two cases of the annotation:

$$\frac{\partial \mathcal{L}_{DSC}}{\partial s_\theta^{(i,k)}} = \begin{cases} \dfrac{-2 \left(U^{(k)} - I^{(k)} \right)}{\left(U^{(k)} \right)^2} & \text{if } y^{(i,k)} = 1, \\ \dfrac{2I^{(k)}}{\left(U^{(k)} \right)^2} & \text{otherwise.} \end{cases} \tag{29}$$

(Source: 20260903_segmentation_01.pdf, slide 28)

Both branches have the sign one expects from gradient descent. For a voxel of class k, $I^{(k)} \le U^{(k)}$ so the first branch is negative: increasing $s_\theta^{(i,k)}$ decreases the loss, i.e. it is rewarded, and the reward vanishes only in the degenerate case where the class is absent from both the annotation and the prediction, since $U^{(k)} - I^{(k)} = \sum_{i \in \Omega} \left[1 - (1 - y^{(i,k)})(1 - s_\theta^{(i,k)}) \right] = |\Omega_y^{(k)} \cup \Omega_s^{(k)}|$ stays positive for any non-empty class. For a voxel outside class k the second branch is positive: increasing $s_\theta^{(i,k)}$ is penalised, in proportion to how much overlap the class already has ($I^{(k)}$) and inversely to the squared total mass $U^{(k)}$. Unlike (22), both branches depend on the *global* sums $I^{(k)}$ and $U^{(k)}$ over the whole image and over all voxels of the class, so the Dice gradient is a region-level quantity, not a per-voxel one. (Source: 20260903_segmentation_01.pdf, slides 26–28)

3.6 Combining the two: the popular mix of losses

The loss actually used in the course (and, as the slide puts it, "fixed parameter of nnU-Net") is the sum of the two,

$$\mathcal{L} = \mathcal{L}_{CE} + \lambda \, \mathcal{L}_{DSC}, \qquad \lambda > 0, \tag{30}$$

so that the same gradient machinery applies unchanged: $\nabla_\theta = \partial \mathcal{L} / \partial s_\theta \cdot \partial s_\theta / \partial l_L \cdots \partial l_1 / \partial l_0$ and $\theta_{t+1} = \theta_t - \gamma \nabla_{\theta_t}$, because the gradient of a sum is the sum of the gradients. The slide lists the loss

function alongside the network architecture and the optimizer as the three things nnU-Net fixes for the user; the architecture itself is described in §2.4. (Source: 20260903_segmentation_01.pdf, slide 30)

The paper calls this equal-mixture the *combo loss* and defines it as the sum of cross-entropy and Dice loss, $L_{DiceCE} = L_{CE} + L_{Dice}$ — i.e. the case $\lambda = 1$ of (30), and the combination that the paper's experiments single out as the most robust family. (Source: 1-s2.0-S1361841521000815-main.pdf, §2.4.1)

The two terms pull in different ways, and Figure 11 shows the difference on one slice.

- **Cross-entropy gradient** (22)**: dense and local.** It is non-zero on essentially every voxel and grows like $1/s_\theta$, so it keeps pushing voxels that are already correct as well as those that are not. Each voxel is treated on its own.
- **Dice gradient** (29)**: region-level and focused.** It depends on the global sums $I^{(k)}$ and $U^{(k)}$, and in the figure it is essentially concentrated on the object boundary, where the two branches have opposite signs and correct the mismatch between the masks.

Summing the two, as in (30), gives the network both a dense per-voxel signal and a region-level signal focused on the boundary. Quiz Q5 asks for one of the two key differences; the lecturer's official answer gives both: (i) the Dice gradient is *much smaller* in magnitude, $|\partial \mathcal{L}_{DSC}/\partial s_\theta^{(i,k)}| \ll |\partial \mathcal{L}_{CE}/\partial s_\theta^{(i,k)}|$ (it is divided by $(U^{(k)})^2$); and (ii) the Dice gradient takes *only two different values* over the image per class — one for $y^{(i,k)} = 1$, one for $y^{(i,k)} = 0$, as (29) shows — whereas the CE gradient $-1/s_\theta^{(i,k)}$ varies voxel by voxel. (Source: quiz2024_annotated.pdf, Q5; 20260903_segmentation_01.pdf, slide 29)

3.7 Weak supervision: losses for incomplete annotations

Full voxel-wise masks are the most expensive annotation, so the course also covers training with *weak labels*: the image is annotated by tags, by a dot inside the object, by scribbles, by a bounding box, or by a full mask — a ladder of decreasing annotation effort and increasing ambiguity. (Source: 20260910_lecture04_segmentation2.pdf, slide 22)

Two families of methods deal with such labels. *Proposal-based* methods introduce the segmentation itself as an additional variable to optimise, $\min_{\theta, y_\theta} \mathcal{L}(y_\theta, s_\theta)$, so the optimiser must produce both a pseudo-mask y_θ consistent with the weak annotation and a network s_θ that matches it; the classical example is DeepCut, which segments objects from bounding boxes. *Direct losses* instead keep a single variable and add a regulariser, $\min_\theta \mathcal{L}(\tilde{y}, s_\theta) + \lambda R(s_\theta)$ with $\lambda \in \mathbb{R}$, pushing the network output s_θ to agree with the weak mask $\tilde{y}$ on the little that is known and using R to fill in the rest. (Source: 20260910_lecture04_segmentation2.pdf, slides 23–24)

The canonical direct loss is the *partial cross-entropy*. Let $\tilde{\Omega} \subset \Omega$ be the subset of annotated pixels and $\tilde{y} \in \{0,1\}^{\tilde{\Omega}} \subset (\hat{\Delta}_K)^{\tilde{\Omega}}$ the weak mask; the slide writes $\tilde{\Delta}_K = \hat{\Delta}_K \cup 0$ for the "relaxed" simplex, i.e. the annotation may now also be the all-zero vector "unknown" in addition to the one-hot vertices. The loss is then (20) restricted to the annotated voxels,

$$\tilde{\mathcal{L}}_{CE}(\tilde{y}, s_\theta) = \sum_{i \in \tilde{\Omega}} \sum_{k \in K} -y^{(i,k)} \log s_\theta^{(i,k)} = \sum_{i \in \Omega} \sum_{k \in K} -\tilde{y}^{(i,k)} \log s_\theta^{(i,k)}, \tag{31}$$

where the second form extends $\tilde{y}$ by zero outside $\tilde{\Omega}$, so that the unlabelled voxels contribute nothing to the gradient. (Source: 20260910_lecture04_segmentation2.pdf, slide 25)

Because the partial cross-entropy leaves the unannotated region entirely free, it is combined with a regulariser carrying the image information: a conditional random field (CRF) term,

$$\min_\theta \ \tilde{\mathcal{L}}_{CE}(\tilde{y}, s_\theta) + \lambda \, \mathrm{CRF}(s_\theta; x), \tag{32}$$

which penalises predictions that are not consistent with the image (this is the "regularized losses for weakly-supervised CNN segmentation" line of work). A CRF on the network output can be implemented as a recurrent network, "conditional random fields as recurrent neural networks". (Source: 20260910_lecture04_segmentation2.pdf, slides 26–27)

3.8 Taxonomy of segmentation losses

The *Loss odyssey* paper reviews twenty plug-and-play losses (architecture-agnostic) and sorts them by *how they are derived*: mismatch in *distribution*, in *region*, in *boundary*, or a *compound* of these. Most distribution-based losses are variants of cross-entropy and most region-based losses are variants of Dice. Below, $\Sigma \equiv \sum_{c=1}^{C} \sum_{i=1}^{N}$, products such as gs stand for $g_i^c s_i^c$ (ground truth g, prediction s), and ε is a smoothing constant. (Source: `1-s2.0-S1361841521000815-main.pdf`, §2)

Category	Loss	Definition / idea
distribution	cross entropy (CE)	$-\frac{1}{N}\,\Sigma\, g \log s$
distribution	weighted CE	$-\frac{1}{N}\,\Sigma\, w_c\, g \log s$, w_c = inverse class frequency
distribution	TopK	CE restricted to the k% worst voxels
distribution	focal	$-\frac{1}{N}\,\Sigma\,(1-s)^{\gamma}\, g \log s$ (down-weights easy voxels)
region	Dice	$1 - \dfrac{2\,\Sigma\, gs}{\Sigma\, g + \Sigma\, s}$
region	IoU (soft Jaccard)	$1 - \dfrac{\Sigma\, gs}{\Sigma\,(g + s - gs)}$
region	Tversky	$1 - \dfrac{\Sigma\, gs}{\Sigma\, gs + \alpha\,\Sigma\,(1-g)s + \beta\,\Sigma\, g(1-s)}$ (FP and FN weighted separatel
region	generalized Dice	class weights $w_c = 1/(\sum_i g_i^c)^2$
boundary	boundary (BD)	see (33)
boundary	Hausdorff (HD)	see (34)
compound	DiceCE, DiceTopK, DiceFocal, …	sum of a region loss and another loss

Table 4: The loss families of the *Loss odyssey* review (selection). Dice is the special case $\alpha = \beta = 0.5$ of Tversky. Source: `1-s2.0-S1361841521000815-main.pdf`, §2.1–§2.4.

Boundary-based losses. Minimising a boundary distance directly is non-differentiable or intractable, so both losses use an integral, region-based approximation. The boundary loss weights the prediction by the level-set (signed distance) function φ_G of the ground-truth boundary,

$$L_{BD} = \int_{\Omega} \varphi_G(p)\, s_\theta(p)\, dp, \tag{33}$$

and the Hausdorff loss weights the error by the squared distance transforms d_G, d_S of ground truth and prediction,

$$L_{HD} = \frac{1}{N} \sum_{c=1}^{C} \sum_{i=1}^{N} (s_i^c - g_i^c) \circ \left(d_{G_i^c}^2 + d_{S_i^c}^2 \right). \tag{34}$$

Both must be coupled with a region loss such as Dice to keep training stable, and both are expensive because distance transforms must be recomputed per sample and class at every iteration (distance transforms: §6). (Source: `1-s2.0-S1361841521000815-main.pdf`, §2.3, §5.3)

One mismatched region, three weightings. With $\Delta M = (S \setminus G) \cup (G \setminus S)$ the mismatched region and D_G, D_S distance maps, in binary segmentation

$$L_{Dice} = \frac{|\Delta M|}{|G| + |S|}, \qquad L_{BD} \approx \int_{\Delta M} D_G(p)\, dp, \qquad L_{HD} \approx \frac{1}{|\Omega|} \sum_{\Omega} \Delta M \circ (D_G + D_S) :$$

all three minimise the same ΔM, and differ only in its weight — the object sizes for Dice, the ground-truth distance map for BD, both distance maps for HD. (Source: `1-s2.0-S1361841521000815-main.pdf`, §2.6)

Empirical findings. On four 3D tasks (liver, liver tumour, pancreas, multi-organ; background:foreground ratios from 42:1 to 1243:1), ranked with the Decathlon's Wilcoxon-based scheme, *no loss is best on every task*, but *compound losses are the most robust*: DiceTopK, DiceFocal, DiceHD and DiceBD lead, and among single losses IoU, Dice and CE are the top three, while weighted CE ranks last. Generalized Dice works for binary but fails for multi-class; TopK and focal loss focus on hard voxels and can therefore learn from mislabelled ones. Four of the top five KiTS 2019 teams used DiceCE or DiceTopK. The recommendation: *use a Dice-related compound loss* — which is what nnU-Net does (30). (Source: `1-s2.0-S1361841521000815-main.pdf`, §4–§6)

4 Registration

Everything seen so far assumed that a scan is simply an array of intensities living on a grid: the segmentation lecture ended with feature maps $x \in \mathbb{R}^{M\times\Omega}$ mapped by a network to a segmentation, $x \mapsto {}^{N_\theta}s_\theta \in \triangle^{K\times\Omega}$, where M counts the modalities or channels ($M = 1$ for a grayscale image, $M = 3$ for an RGB image). Registration starts from a different question: two such scans of the same patient—taken at different times, with different modalities, or one of them being an atlas—usually do not sit in the same place, so before any voxelwise comparison or segmentation transfer one has to *align* them, i.e. overlap them in a single coordinate frame. The moving image $\mathcal{M}$ is the one that will be displaced; the fixed image $\mathcal{F}$ is the reference frame. (Source: `20260908_registration.pdf`, slides 5–7)

4.1 The registration problem

Formally, one image lives on the grid of the other: $\mathcal{F} \in \mathbb{R}^{\Omega_1}$ is the fixed image and $\mathcal{M} \in \mathbb{R}^{\Omega_2}$ the moving image. Registration searches for a transformation

$$\mathcal{T} : \Omega_2 \to \Omega_1, \qquad \mathcal{T}(\mathcal{M}) \text{ is the moving image resampled on } \Omega_1, \tag{35}$$

and the best transformation is the one that minimises a dissimilarity between the fixed image and the transformed moving image,

$$\min_{\mathcal{T}} \ \mathcal{L}\big(\mathcal{F},\ \mathcal{T}(\mathcal{M})\big). \tag{36}$$

The loss must be a mapping $\mathcal{L} : \Omega_1 \times \Omega_1 \to \mathbb{R}$ that answers the question "how good are the images aligned?". Notice that the objective is written in terms of the pair $(\mathcal{F}, \mathcal{M})$, so it is the transformation, not a set of weights, that carries the parameters to be optimised at this point. (Source: `20260908_registration.pdf`, slides 8–9)

4.2 Similarity and dissimilarity losses

The simplest dissimilarity is the sum of squared intensity differences, i.e. the squared L_2 norm of the residual image,

$$\mathcal{L}_2\big(\mathcal{F}, \mathcal{T}(\mathcal{M})\big) = \sum_{i\in\Omega_1} \Big|\mathcal{F}(i) - \mathcal{T}(\mathcal{M})(i)\Big|^2, \tag{37}$$

which is small exactly when corresponding voxels have similar intensities. This is the right criterion when both images come from the same modality, because equal intensities then mean the same tissue. Across modalities it breaks down: the same anatomical structure can be bone-white in CT and dark in MRI, so an intensity difference is not meaningful and the minimum of (37) is not the correct alignment. The remedy is to measure not how similar the intensities are, but how much *information* the two images share. Mutual information compares the joint intensity distribution p of the two images with the product of their marginals,

$$\mathrm{MI}\big(\mathcal{F}, \mathcal{T}(\mathcal{M})\big) = \sum_{i,j\in\Omega_1} p\big(\mathcal{F}(i), \mathcal{T}(\mathcal{M})(j)\big) \log \frac{p\big(\mathcal{F}(i), \mathcal{T}(\mathcal{M})(j)\big)}{p_{\mathcal{F}}\big(\mathcal{F}(i)\big)\, p_{\mathcal{T}(\mathcal{M})}\big(\mathcal{T}(\mathcal{M})(j)\big)}, \tag{38}$$

which is large when knowing the intensity of one image tells us the intensity of the other. Here p is the joint histogram over pairs of intensity values, and $p_{\mathcal{F}}$, $p_{\mathcal{T}(\mathcal{M})}$ are its marginals. Equivalently,

$$\mathrm{MI}\big(\mathcal{F}, \mathcal{T}(\mathcal{M})\big) = H(\mathcal{F}) + H\big(\mathcal{T}(\mathcal{M})\big) - \mathrm{CE}\big(\mathcal{F}, \mathcal{T}(\mathcal{M})\big), \tag{39}$$

where the cross-entropy is taken between the two intensity distributions, not between a prediction and a label as in §3.3, although it carries the same symbol. Reading it: the two entropies are the uncertainty of each image alone, the cross-entropy measures how badly one image predicts the other, and their difference is the uncertainty removed once the other image is known, which is largest when the images are aligned. Because MI only needs a statistical dependency between intensities, it survives the CT–MRI or PET–MRI setting where SSD (37) fails. (Source: `20260908_registration.pdf`, slides 9–10)

The deck compares the standard choices side by side: SSD assumes an identity relationship between the intensities of the two images, is not robust to intensity changes and is therefore best for same-modality pairs such as CT–CT; NCC (normalised cross-correlation) assumes a linear relationship, is robust to global brightness changes and suits same-modality images with such variations; NGF (normalised gradient fields) assumes the gradient directions match, robustly, and suits multi-modal images when edges need to be aligned; MI assumes the images share statistical dependencies, is robust and is the tool of choice for multi-modal pairs such as CT–MRI or PET–MRI; MIND (modality independent neighbourhood descriptor) assumes self-similarity patterns are preserved and is designed for highly different modalities. Table 5 reproduces this comparison. (Source: `20260908_registration.pdf`, slide 11)

Table 5: Similarity measures for registration, by the assumption they make about the relationship between the two images. Source: `20260908_registration.pdf`, slide 11.

Loss	Assumption	Robust to intensity changes	Best for
SSD (sum of squared differences)	identity relationship between intensities	no	same-modality images (e.g. CT–CT)
NCC (normalised cross-correlation)	linear relationship between intensities	yes (to global changes)	same-modality images with brightness variations
NGF (normalised gradient fields)	gradient directions match between images	yes	multi-modal images with edge alignment
MI (mutual information)	images share statistical dependencies	yes	multi-modal images (e.g. CT–MRI, PET–MRI)
MIND (modality independent neighbourhood descriptor)	self-similarity patterns are preserved	yes	highly different modalities

4.3 Coordinates and the affine matrix

A transformation is only meaningful once the frames it moves between are fixed. Three frames are involved: *world* coordinates (the physical space of the scanner room), *patient* coordinates (an anatomical frame attached to the subject), and *image* coordinates (the integer voxel grid the network sees); see Figure 12. The object that bridges the last two is the affine matrix of the scan, stored in the metadata of the file (e.g. NIfTI or DICOM) (Source: `UvA__AI4MIv0.1.apkg`): it lets us go from image coordinates to patient or world coordinates,

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \underbrace{\begin{pmatrix} a_{11} & a_{12} & a_{13} & t_x \\ a_{21} & a_{22} & a_{23} & t_y \\ a_{31} & a_{32} & a_{33} & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}}_{A} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}, \tag{40}$$

where the fourth row and the appended 1 implement the homogeneous-coordinate trick that turns an affine map into a matrix product. In other words, A records where the scan sits in the real world and how it is oriented there—the position and the rotation of the patient in physical space—which is precisely the information the scan carries about its own placement. (Source: `20260908_registration.pdf`, slides 12–13. Addresses exam §8a.)

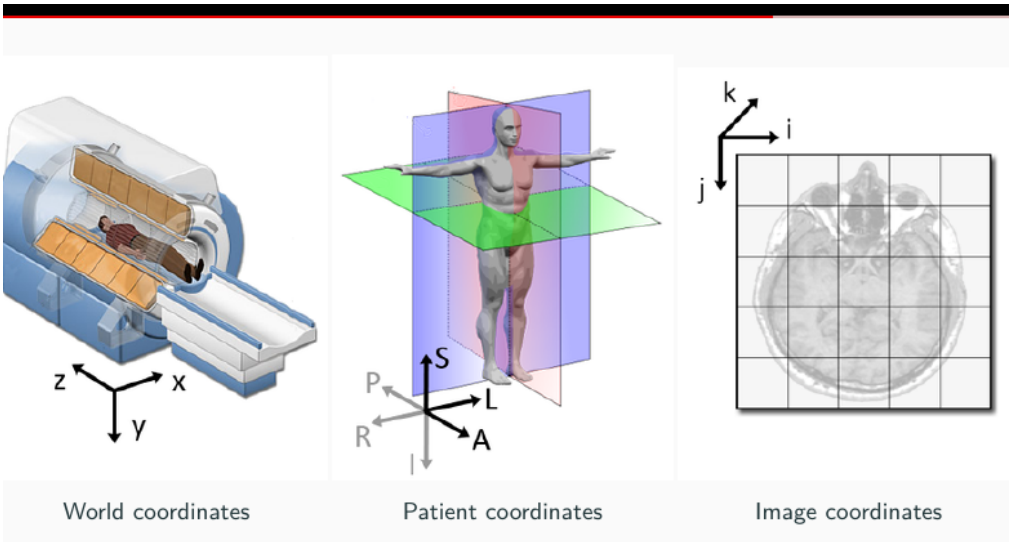


Figure 12: The three coordinate frames of a scan: world, patient and image coordinates; the affine matrix of the scan converts between the image frame and the physical (patient/world) frames. Source: `20260908_registration.pdf`, slide 12.

The affine matrix is convenient because each of its components has a name and a role, and each can be written as its own 4×4 matrix. Translation moves the image without changing its shape,

$$T := \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}, \tag{41}$$

scaling stretches each axis independently,

$$S := \begin{pmatrix} a & 0 & 0 & 0 \\ 0 & b & 0 & 0 \\ 0 & 0 & c & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \tag{42}$$

which is also a way to *encode* the pixel resolution (a voxel of $0.7 \times 0.7 \times 3\,\text{mm}$ is a diagonal scaling of a unit-spacing grid), rotation turns the volume about the z axis,

$$R := \begin{pmatrix} \cos(\alpha) & -\sin(\alpha) & 0 & 0 \\ \sin(\alpha) & \cos(\alpha) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \tag{43}$$

and shear slants the grid along the axes,

$$C := \begin{pmatrix} 1 & c_x & 0 & 0 \\ c_y & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}. \tag{44}$$

Composing these primitives as $A = C \cdot R \cdot S \cdot T$—shear, rotation and scaling, then translation, applied to a point in that right-to-left order—reproduces the general affine matrix,

$$A = C \cdot R \cdot S \cdot T, \tag{45}$$

so the six or more numbers of A can be read as "how much translation, scaling, rotation and shear" the transformation contains. (Source: `20260908_registration.pdf`, slides 14–18)

Registration with an affine transformation is then nothing but optimisation of those numbers: writing $\mathcal{T}_A$ for the transformation induced by A, the problem (36) becomes

$$\min_A \ \mathcal{L}\big(\mathcal{F}, \mathcal{T}_A(\mathcal{M})\big), \tag{46}$$

where the optimiser searches the entries of A (the standard implementations of such resamplings, e.g. `scipy.ndimage.affine_transform`, take exactly this matrix as input). This family (rigid, plus scaling and shear) is used for a broad range of alignment tasks shown in the deck: comparing a patient over time, aligning different modalities of the same anatomy, atlas-to-patient alignment, following knee cartilage in dGEMRIC MRI (Bron et al., 2013), and fusing the histology of a carotid plaque with in-vivo CTA and ex-vivo μCT/μMRI (Groen et al., 2010; a 3D chain CTA–μCT–μMRI plus a 2D registration of the histology slice). (Source: `20260908_registration.pdf`, slides 18–23)

4.4 Elastic (non-rigid) registration

Everything above is rigid, and rigidity is a good model for bones and for organs that deform little: a skull or a femur moves as a whole and never bends. For the remaining anatomy it is wrong. If the patient breathes, if the heart beats, if the tissue is cut and pressed on a slide, the two images are related by a spatially varying displacement rather than by a single matrix, and one must predict a *deformation field*

$$i \in \Omega_2 \ \mapsto \ \mathbb{R}^3, \tag{47}$$

i.e. a three-component displacement for every voxel of the moving image, instead of a handful of matrix entries. This is what elastic (non-rigid, deformable) registration does; Figure 13 shows a fixed image, a deformed moving image, and the deformation grid that produced it. (Source: `20260908_registration.pdf`, slides 24–25)



Figure 13: Non-rigid registration predicts a deformation field per voxel: fixed image, deformed moving image and the corresponding deformation grid. Source: `20260908_registration.pdf`, slide 26.

The price of that freedom is that the problem becomes *ill-posed*: with one displacement per voxel there are vastly more unknowns than the images provide constraints, so very different deformation fields explain the observed alignment equally well (the slide shows two different deformations, both of which map the moving image onto the same fixed image). The standard fix is to add a regulariser to the objective,

$$\min_{\mathcal{T}} \ \mathcal{L}\big(\mathcal{F}, \mathcal{T}(\mathcal{M})\big) + \lambda\, \mathcal{R}(\mathcal{T}), \tag{48}$$

where $\mathcal{R}$ penalises implausible deformations—typically it encourages neighbouring pixels or voxels to move in a similar direction, as in the diffusion and curvature regularisers—with λ trading data fit

against regularity. This is the same data term as (36), so the loss from Section 4.2 is reused unchanged. Regularisation mainly fights deformations that fold onto themselves; the practical toolboxes for this setting, e.g. `elastix` (Klein, Staring, Murphy, Viergever and Pluim, IEEE TMI 2010), expose many optimisers, options and strategies because no single one dominates. (Source: `20260908_registration.pdf`, slides 26–28)

Two structural strategies reduce the difficulty. The first is to *decompose the problem into sub-steps*: instead of solving one very ill-posed deformable problem, one first aligns the centres of mass of the two volumes, then estimates a translation, then a rigid/affine transformation, and only then a deformable field—each step initialising the next, so that the last, hardest step starts from a nearly aligned pair; Figure 14 shows such a chain on a whole-body scan. The second is the *pyramidal* (multi-resolution) approach: the transformation is estimated on heavily downsampled versions of both images, where only the gross misalignment matters, and the result is then used to initialise the estimate at a finer resolution, repeating until the original resolution is reached; coarse-to-fine avoids the optimiser being trapped by the local structure of the image and speeds up the early iterations. (Source: `20260908_registration.pdf`, slides 29–30)

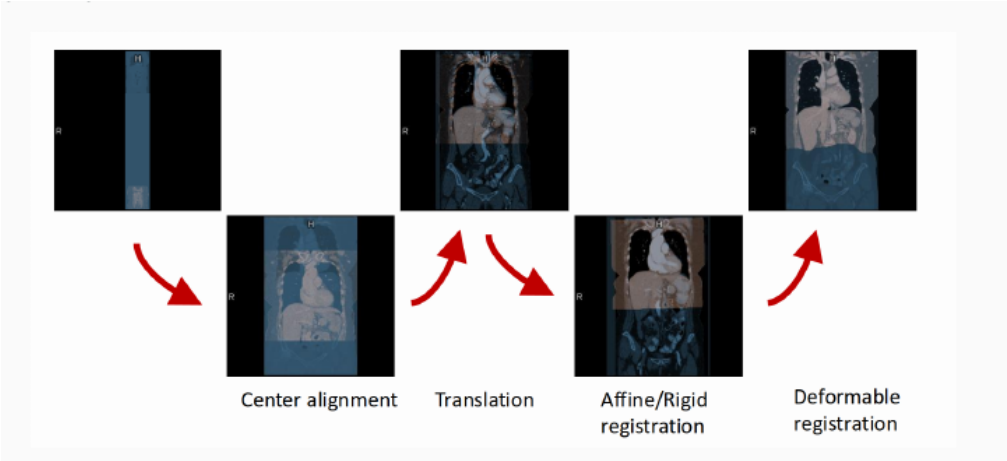


Figure 14: Decomposing registration into sub-steps: centre-of-mass alignment, then translation, then affine, then deformable registration, each initialising the next. Source: `20260908_registration.pdf`, slide 29 (figure by Alessa Hering).

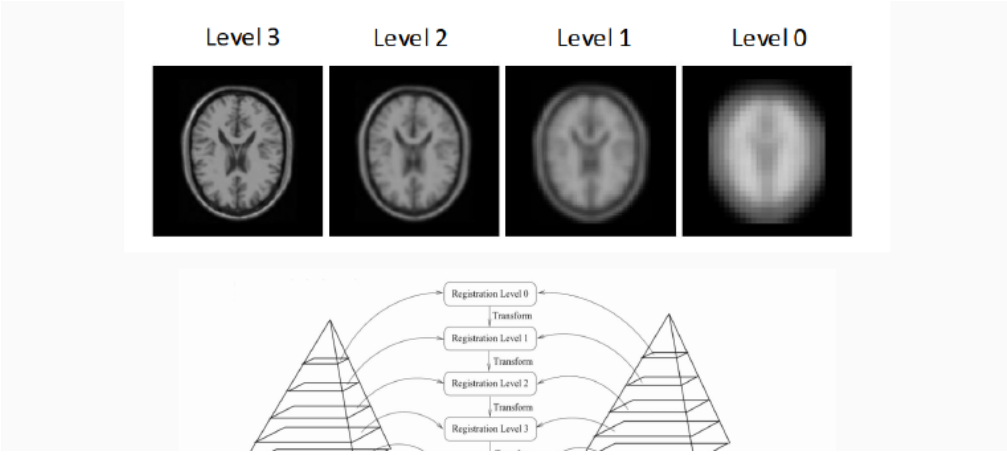


Figure 15: The pyramidal approach: the transformation is estimated coarse-to-fine, each level initialising the next. Source: `20260908_registration.pdf`, slide 30.

4.5 Learned registration

Classical registration needs to run an optimiser for each pair of images, which is expensive when thousands of pairs have to be aligned; the natural question is whether a network can be trained to do it instead. VoxelMorph (Balakrishnan, Zhao, Sabuncu, Guttag and Dalca, IEEE TMI 2019) answers yes, and keeps the U-Net architecture: a U-Net g_θ takes the pair $(\mathcal{F}, \mathcal{M})$ and outputs a registration field ϕ,

$$g_\theta(\mathcal{F}, \mathcal{M}) = \phi, \qquad x \mapsto {}^{R_\theta}t_\theta \in \mathbb{R}^{3\times\Omega}, \tag{49}$$

i.e. the registration network R_θ predicts the transformation t_θ as three displacement channels over the voxel grid Ω—the same notation pattern as the segmentation network N_θ predicting s_θ over K classes, with the output now living in $\mathbb{R}^{3\times\Omega}$ rather than in the simplex. A differentiable spatial transform layer then warps the moving image, $\mathcal{M} \circ \phi$, and the network is trained with

$$\mathcal{L}_{sim}\big(\mathcal{F}, \mathcal{M} \circ \phi\big) + \mathcal{L}_{smooth}(\phi), \tag{50}$$

a dissimilarity between the fixed image and the warped moving image plus a regulariser on the field. This is exactly the regularised objective (48) with the transformation predicted by a network; Figure 16 shows the architecture, the spatial transform and the two loss terms. (Source: `20260908_registration.pdf`, slides 31–33)

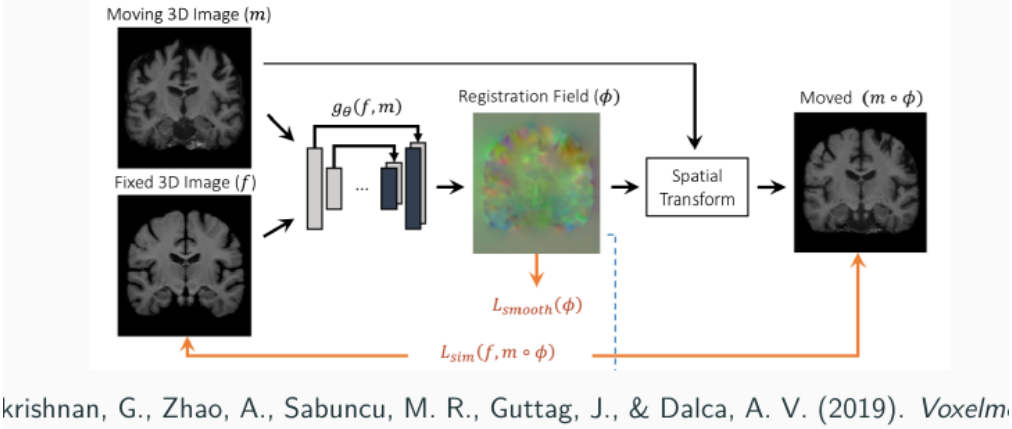


Figure 16: VoxelMorph: a U-Net $g_\theta(\mathcal{F}, \mathcal{M})$ predicts the registration field ϕ, a spatial transform warps the moving image, and training minimises $\mathcal{L}_{sim}$ together with the smoothness term $\mathcal{L}_{smooth}(\phi)$. Source: `20260908_registration.pdf`, slide 33.

The key observation is that no ground-truth labels are needed. The same objective function is simply summed over a dataset of *pairs*,

$$\mathcal{D} := \big(\mathcal{F}_n, \mathcal{M}_n\big)_{n=1}^{N}, \qquad \min_\theta \sum_{n\in\mathcal{D}} \mathcal{L}\big(\mathcal{F}_n,\, t_\theta(\mathcal{M}_n)\big), \tag{51}$$

so supervision comes from the images themselves: the network is trained to make the warped moving image similar to the fixed image, and the smoothness term supplies the rest of the constraint. Training labels for VoxelMorph are therefore *none*—it is an unsupervised (self-supervised) method, in contrast to segmentation, which needs a mask for every training image. (Source: `20260908_registration.pdf`, slide 34)

Extra regularisers can be added when more information is available. If the fixed and moving images come with segmentations, s_f and s_m, one can require that the segmentation of the moving image, after the spatial transform, matches the segmentation of the fixed image,

$$\mathcal{L}_{seg}\big(s_f,\ s_m \circ \phi\big), \tag{52}$$

which is exactly the explicit statement that the segmentation must match after transformation, and may also be used with other landmarks. Alternatively a non-fold constraint can be pushed into the

loss: the penalised quantity must remain positive so that the deformation does not fold over itself, and the deck compares three penalty functions with this property—the interior-point method of Rühaak et al. (2017), a log-barrier extension, and a proposed penalty—plotting the penalty against the penalised quantity z, all of them exploding on the wrong side of $z = 0$. The slide states the requirement as: the curvature regulariser $\mathcal{R}$ must be greater than 0, and ideally close to 1. (Source: `20260908_registration.pdf`, slides 35–36)

The multi-resolution strategy of Section 4.4 transfers to the learned setting as well, e.g. in the multilevel variational image registration network mlvirnet (Hering, Van Ginneken and Heldmann, MICCAI 2019), which performs the estimation level by level inside the network. As a benchmark for such methods, the Learn2Reg challenge collects registration tasks and datasets on which the learned and the classical approaches are compared. (Source: `20260908_registration.pdf`, slides 37–38)

4.6 Rigid or non-rigid? Choosing by anatomy, and the trade-offs

The deck's rule of thumb: rigid registration is a good model for bones and for organs that deform little, while anything that moves or deforms during or between acquisitions needs a deformation field. Applied organ by organ, this settles the typical exam cases (Table 6).

Table 6: Rigid or non-rigid registration, by organ. Source: `20260908_registration.pdf`, slides 25–26; rows checked against the lecturer's model answer to quiz Q6 in `quiz2024_annotated.pdf`.

Organ	Transformation	Why
Skull, femur	rigid / affine	Bones cannot deform, so the only difference between scans is position and orientation, which the matrix (40) and its decomposition (45) absorb.
Ribs	either accepted	Each rib is a rigid bone, but the rib cage as a whole moves and changes shape with breathing.
Lungs	non-rigid	They inflate and deflate with breathing and are compressible by the rib cage.
Heart	non-rigid	It contracts and relaxes through the cardiac cycle.
Kidneys	non-rigid	Soft abdominal organ that shifts and deforms with breathing, posture and the surrounding organs.

The time between scans matters as much as the organ. In quiz Q6 the scans are one week apart in an adult, so bones are unchanged and rigid registration suffices for them. In exam §8b the scans are *years* apart in patients aged 13 to 25, i.e. during growth: bones such as the femur and skull change size and shape, so even they need at least an affine transform with scaling, and in general a non-rigid one — a kid is not just a small adult (Table 8). This is a derivation from the deck's rule, not a model answer. (Source: `quiz2024_annotated.pdf`, Q6; `exams/exam2025.pdf`, §8b; `20260917_lecture06_ethics.pdf`)

Learned registration beats the classical one on exactly one axis: the classical approach must run an optimiser for every pair of images, whereas a trained network produces the transformation for a new pair in a single forward pass, having amortised the search over the training pairs (51). The counterpart is its dependence on data: the network's quality is bounded by the pairs it was trained on, and its regularisation and smoothness terms are learned rather than fitted per case, while the classical optimiser attacks the objective of the very pair in front of it, at the cost of that per-pair optimisation. (Source: `20260908_registration.pdf`, slides 32–34. Addresses exam §8c.) The flashcards condense this into one line each: classical (optimiser-based) registration gives the best, correct result but is potentially very slow; learned registration has fast inference but does not guarantee correctness at inference. (Source: `UvA__AI4MIv0.1.apkg`)

Finally, the changes needed to convert a segmentation U-Net into a registration network are minimal, precisely because the two tasks share the encoder–decoder backbone. The output layer must produce three channels instead of K class scores, so that the network emits the displacement field $t_\theta \in \mathbb{R}^{3\times\Omega}$ of (49) rather than class probabilities in the simplex; a differentiable spatial-transform layer must be appended so the predicted field can warp the moving image; and the loss must be

replaced, from the labelling losses (cross-entropy, Dice) to the pair of terms $\mathcal{L}_{sim} + \mathcal{L}_{smooth}$ of (50), which as a side effect removes the need for any label. On the input side, the network takes the *pair* of images, $\mathcal{R}_\theta : \mathbb{R}^{\Omega_1} \times \mathbb{R}^{\Omega_2} \to \mathbb{R}^{3\times\Omega}$, so the fixed and moving images enter as two input channels (Source: `UvA__AI4MIv0.1.apkg`). The encoder–decoder, the skip connections and the convolutional machinery stay as they are. (Source: `20260908_registration.pdf`, slides 33–34. Addresses exam §8d.)

5 Metrics: Selection, Dice, Reporting

Once a segmentation network is trained, its quality is asserted through metrics: numbers that are supposed to summarise, for a given biomedical question, how good the produced masks are. The first metrics lecture keeps returning to one theme: the number is only as good as the question it answers, and the question is only as good as the data it is computed on. The lecture opens with "horror stories" about labels: borked annotations and data are unfortunately common, so one should always inspect and visualise the data before anything else, and think about whether the framing and the objective make sense for the problem at hand. A perfect score against a wrong reference is worthless. (Source: `20260915_metrics01.pdf`, slide 5)

The same concern motivates the *Metrics Reloaded* framework, which starts from the observation that flaws in machine-learning validation are an underestimated global problem: in biomedical image analysis the chosen performance metrics often do not reflect the domain interest, and therefore fail to measure scientific progress and hinder translation into practice. Its authors group validation failures into three categories: *inappropriate choice of the problem category*, where the metrics do not reflect the biomedical need (for example an object-detection problem framed as segmentation, so that the localisation of all objects is never measured); *poor metric selection*, where mathematical properties of a metric are neglected (for example using the Dice similarity coefficient on particularly small structures); and *poor metric application*, where a well-chosen metric is applied naively to a dataset (for example ignoring the hierarchical data structure when aggregating, such as averaging over images when the relevant unit is the patient). These problems are aggravated because poor practice grows historically and is propagated between generations of researchers. (Source: `metrics reloaded paper.pdf`, §1, §2)

5.1 What to measure depends on the task and on the problem

The metric one picks depends first on the task. The deck lists: *classification*, where accuracy and sensitivity are the natural choices; *localisation*, where one looks at intersection over union and missed boxes; *segmentation*, which is the focus of this lecture and of the next, with the Dice score and the Hausdorff distance; and *registration*, treated in the guest lecture. But two tasks with the same name still need different metrics, because the difficulty of the problem varies: is it a multi-class problem? Are the objects single or multiple connected components, is their number variable, and what is the shape of the objects? Only after these questions are answered can a metric be chosen. (Source: `20260915_metrics01.pdf`, slides 8–9)

The two metrics lectures are organised as follows. The present one covers the Dice score — how to compute it, several of its properties, and its "weirdness" — and how to report and visualise metrics. The next one covers the Hausdorff distance (chapter 6) and calibration (section 5.7). (Source: `20260915_metrics01.pdf`, slide 10)

For **classification**, Metrics Reloaded distinguishes two properties. *Discrimination* is the ability to separate classes, measured by counting metrics on a fixed confusion matrix: sensitivity, specificity or F_1 in the binary case, the Matthews correlation coefficient (MCC) in the multi-class case. Fixing a confusion matrix requires an often arbitrary threshold on the predicted scores, so this is complemented by *multi-threshold* metrics such as the area under the ROC curve (AUROC), which vary the threshold and show the trade-off between error types. The deck's own classification metrics (accuracy, sensitivity) are of the first kind. Multi-class metrics summarise everything in one number but can hide poor performance on individual classes, so the framework recommends always adding multi-threshold metrics and optionally per-class counting metrics. When errors have unequal severity (ordinal data,

asymmetric costs), use the expected cost (EC), a generalisation of accuracy; for clinical prediction, net benefit puts benefits and harms on one scale. (Source: `metrics reloaded paper.pdf`, §7)

5.2 Problem fingerprints: choosing metrics from the problem

Metrics Reloaded selects metrics from a *problem fingerprint*: a structured description of the problem that captures every property relevant to metric choice, so that the recommendation does not depend on modality or application. It distinguishes four problem categories — image-level classification, object detection, semantic segmentation and instance segmentation — and proceeds in three steps: (1) *fingerprinting* (map the problem to its category and answer the fingerprint questions), (2) *metric selection* by following that category's decision tree, (3) *metric application*. The fingerprint items are grouped by *domain interest* (are boundaries, centres or volumes important? should annotation imprecision be compensated? is confidence important?), *target structure* (small relative to the pixel size? highly variable in size? tubular?), *dataset* (class imbalance, non-independent cases, references that may lack the target) and *algorithm output* (are class scores available? can the prediction be empty?). (Source: `metrics reloaded paper.pdf`, §3, §5–6)

For **semantic segmentation** the recipe is: (1) an *overlap-based* metric, by default Dice or IoU (F_β if FP and FN should weigh differently, clDice for tubular structures); plus (2) a *boundary-based* metric, by default the normalised surface distance (NSD). Metrics are computed *per image* and then aggregated, never pooled over all pixels, since pixels of one image are highly correlated; and Dice is mathematically the pixel-level F_1 score. Overlap metrics are avoided when structures are consistently small relative to the grid and the reference is noisy. For object detection there are no true negatives, so accuracy, specificity and AUROC are invalid. (Source: `metrics reloaded paper.pdf`, §7, §9–10)

Two warnings accompany the recipes. *Do not inherit the metric from the loss*: training with the Dice loss does not justify validating with Dice; the domain interest picks the metric. And *use several complementary metrics from different families*: overlap-based, boundary/distance-based (HD, ASSD, MASD, NSD), volume-based, information-theoretic (Rand index, variation of information) and probabilistic (Brier score, for calibration). (Source: `metrics reloaded paper.pdf`, §9, §11–12)

5.3 The Sørensen–Dice coefficient

For a discrete ground truth y and a discrete segmentation s over the image space Ω, with $\mathcal{K}$ the set of classes, let

$$\Omega_y^{(k)} \subseteq \Omega \quad \text{denote the subset labelled as } k, \qquad \Omega_s^{(k)} \subseteq \Omega \quad \text{the subset predicted as } k.$$

The Sørensen–Dice coefficient for class k is the ratio of twice the agreement to the total size of the two sets,

$$\mathrm{DSC}(y, s; k) := \frac{2\left|\Omega_y^{(k)} \cap \Omega_s^{(k)}\right|}{\left|\Omega_y^{(k)}\right| + \left|\Omega_s^{(k)}\right|}. \tag{53}$$

(Source: `20260915_metrics01.pdf`, slide 12)

In practice y and s are binary maps, $y^{(i,k)} \in \{0,1\}$ and $s^{(i,k)} \in \{0,1\}$ at pixel i, and the set cardinalities become sums over the image space, which is how the metric is implemented:

$$\mathrm{DSC}(y, s; k) = \frac{2\sum_{i\in\Omega} y^{(i,k)}\, s^{(i,k)}}{\sum_{i\in\Omega}\left[\, y^{(i,k)} + s^{(i,k)} \,\right]}. \tag{54}$$

The Dice score is also used as a training loss; its loss form and the derivation of its gradient are the canonical content of chapter 3, see (24), and are not repeated here. (Source: `20260915_metrics01.pdf`, slide 13)

5.3.1 Over-segmentation versus under-segmentation

The Dice score is not a neutral summary of errors: it favours over-segmentation. The deck announces the claim on cardiac segmentations taken from Camaras et al. (MIDL 2022), where several methods produce contours slightly smaller or slightly larger than the reference, states *Dice favours over-segmentation*, and then proves it. Let S be a perfect predicted segmentation and G the ground truth, so that

$$\mathrm{DSC}(S, G) = \frac{2|S \cap G|}{|S| + |G|} = 1. \tag{55}$$

Now consider two errors of the *same area* (same number of voxels) but of different type: $E_1 \not\subset G$ and $E_2 \subset G$, with $|E_1| = |E_2| = N$. The set $S \cup E_1$ represents an *over*-segmentation, whereas $S \setminus E_2$ represents an *under*-segmentation. Both errors misclassify exactly the same number of pixels, so a fair metric should score them equally. For the over-segmentation, using $|S| = |G|$ and $(S \cup E_1) \cap G = S \cap G = G$,

$$\begin{aligned} \mathrm{DSC}(S \cup E_1, G) &= \frac{2\big|(S \cup E_1) \cap G\big|}{|S \cup E_1| + |G|} = \frac{2|S \cap G|}{|S| + |E_1| + |G|} \\ &= \frac{2|G|}{2|G| + N} = \frac{4|G|^2 - 2|G|N}{4|G|^2 - N^2}, \end{aligned} \tag{56}$$

whereas for the under-segmentation, using $|S \setminus E_2| = |G| - N$ and $(S \setminus E_2) \cap G = G \setminus E_2$,

$$\begin{aligned} \mathrm{DSC}(S \setminus E_2, G) &= \frac{2\big|(S \setminus E_2) \cap G\big|}{|S \setminus E_2| + |G|} = \frac{2\big(|G| - |E_2|\big)}{2|G| - N} \\ &= \frac{4|G|^2 - 2|G|N - 2N^2}{4|G|^2 - N^2}. \end{aligned} \tag{57}$$

The two scores share the same denominator $4|G|^2 - N^2$ (positive since $0 < N \leq |G|$), and the numerator of the over-segmentation exceeds the numerator of the under-segmentation by $2N^2$. As both $|G|$ and N are greater than 0, this proves that

$$\mathrm{DSC}(S \cup E_1, G) > \mathrm{DSC}(S \setminus E_2, G) :$$

for two mis-segmentations of the same area, the over-segmentation is "favoured" by the Dice metric. (The deck prints $4|G|^2 + N^2$ in the denominators of equations (56) and (57); that sign is a typo, since putting $2|G|/(2|G| + N)$ and $2(|G| - N)/(2|G| - N)$ over the common denominator $(2|G| + N)(2|G| - N) = 4|G|^2 - N^2$ is what makes the comparison follow, and that common denominator is positive.) (Source: `20260915_metrics01.pdf`, slides 14–17)

5.3.2 What Dice does not capture

The Dice score is not all-encompassing. Given the same ground truth for a heart, two very different predicted shapes — one that is too small and one that is too large and blocky — obtain almost the same Dice score, around 0.66; the metric does not distinguish them. This is the same over-segmentation bias seen from the qualitative side: the metric rewards agreement in area at least as much as agreement in shape. (Source: `20260915_metrics01.pdf`, slide 18)

More drastically, Dice loses all notion of *spatial relationship*. The deck keeps a cardiac image, its ground truth and a prediction, and then progressively destroys the image while leaving the labels untouched: first keeping only the normal image, then only "shuffled lines" (the image rows permuted), then only "shuffled pixels" (pixels permuted at random). The Dice score is 0.92 in all three cases, because Dice only compares the two label sets and is blind to where the intensities came from. A distance-based metric such as the Hausdorff distance, however, is not fooled: for the same three cases it reports different values per class, e.g. 5.0000, 4.4721 and 3.0000 for the three classes on the normal image, 13.0000, 11.1803 and 3.0000 on the shuffled-lines image, and 7.0000, 11.3137 and 8.2462 on the shuffled-pixels image. The Hausdorff distance itself is treated in chapter 6; the example is listed here because it is the sharpest argument for reporting more than one metric family. (Source: `20260915_metrics01.pdf`, slide 19)

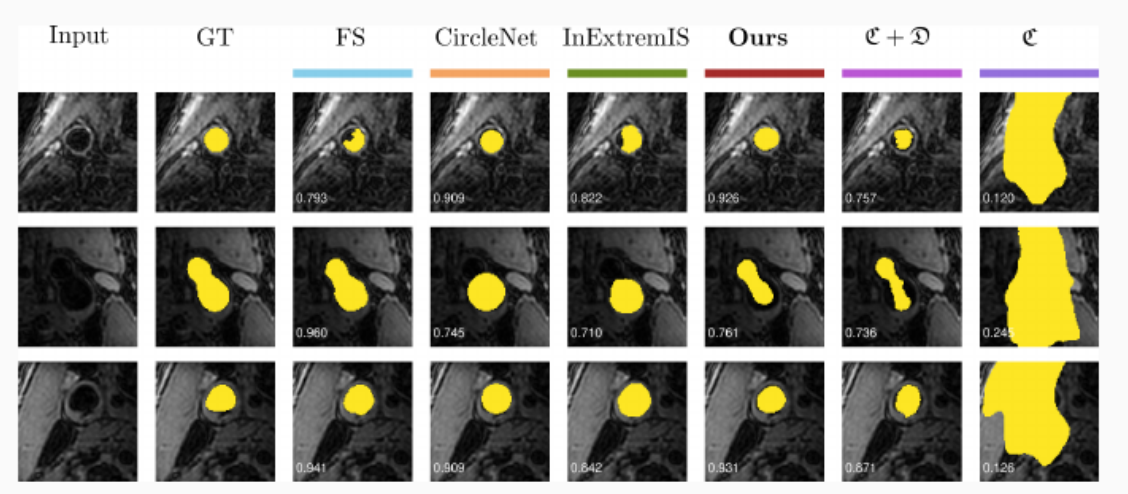


Figure 17: Contours produced by several segmentation methods on cardiac images, with the Dice value in the corner of each panel, against the input and the ground truth. The deck uses this example to announce that Dice favours over-segmentation, a claim proved in the following slides. Source: `20260915_metrics01.pdf`, slide 14.

5.3.3 Corner cases and the ε-modification

Dice has corner cases that every other metric also has, but solved differently. There are two axes: whether there is something to predict in the ground truth, and whether the prediction contains something. The textbook definition only covers the case where both are non-empty. The remaining three cells are all problematic, and the convention is different for every metric:

Table 7: The Dice corner cases. *Different for all metrics.* Source: `20260915_metrics01.pdf`, slide 20.

	Something to predict	Nothing to predict
I predicted something	textbook definition	?0
I did not predict anything	?0	?1

Concretely: if the ground truth is empty and the prediction is not, (53) gives $2 \cdot 0/(|pred| + 0) = 0$, which is arguably harsh but well defined; if both are empty the formula gives the indeterminate form $0/0$. To avoid an `if`/`else` in the code, the equation can be modified by adding a small $\varepsilon > 0$ to numerator and denominator,

$$\mathrm{DSC}(y, s; k) := \frac{2\left|\Omega_y^{(k)} \cap \Omega_s^{(k)}\right| + \varepsilon}{\left|\Omega_y^{(k)}\right| + \left|\Omega_s^{(k)}\right| + \varepsilon}, \tag{58}$$

so that the empty-empty case evaluates to $\varepsilon/\varepsilon = 1$, while an empty prediction against a non-empty ground truth evaluates to $\varepsilon/(|G| + \varepsilon) \approx 0$. This is the price of the trick, and it biases the metric: a class or a sample in which the structure is simply absent is automatically counted as a perfect prediction, so on a dataset where the structure is missing from many samples the aggregated score is inflated — the network is rewarded for a task it never had to perform. The strict definition instead leaves that case undefined (or, by convention, zero). Other options for the corner cases are discussed with the distance metrics in chapter 6. (Source: `20260915_metrics01.pdf`, slides 20–21)

5.4 Picking your metrics

The rule that follows from the previous section is that one should always use more than one metric; the question is which ones. The lecture's first pair is Dice and Jaccard. The Jaccard index (intersection

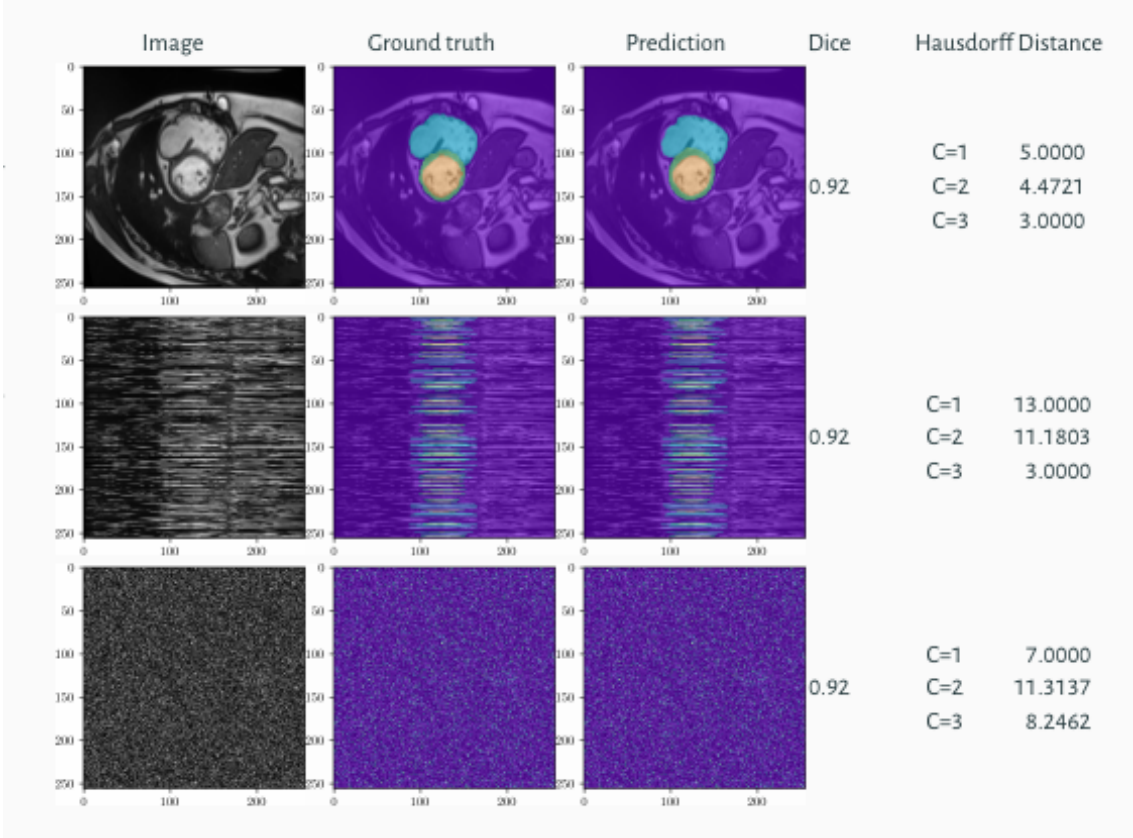


Figure 18: The loss of spatial relationship. Ground truth and prediction are identical in all three rows, so Dice is 0.92 every time, while the Hausdorff distance (per class $C = 1, 2, 3$) changes with the shuffled input. Source: `20260915_metrics01.pdf`, slide 19.

over union) for class k is

$$\text{Jaccard}(y, s; k) := \frac{\left|\Omega_y^{(k)} \cap \Omega_s^{(k)}\right|}{\left|\Omega_y^{(k)}\right| + \left|\Omega_s^{(k)}\right| - \left|\Omega_y^{(k)} \cap \Omega_s^{(k)}\right|} = \frac{\left|\Omega_y^{(k)} \cap \Omega_s^{(k)}\right|}{\left|\Omega_y^{(k)} \cup \Omega_s^{(k)}\right|}. \tag{59}$$

The two metrics are monotonically related: if D denotes the Dice score and J the Jaccard index of the same pair of sets, then

$$J = \frac{D}{2 - D}, \qquad \text{equivalently} \qquad D = \frac{2J}{1 + J}. \tag{60}$$

On the numbers of the lecture, two methods have average Dice 93.69 and 93.91; the deck's own table reports their average Jaccard as 88.47 and 88.68, while applying the conversion to the average Dice gives $0.9369/(2 - 0.9369) = 0.8813$ and $0.9391/(2 - 0.9391) = 0.8852$ — the deck prints both and they differ, a reminder that averaging a metric is not the same as converting the average. Either way the Jaccard index is systematically lower than the Dice it corresponds to, for the same two masks. (Source: `20260915_metrics01.pdf`, slides 24–25)

The relationship itself is proved as follows. Write D and J for two sets A and B,

$$D = \frac{2|A \cap B|}{|A| + |B|}, \qquad J = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|}.$$

Starting from X and Y as generic sets, invert the Dice score,

$$\frac{1}{D} = \frac{|X| + |Y|}{2|X \cap Y|},$$

then multiply both sides by 2 and subtract 1, which puts the right-hand side over the common denominator $|X \cap Y|$,

$$\frac{2}{D} - 1 = \frac{|X| + |Y| - |X \cap Y|}{|X \cap Y|} = \frac{1}{J}.$$

Solving $2/D - 1 = 1/J$ for D gives $D = 2J/(1 + J)$, and solving for J gives $J = D/(2 - D)$. (Source: `20260915_metrics01.pdf`, slide 26)

Beyond Dice and Jaccard, the metrics should be taken from several *families*. For segmentation the deck distinguishes *overlap-based* metrics (the Dice score and its derivatives) and *distance-based* metrics (the Hausdorff distance and its derivatives), and adds "others if relevant" — notably clinical metrics — pointing at the challenge repository `grand-challenge.org/challenges` for examples of how challenges report them. Metrics Reloaded makes the same point with its own family names: overlap-based, boundary-based (a subset of distance-based), volume-based, information-theoretic and probabilistic/calibration families, selected from different families precisely because no single metric covers the requirements of the problem. (Source: `20260915_metrics01.pdf`, slide 27; `metrics reloaded paper.pdf`, §5)

Finally, one must decide *when* in the pipeline the metric is computed. If the goal is to evaluate the method, or some new technique, measuring without post-processing may be favourable, so that the raw output of the network is judged; if the goal is to push performance to its limit, then measuring with post-processing may make sense, since post-processing is part of the deployed system. Reporting both is always an option, and can shed light on the failure cases and on the impact — or lack thereof — of the post-processing step. (Source: `20260915_metrics01.pdf`, slide 28)

5.5 Reporting your metrics

At the beginning there was the average — the deck refers to its own `distrib.py` script — and the average is still the baseline way of reporting a dataset score. But the average hides the distribution, so the lecture recommends visualisation frameworks: Matplotlib, and notably the scientific-visualisation book companion, seaborn and plotly. The advice is to be creative about how metrics and their distribution or relationship are reported and interpreted, rather than reporting a single number per method. As an illustration of handling multiple metrics at once, the deck points to a complex case study on arXiv (`2411.08629`) and, immediately before the statistics discussion, to the `xkcd` comic "Significant" (#882), which warns about the same family of mistakes. (Source: `20260915_metrics01.pdf`, slides 30–33)

Statistical testing must not come before defining what a sample is, and the definition must be stated explicitly. The candidate definitions offered are: one run of a K-fold validation (a K-element sample of dataset-averaged Dice scores); one scan of a single run (a few dozen values, depending on the dataset); one slice of a scan (a few thousand); or one voxel (a few million). The last two, and especially the last, seriously border on p-hacking: there is no meaningful independence left at those granularities, so any difference becomes significant given enough voxels. This choice is exactly the hierarchical-data pitfall that Metrics Reloaded highlights for aggregation — computing metrics per image, then aggregating over images or patients, and testing on the aggregated units. (Source: `20260915_metrics01.pdf`, slide 34; `metrics reloaded paper.pdf`, §9)

5.6 The limits of metrics and of annotations

The final caveat of the lecture is about the reference itself: in many applications different raters will be north — or even south — of 90% DSC with respect to each other. If two human experts on the same task can disagree that widely (the lecture gives no exact figure), then a model whose score differs from the reference by a few points cannot be declared worse: the reference annotation is only an approximation of the forever unknown ground truth. Metrics Reloaded formalises the same point as a fingerprint property (compensation for annotation imprecision requested), which for boundaries leads to recommending the normalised surface distance as the default boundary metric, and to outlier-robust metrics when the reference may contain spatial outliers. It also stresses that

correctness metrics are not the whole story: validation should also cover robustness (similar accuracy across hospitals and protocols) and reliability (a model should communicate its confidence and raise a flag when uncertain), and an algorithm that is reliable or robust is not necessarily the most accurate one, so the trade-off must be balanced deliberately. (Source: `20260915_metrics01.pdf`, slide 36; `metrics reloaded paper.pdf`, §5, §11, §12)

5.7 Calibration

A complementary property of a model — the one the second metrics lecture closes with — is calibration. The intuition is simple: the confidence that a network attaches to its prediction should match the accuracy it actually achieves, i.e. when a model predicts with 0.8 confidence, it should be right about 80% of the time. Calibration is assessed with *reliability diagrams*: the predicted confidences are bucketed into bins on the x-axis, the accuracy measured inside each bin is the bar height on the y-axis, and the diagonal $y = x$ marks perfect calibration. In the deck's example the bars fall below the diagonal for the intermediate bins (mild over-confidence) and straddle it at the extremes. (Source: `20260914_lecture07_metrics02.pdf`, slides 39–40)

The distance to that diagonal is quantified by the expected calibration error. With N the number of samples, M bins B_m and p selecting the ℓ_p norm of the discrepancy, the

$$\mathrm{ECE} \;=\; \sum_{m=1}^{M} \frac{|B_m|}{N} \left\| \mathrm{Accuracy}(B_m) - \mathrm{Confidence}(B_m) \right\|_p^p, \tag{61}$$

is a weighted average of the per-bin gaps, where the weight is the fraction of samples falling in the bin (in the slide the weights are drawn as the varying widths of the boxes); typically $p = 1$, and the value lies in $[0, 1]$, smaller being better. (Source: `20260914_lecture07_metrics02.pdf`, slide 40)

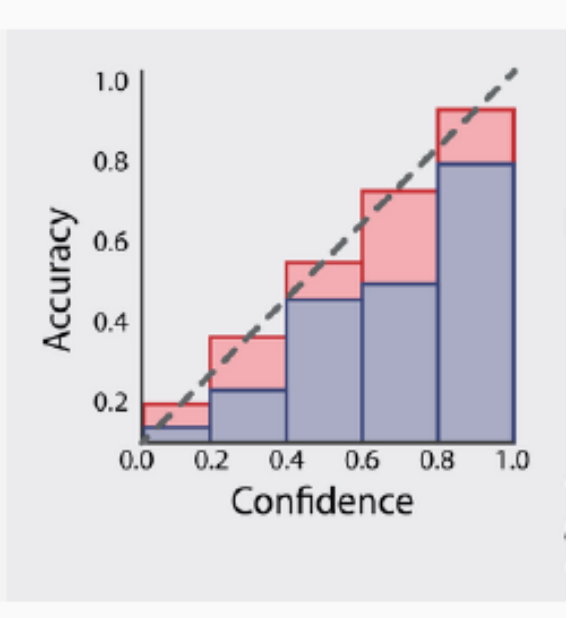


Figure 19: Reliability diagram: bin-wise accuracy (bar height) against predicted confidence (bin), with the dashed diagonal of perfect calibration. Source: `20260914_lecture07_metrics02.pdf`, slide 40.

The implementation follows the definition directly: put the predictions into bins of confidence; choose how many bins, since the number of bins may impact the results; compute the accuracy within each bin; subtract the difference for each bin; and aggregate the whole. The deck closes with a final slide titled "Implementation example"; it carries no static content (it is the live demo of this pipeline). (Source: `20260914_lecture07_metrics02.pdf`, slide 41; slide 42 is the title-only demo page)

Calibration is not a substitute for discrimination but a complement to it, and Metrics Reloaded recommends adding one or more calibration metrics to the metric pool whenever confidence assessment is requested in the fingerprint, with the expected calibration error (top-label) and the root Brier score as the standard choices for its use cases. Calibration must also not be over-interpreted: as the framework's discussion stresses, the predicted scores of a well-calibrated model do not express the true posterior probability of the input belonging to a class, which is why some argue for basing calibration assessment on proper scoring rules such as the Brier score instead of stand-alone calibration metrics. (Source: `metrics reloaded paper.pdf`, §5, §7, §12)

6 Distance-based Metrics and Their Implementation

Overlap metrics such as the Dice score (see 5.3) summarise a segmentation by counting how many voxels the prediction and the label agree on. They are blind to *where* the disagreement sits: a prediction that is uniformly one voxel too small and a prediction whose errors are scattered across the volume can produce the same Dice value, and both saturate as soon as the overlap is large. Distance-based metrics close that gap by measuring how far the boundary of a prediction is from the boundary of the reference. This chapter defines the Hausdorff distance and its percentile/average variants, re-derives the exact four-step pipeline used to implement them, and describes the implementation pitfalls that make nominally identical metrics return different numbers in different open-source tools.

6.1 Significant numbers and measurement error

A reported number carries an implicit claim about its own uncertainty. Writing 7.00 asserts that the value is known to two decimals, i.e. to within ± 0.005; a measurement whose error is ± 0.5 therefore satisfies $7 = 7.0$ but *not* $7 = 7.00$, since $(7 \pm 0.5) \neq 7.00$. The measurement itself, and not the number of digits printed by a library, fixes how many figures may be written down. (Source: `20260914_lecture07_metrics02.pdf`, slide 5)

Two different qualities are hidden behind a single number: *accuracy* is how close the measurements are to the true value, whereas *precision* is how close repeated measurements are to one another. A method can be precise yet inaccurate (a systematic bias that shifts every prediction the same way) or accurate on average yet imprecise (a large spread between runs); the classical dart-board illustration is on the slide. (Source: `20260914_lecture07_metrics02.pdf`, slide 6)

How many digits, then, should a metric be reported with? The answer is dictated by the measurement error, not by the method's apparent performance: human inter-rater agreement on a segmentation task is only around 90% DSC (often even less), so two methods whose Dice differ in the third decimal cannot be separated by such annotations and quoting those digits is spurious. (Source: `20260914_lecture07_metrics02.pdf`, slide 7)

An honest error bar aggregates every relevant source of uncertainty: the inter- and intra-rater variability (or the inter-rater score itself), the size of the object (a small structure is relatively harder to delineate), the variance across cross-validation folds, the variance across repeated runs (i.e. across random seeds), and any other source of uncertainty in the pipeline. In practice most papers report three or four significant figures for a metric. (Source: `20260914_lecture07_metrics02.pdf`, slide 8)

6.2 The Hausdorff distance

The Hausdorff distance is informally described as "the maximum distance between two segmentations". Computed naively over two masks, however, that phrase hides a choice: should the distance be measured between the two *sets* of voxels, or between their *boundaries*? The deck poses the question explicitly and answers it in the formal definition below: the distance is measured over the boundaries $\partial\Omega_S$ and $\partial\Omega_Y$, which is what makes the metric sensitive to contour errors rather than to bulk overlap. (Source: `20260914_lecture07_metrics02.pdf`, slides 10–11)

6.2.1 Preliminary notation

The image space is a discrete grid $\Omega \subset \mathbb{R}^3$. A voxel is written $i \in \Omega$ and a class $k \in \mathcal{K}$; the prediction is a binary indicator $S^{(i,k)} \in \{0,1\}$ and the reference label is $Y^{(i,k)} \in \{0,1\}$. The subset predicted as class k, and the subset labelled as class k, are

$$\Omega_S^{(k)} := \{\, i \in \Omega \mid S^{(i,k)} = 1 \,\}, \qquad \Omega_Y^{(k)} := \{\, i \in \Omega \mid Y^{(i,k)} = 1 \,\}, \tag{62}$$

and the classes form a partition of the image space: $\Omega_S^{(0)} \cup \Omega_S^{(1)} \cup \cdots \cup \Omega_S^{(K)} = \Omega$ and, for all $k \neq l$, $\Omega_S^{(k)} \cap \Omega_S^{(l)} = \emptyset$. (Source: 20260914_lecture07_metrics02.pdf, slide 12)

In the binary setting the class index is dropped, $\Omega_S := \{\, i \in \Omega \mid S^{(i)} = 1 \,\}$, and the boundary is defined as the set of foreground voxels that lie arbitrarily close to the background,

$$\partial\Omega_S \subset \Omega_S, \qquad \partial\Omega_S := \{\, i \in \Omega_S \mid \forall \epsilon > 0,\ \exists j \notin \Omega_S : d(i,j) < \epsilon \,\}. \tag{63}$$

(Source: 20260914_lecture07_metrics02.pdf, slide 13)

Finally, voxels carry *coordinates*, $i \equiv (x_i, y_i, z_i) \in \Omega$, so that the Euclidean distance between two voxels and the distance of a voxel to a set are

$$d(i,j) := \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2 + (z_i - z_j)^2}, \qquad \varphi_A(b) := \min_{a \in A} d(a,b). \tag{64}$$

Here b is a voxel and A a set of voxels; the coordinates (and hence d) are expressed in physical units, so a voxel's size is part of the formula, not an afterthought. (Source: 20260914_lecture07_metrics02.pdf, slide 14)

6.2.2 Formal definition, continuous and discrete

With these ingredients the Hausdorff distance between a prediction S and a reference Y is the larger of the two directed deviations between the two boundaries,

$$\mathrm{HD}(S,Y) := \max\left(\max_{i \in \partial\Omega_S} \min_{j \in \partial\Omega_Y} d(i,j),\ \max_{i \in \partial\Omega_Y} \min_{j \in \partial\Omega_S} d(i,j) \right) = \max\left(\max_{i \in \partial\Omega_S} \varphi_{\partial\Omega_Y}(i),\ \max_{i \in \partial\Omega_Y} \varphi_{\partial\Omega_S}(i) \right), \tag{65}$$

i.e. every boundary voxel of one segmentation is matched to its nearest boundary voxel of the other, and the worst such distance is kept. The first directed term measures how far the prediction's boundary is from the reference boundary; the second measures the reverse, which is why the metric is symmetric. The Hausdorff distance is the only one of the metrics discussed here that is mathematically well defined on a discretised grid without further interpretation, precisely because it is a maximum of maxima and therefore insensitive to how the distances are aggregated. (Source: 20260914_lecture07_metrics02.pdf, slide 15; 2410.02630v2.pdf, §5)

In the continuous setting the same object is written with a supremum outside and an infimum inside,

$$\mathrm{HD}(X,Y) = \max\left(\sup_{x \in X} \inf_{y \in Y} d(x,y),\ \sup_{y \in Y} \inf_{x \in X} d(x,y) \right), \tag{66}$$

and the discrete formula (65) is its grid counterpart: on a voxel grid the minimum distance from the boundary voxels of A to the boundary of B (solid arrows on the slide) and from the boundary voxels of B to the boundary of A (dashed arrows) must both be computed, and the largest value over the two directions is the metric. (Source: 20260914_lecture07_metrics02.pdf, slide 16)

6.3 Computing the Hausdorff distance

6.3.1 The four steps

Computing the Hausdorff distance — and, with a different final aggregation, HD95 and ASSD as well — is a four-step procedure:

1. extract the two boundaries $\partial\Omega_S$ and $\partial\Omega_Y$;
2. compute the Euclidean Distance Transform (EDT) of the two boundaries, giving $\varphi_{\partial\Omega_S}$ and $\varphi_{\partial\Omega_Y}$;
3. perform a masking of each EDT with the *other* boundary mask;
4. pick the maximum, the 95-th percentile, or the average of the masked values, depending on the metric being computed.

The masks must be binary (one-hot) before step 1, since the boundary is defined per class; nothing else — in particular no upsampling "for improved precision" — belongs to the pipeline. (Source: `20260914_lecture07_metrics02.pdf`, slide 17; `distorch_poster.pdf`, slide 1)

6.3.2 Worked example

The practicals implement the pipeline on a synthetic 2D example: a grid `Xs, Ys` = mgrid[−128:128, −128:128] on which the prediction is the disk $S = \left(\sqrt{X_s^2 + Y_s^2} < 80\right)$ and the reference is the oblique elliptical shape $Y = \left(\sqrt{X_s^2 + 2Y_s^2 + X_s Y_s} < 100\right)$. (Source: `20260914_lecture07_metrics02.pdf`, slides 18–19)

```
kernel = np.array([[1,1,1],[1,0,1],[1,1,1]])     # 8-neighbourhood
nS = convolve2d(S, kernel, mode='same')          # number of neighbours in S
bS = (nS != kernel.sum()) * S                    # extracted boundary
phiS = distance_transform_edt(1 - bS)            # EDT of the boundary
nY  = convolve2d(Y, kernel, mode='same')
bY  = (nY != kernel.sum()) * Y
phiY = distance_transform_edt(1 - bY)
dS = phiS * bY                                   # distances of the Y boundary to the S boundary
dY = phiY * bS                                   # distances of the S boundary to the Y boundary
hd = max(dS.max(), dY.max())                     # Hausdorff distance
```

6.3.3 Boundary extraction

A foreground voxel belongs to the boundary when it is not completely surrounded by foreground. The deck realizes this with a convolution: with the 8-neighbour kernel k above the number of foreground neighbours of voxel i is $n_S(i) = \sum_j k(i-j)\,S(j)$, and the boundary is

$$\partial\Omega_S = \left\{\, i \in \Omega_S \;\middle|\; n_S(i) \neq |k| \,\right\} \cdot \Omega_S, \qquad |k| = 8, \tag{67}$$

which in code is simply `(nS != kernel.sum()) * S`. The variant kernel `[[0,1,0],[1,0,1],[0,1,0]]` (4-connectivity) is also offered. Note that with convolution padding the voxels on the outer face of the array always have fewer than eight neighbours, so they are classified as boundary whatever the mask; this is exactly the kind of convention difference discussed in §6.5 below. (Source: `20260914_lecture07_metrics02.pdf`, slide 21)

The extraction is not unique, and this is the first big implementation pitfall. The same ground truth — a cross-shaped object plus one isolated voxel — yields four different boundary sets: the mask itself, a 3×3 square-connectivity erosion, a full-connectivity erosion, and the boundary obtained by shifting the grid half a pixel so that it sits at the interface between foreground and background. Nine of the eleven open-source tools compared in `2410.02630v2.pdf` use a boundary-based calculation, but they do not agree on which boundary; two of them (EvaluateSegmentation, SimpleITK) skip the extraction altogether and use *all* foreground voxels as query points, which biases the distance distribution towards zero because overlapping regions contribute zero distances. (Source: `20260914_lecture07_metrics02.pdf`, slide 20; `2410.02630v2.pdf`, §4)

6.3.4 Euclidean distance transform

The EDT turns a boundary set into a distance field: for every voxel b, $\varphi_A(b) = \min_{a \in A} d(a, b)$ is the distance from b to the closest element of A. Implementations compute the EDT of the *complement* of

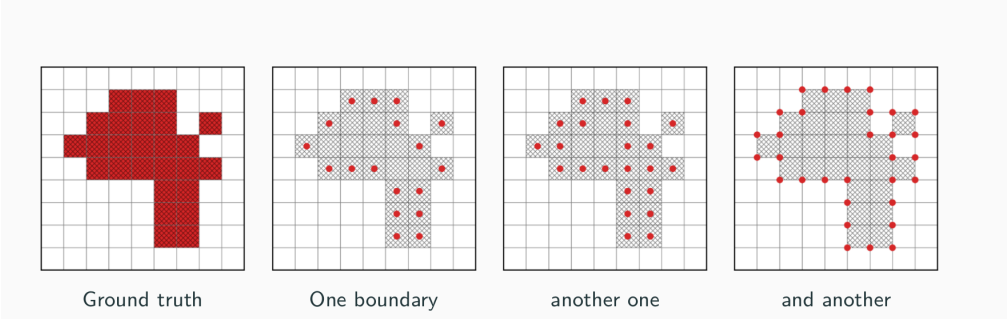


Figure 20: Four ways of representing the boundary of the same ground-truth mask (a cross-shaped object plus one isolated voxel): the mask itself, and the boundary produced by 3×3 square connectivity, full connectivity, and the half-pixel grid shift. Source: `20260914_lecture07_metrics02.pdf`, slide 20 (figure reproduced from Podobnik & Vrtovec, arXiv:2410.02630v2, Fig. 2).

the boundary mask (`distance_transform_edt(1 - bS)`), because the routine returns the distance to the closest zero-valued voxel; passing $1 - \partial\Omega_S$ makes the boundary the zero level, so the result is the distance to the boundary itself. The values are in physical units when the voxel spacing is passed along, and in voxels otherwise. (Source: `20260914_lecture07_metrics02.pdf`, slides 23–24)

$$\varphi_{\partial\Omega_S}(b) \;=\; \min_{i\in\partial\Omega_S} d(i,b) \;=\; \mathrm{EDT}_{\partial\Omega_S}(b) \;\geq\; 0, \qquad \varphi_{\partial\Omega_S}(b) = 0 \iff b \in \partial\Omega_S. \tag{68}$$

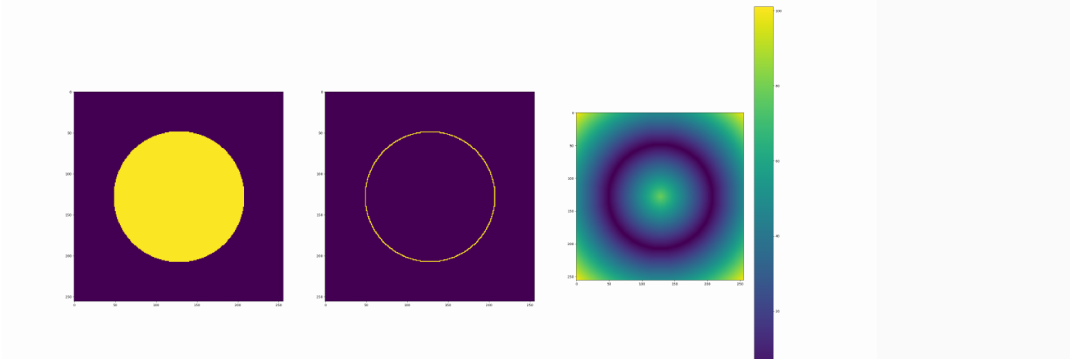


Figure 21: The distance transform of a boundary: the binary mask S (left), its extracted boundary $\partial\Omega_S$ (middle) and the EDT $\varphi_{\partial\Omega_S}$, evaluated at every voxel (right; colour encodes the distance, the boundary is the zero level). Source: `20260914_lecture07_metrics02.pdf`, slide 24.

6.3.5 Masking

Masking keeps, in each distance field, only the voxels that belong to the *other* boundary. Element-wise multiplication by the boundary mask does the job:

$$d_S := \varphi_{\partial\Omega_S} \odot \partial\Omega_Y \qquad \text{and} \qquad d_Y := \varphi_{\partial\Omega_Y} \odot \partial\Omega_S, \tag{69}$$

so that d_S contains, for every voxel of $\partial\Omega_Y$, its distance to $\partial\Omega_S$ (and 0 elsewhere), and d_Y the mirror quantity. (Source: `20260914_lecture07_metrics02.pdf`, slides 25–26)

One implementation detail matters for the averaging variants: because the multiplication leaves zeros outside the mask, an average taken over the whole array would be diluted by those zeros. Averages and percentiles must be taken over the masked entries only (the deck's ASSD/HD95 aggregations do exactly this), or the non-boundary entries must be excluded explicitly. (Source: `2410.02630v2.pdf`, §6–7)

6.3.6 Aggregating

The last step collapses the two distance fields into one number. On the worked example the two directed maxima are $\max d_S = 32.31$ and $\max d_Y = 32.20$, so

$$\mathrm{HD}(S, Y) = \max\big(\max d_Y, \max d_S\big) = 32.31,$$

reported here in voxels, since the synthetic example uses unit spacing. Replacing the outer maximum by the 95-th percentile of the masked distances gives HD95, and replacing it by the mean gives the average surface distance; both are discussed in §6.4. (Source: `20260914_lecture07_metrics02.pdf`, slide 27)

6.3.7 Runtime

Three operations dominate the running time of the textbook pipeline: the boundary extraction (convolution plus masking), the EDT, and the masking. The EDT is the bottleneck and is usually CPU-bound: on a 3D volume it easily takes dozens of seconds, it has to be computed twice (once per boundary), and it must be repeated for every class. For a task with K classes plus background, the textbook pipeline therefore performs two EDTs per evaluated class — $2K$ for the K foreground classes, or $2(K+1)$ if the background class is evaluated too — regardless of whether the aggregation is HD or ASSD, because the two metrics share the same distance fields; Dice requires none, since it never leaves the voxel-counting world of 5.3. (Source: `20260914_lecture07_metrics02.pdf`, slides 17, 28–29; `distorch_poster.pdf`, slide 1)

6.3.8 DisTorch: a GPU alternative

DisTorch takes the opposite route and never computes an EDT. It observes that the boundary of a segmentation is far smaller than the segmentation itself, $|\partial\Omega_S^{(k)}| \ll |\Omega_S^{(k)}|$ — a blob-like shape in 2D has a contour of roughly $2(h+w)$ elements — and therefore computes all pairwise distances between the two boundary sets directly, at the price of quadratic complexity:

$$\begin{aligned}\mathrm{HD}(s, g; k) = \max\Big\{ &\max_i \min_j \{\, d(i,j) \mid i \in (\partial\Omega_s^{(k)} \setminus \partial\Omega_g^{(k)}),\ j \in \partial\Omega_g^{(k)} \,\}, \\ &\max_i \min_j \{\, d(i,j) \mid i \in (\partial\Omega_g^{(k)} \setminus \partial\Omega_s^{(k)}),\ j \in \partial\Omega_s^{(k)} \,\}\Big\}.\end{aligned} \tag{70}$$

Boundary elements common to both segmentations are skipped because their distance is zero. Materialising such a pairwise (kernel) matrix would exhaust memory, so the reduction is delegated to the KeOps library, which evaluates sums/maxima/minima over kernel matrices without ever forming them, on GPU registers; a Triton backend is also available. The implementation keeps the metric consistent with MONAI, adds only a few dependencies, and needs extra care for edge cases such as empty boundary sets and identical (matching) boundary sets. (Source: `20260914_lecture07_metrics02.pdf`, slide 30; `108_DisTorch_A_fast_GPU_implem.pdf`, §2–3; `distorch_poster.pdf`, slide 1)

On three 3D datasets (SegTHOR, OAI, WMH) DisTorch runs in tens of milliseconds per sample, two to three orders of magnitude faster than the CPU tools MedPy and MeshMetrics ($\sim 10^4$ ms), on par with MONAI's EDT accelerated by cuCIM, and with lower GPU memory; it is especially fast for small objects. It degrades on arbitrarily mispredicted volumes — where, the authors argue, reporting HD is meaningless anyway: compute it only once Dice is sufficiently high. (Source: `108_DisTorch_A_fast_GPU_implem.pdf`, §3–4)

6.4 Other distance-based metrics

The same two masked distance fields d_S and d_Y support three aggregations, hence three metrics. HD is the maximum distance over both directions, HD95 is the 95-th percentile of the distances, and ASSD is their average:

$$\mathrm{HD}_{95}(S, Y) := \max\big(P_{95}\big(\{\varphi_{\partial\Omega_Y}(i)\}_{i\in\partial\Omega_S}\big),\ P_{95}\big(\{\varphi_{\partial\Omega_S}(i)\}_{i\in\partial\Omega_Y}\big)\big)\,, \tag{71}$$

$$\mathrm{ASSD}(S,Y) := \frac{\sum_{i\in\partial\Omega_S}\varphi_{\partial\Omega_Y}(i) \;+\; \sum_{i\in\partial\Omega_Y}\varphi_{\partial\Omega_S}(i)}{|\partial\Omega_S| \;+\; |\partial\Omega_Y|}, \tag{72}$$

while the mean average surface distance (MASD) averages each directed mean separately and then takes the mean of the two,

$$\mathrm{MASD}(S,Y) := \frac{1}{2}\left(\frac{1}{|\partial\Omega_S|}\sum_{i\in\partial\Omega_S}\varphi_{\partial\Omega_Y}(i) + \frac{1}{|\partial\Omega_Y|}\sum_{i\in\partial\Omega_Y}\varphi_{\partial\Omega_S}(i)\right). \tag{73}$$

Note that $\mathrm{HD} = \mathrm{HD}_{100}$, and that all of these are *absolute* metrics: they are measured in millimetres, have no upper bound, and lower is better. By contrast, relative surface metrics such as the normalised surface distance (surface Dice) and the boundary IoU are unitless, bounded, and higher is better; they compare boundaries within a user-defined tolerance τ and are not derived from the same aggregation. Variants of all these definitions exist, and Metrics Reloaded compares them systematically. (Source: `20260914_lecture07_metrics02.pdf`, slide 32; `2410.02630v2.pdf`, §5–6)

6.5 Implementation pitfalls across open-source tools

A review of eleven open-source tools (Anima, EvaluateSegmentation, Google DeepMind, MedPy, Metrics Reloaded, MISeval, MONAI, Plastimatch, pymia, seg-metrics, SimpleITK) finds that the same masks give HD values differing by more than 100 mm, and statistically significant differences between most tools — one can "achieve" a significant improvement simply by switching library. Three families of divergence explain it. (Source: `20260914_lecture07_metrics02.pdf`, slide 31; `2410.02630v2.pdf`, §1, §8)

- **Boundary extraction.** Two tools skip it and use *all foreground voxels* (overlap contributes zero distances, so HD95/ASSD are biased low); the others use square-connectivity erosion, full-connectivity erosion, or a half-pixel grid shift that places the boundary at the foreground/background interface (Figure 20).
- **Definitions.** HD95 has three incompatible versions: percentile of each directed set then max (MONAI, Metrics Reloaded, Google DeepMind); percentile of the *union* of both sets; or the *average* of the two directed percentiles, which even breaks $\mathrm{HD}_{100} = \mathrm{HD}$. MASD/ASSD and NSD also vary (plain vs. boundary-size-weighted means), and some tools ignore voxel size.
- **Edge cases and units.** With an empty mask, tools return NaN, ∞, huge integers, errors or even random values; errors explode at non-unit and anisotropic spacing.

Recommendations: report the tool's name and version, do not trust outputs on empty masks, always pass the voxel spacing, and prefer Google DeepMind's implementation (or a mesh-based one) for HD, HD95, MASD and ASSD. (Source: `2410.02630v2.pdf`, §4–5, §9–12)

6.6 Corner cases and undefined distances

The degenerate configurations are best summarised in a small table with two rows (did I predict anything?) and two columns (was there anything to predict?). When something is predicted and something exists, the textbook definition applies. When nothing is predicted but the reference is non-empty — and symmetrically, when the reference is empty but the prediction is not — the distance is undefined. When both are empty the distance is naturally 0. (Source: `20260914_lecture07_metrics02.pdf`, slide 33)

Undefined here means undefined in the sense of C/C++: anything is a valid output, and implementations disagree wildly (see the edge-case discussion above). If a number must be produced, the "least worst" option is to fall back on the maximum diameter of the scan (the scan diagonal, in mm), or on a fixed penalty — keeping in mind that any default value biases the comparison between methods according to their propensity to miss objects (Source: `UvA__AI4MIv0.1.apkg`); invented alternatives such as the diameter of the pizza one had for lunch or of the Milky Way are exactly as

defensible, which is the point of the joke. The situation matters in practice because a model that does not predict every class leaves some $\Omega_S^{(k)}$ empty, and the corresponding boundary is empty too,

$$\Omega_S^{(k)} = \emptyset \implies \partial\Omega_S^{(k)} = \emptyset,$$

so the directed maxima in (65) range over an empty set. (Source: 20260914_lecture07_metrics02.pdf, slide 34; hausdorff_summary.png)

The boundary can also swallow the whole mask. The extraction (67) removes only voxels completely surrounded by foreground, so $\partial\Omega_S = \Omega_S$ whenever the mask has *no interior voxel*. Intuitively, if every foreground voxel touches the background, nothing gets removed. This happens in two situations. The first is thinness: a sheet, a curve, a skeleton, a single isolated voxel, or any object thinner than the structuring element (a 2×2 block, for instance). The second is the empty prediction seen above, where both sets are empty. In both cases the "boundary-based" and "foreground-based" calculations coincide, so the metric inherits all the sensitivity to the boundary convention. (A mask that fills the whole image is the opposite case: with zero padding, only the outer shell of the array touches the outside, and that shell is the extracted boundary.) (Source: 20260914_lecture07_metrics02.pdf, slides 13, 21)

One last corner case separates counting metrics from distance metrics. Suppose a 2D network is run slice by slice and the 3D volume is stitched back with a bug in the slice order (say $1, 12, 13, 2, 21, 22, \ldots$ instead of $1, 2, 3, \ldots$), identically for prediction and label. **Dice is unchanged:** it only counts overlapping voxels and knows nothing about where they are, and a permutation applied to both volumes leaves those counts intact. **ASSD and HD change:** they are built from Euclidean distances, and shuffling the slices moves boundary voxels to different physical positions, so the same pair of masks is now compared in a distorted geometry. This is the practical reason Dice can look healthy while a distance metric collapses, and it follows directly from (65) and (72). (Source: 20260914_lecture07_metrics02.pdf, slides 17, 32)

These degeneracies also drive the classic optimisation of the pipeline: when the model does not predict all classes, the EDT of an absent class is not merely useless but undefined, so those classes should be skipped (and, symmetrically, classes absent from the reference); because HD and ASSD share the same two EDTs they should be computed once and aggregated three ways (maximum, percentile, mean); and the boundary, not the full mask, is the right input to the transform. Beyond that, the runtime can only be reduced by changing algorithm — a GPU implementation without EDT such as DisTorch, or an accelerated EDT such as MONAI with cuCIM. (Source: 20260914_lecture07_metrics02.pdf, slides 17, 29–30)

6.7 Resolution, units and significant figures

Distance-based metrics carry units, overlap metrics do not. Dice is unitless, whereas the Hausdorff distance is measured in millimetres, because voxels have a size and scans have a resolution: the distance in (64) is a physical distance, computed from the physical coordinates of the voxels, not from their indices. Any implementation that forgets the voxel spacing silently changes the metric — this is precisely the failure mode that produces deviations of hundreds of millimetres in the tool comparison above. (Source: 20260914_lecture07_metrics02.pdf, slide 36)

The distance transform must therefore be evaluated with the spacing: the reference implementation `distance_transform_edt` accepts a `sampling` argument (one value per dimension), which is how anisotropic scans are handled. The same discretisation also makes the set of achievable distances finite, which is the quantised distance effect discussed above. (Source: 20260914_lecture07_metrics02.pdf, slide 37)

Resolution finally bounds the number of significant figures a metric deserves, which closes the loop with the first section of this chapter. If a scan is isotropic with 2 mm per voxel, the coordinates of a voxel are only known to 2 mm, so it makes little sense to report a dataset average of 1.36 mm: the digits after the millimetre are below the resolution of the measurement, and the value should be rounded to something like 1–2 mm, consistent with the discretisation. Reporting more figures

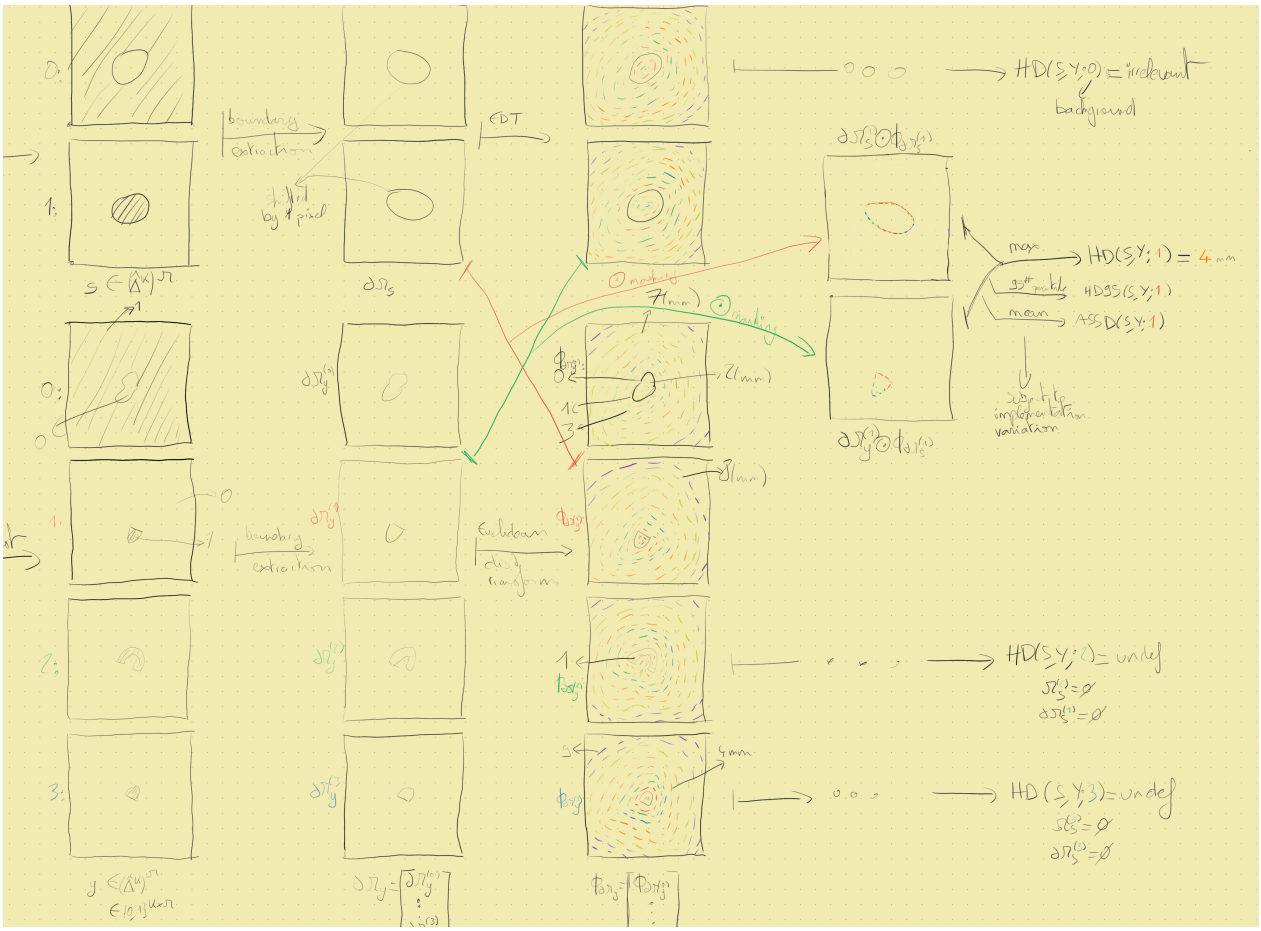


Figure 22: Handwritten summary of the whole pipeline. Top and middle: the two masks S and Y (with the classes $\{0, 1, 2, 3\}$, class 0 being the background) and, column by column, boundary extraction, the EDT, the masking of one distance field by the other boundary, and the aggregation into HD (maximum), HD_{95} (95-th percentile) and ASSD (mean). Right and bottom: the corner cases — the background class is irrelevant, and when a class is absent both $\Omega_S^{(k)}$ and its boundary are empty, so the distance is undefined. Source: `hausdorff_summary.png` (lecturer's handwritten notes).

than the measurement error supports is the same mistake as writing 7.00 for a value known to ± 0.5. (Source: `20260914_lecture07_metrics02.pdf`, slide 38)

The deck points to Alvy Ray Smith's memo *A Pixel Is Not A Little Square* as a tangential read: a pixel is a *point sample* of a continuous image, not a little square (the square look only comes from box-replicating samples when zooming in), and "non-square pixels" are really non-square *spacings* between samples. For metrics this means a voxel is a point at a physical position with spacing (d_x, d_y, d_z), distances must be computed in physical units, and any grid boundary only approximates the continuous object. (Source: `Smith95b.pdf`, §2–§7)

7 Ethics, Data Management and Reproducibility

The last lecture of the course splits into three parts: the rules that govern how medical data may be collected, stored and shared; what it means for a piece of research to be reproducible; and what happens once a model leaves the laboratory, together with the responsibility that comes with it. The lecture's treatment of federated learning and differential privacy (slides 22–25) belongs to the chapter on domain adaptation and federated learning; this chapter covers the remaining slides 4–21 and 26–42. (Source: `20260917_lecture06_ethics.pdf`, slide 4)

7.1 Data collection and management

7.1.1 Regulatory frameworks

Medical-imaging research and products are subject to regulatory frameworks, and the slide stresses four properties of them rather than naming any single regulation: they apply "especially in Europe", they are *subject to change*, they are *currently hotly debated*, and they *cannot be ignored*, especially in industry. The practical consequence for a project is that a legal position is not a one-off acquisition but something that must be re-checked while the project runs. (Source: `20260917_lecture06_ethics.pdf`, slide 5)

7.1.2 The GDPR

The concrete European instrument referred to is the **General Data Protection Regulation** (GDPR), an EU regulation. The slide reduces its requirements to three items a researcher must be able to answer for: *patient consent*, a *purpose* (the specific use for which the data are processed), and *good care and handling* of the data. Consent is therefore not a blanket permission: it is tied to a stated purpose, and how the data are handled afterwards is itself part of the obligation. (Source: `20260917_lecture06_ethics.pdf`, slide 6)

7.1.3 Retrospective studies and consent

When clinical data have already been collected and one wants to re-use them, the lecture states that retrospective studies *require one to ask consent for that specific use* — consent is purpose-bound, so the original clinical consent does not automatically cover a research purpose. Two exceptions are recognised: cases where *consent cannot be asked*, and cases where it *"cannot be reasonably required"*. A question that follows naturally from these exceptions is what to do when part of the cohort has died since the data were acquired; the answer the lecture's structure supports is that consent genuinely cannot be asked from a deceased patient, so that case falls into the stated exception and the project must instead obtain approval from the relevant ethical committee (see below) to re-use the data. (Source: `20260917_lecture06_ethics.pdf`, slide 7; `exams/exam2025.pdf`, §7b)

7.1.4 Ethical committees in the Netherlands

In the Netherlands the committees that must be involved are, at least: the **WMO** (the act on medical research involving human subjects), the **METC** (*Medisch Ethische Toetsingscommissie*, the medical-ethical review committee), and, in addition, "probably an internal one" in one's own institute or company. A project can therefore need several distinct approvals rather than a single one. (Source: `20260917_lecture06_ethics.pdf`, slide 8)

7.1.5 Data Management Plans

A **Data Management Plan** (DMP) has to be laid out *before* the data are collected, and it must answer three questions: how the *security* of the data will be guaranteed, how their *quality* will be guaranteed, and *what and how* one will comply with the applicable laws and regulations. In practice one rarely starts from a blank page, because such internal guidelines "usually already exist" in the institution — the slide points to a presentation given at the AUMC as an example of a template already in use. (Source: `20260917_lecture06_ethics.pdf`, slides 9–10)

7.1.6 Anonymous versus pseudonymous data

The lecture separates a record into two parts: the *data* — the scan, the image proper — and the *metadata* — patient name, date, sex, weight, and so on. De-identification means removing personal identifiers and replacing the name or identifier by a random number, and the difference between the two regimes lies entirely in what happens to the mapping between the random number and the

real identity: in **pseudo-anonymisation** someone still has that mapping, somewhere, whereas in **anonymisation** the mapping has been deleted. That is the core difference between the two notions: the presence or absence of a re-identification key, not the amount of identifying information removed from the images themselves. (Source: `20260917_lecture06_ethics.pdf`, slide 11; `exams/exam2025.pdf`, §7d)

7.1.7 The limits of anonymisation: patients are unique

The slide immediately following the definitions warns that anonymisation "has its limits", and the remainder of the first part of the course is devoted to why. Two examples are shown: a dataset that is easy to anonymise, and *face removal in MRI*, i.e. the fact that a head scan can contain the patient's face, so that defacing or skull-stripping is part of the anonymisation pipeline for such data. (Source: `20260917_lecture06_ethics.pdf`, slides 12–13)

The deeper reason is that an "average" patient does not exist. The lecture cites Todd Rose's *The end of average* (2016) to make the point that individuals are not scaled copies of an average case, so the very fact that a record describes *one* specific person is itself information that can single that person out. The lecture then runs this argument as a live experiment: finger length as a finger-printing technique — measurements that look innocuous in isolation combine into a quasi-identifier. The same point is made anatomically with a comparison of body proportions from the new-born to the adult: a child is not a small adult, because the proportions, the heart rate, the blood pressure and the respiration rate all change by large factors along development, as the table below shows. (Source: `20260917_lecture06_ethics.pdf`, slides 14–16)

	Newborn	Toddler	Schoolchild	Adolescent	Adult
Body height (cm)	50	85	115	150	165 (female) / 175 (male)
Body weight (kg)	3.3	12	20	40	60 (female) / 67 (male)
Heart rate (per min)	140	120	95	85	70
Blood pressure (mmHg)	60/35	90/50	100/60	110/70	120/80
Respiration rate (per min)	40	25	20	17	15

Table 8: Normal anatomical and physiological values from the new-born to the adult, used in the lecture to argue that children are not scaled-down adults. Source: `20260917_lecture06_ethics.pdf`, slide 16 (after *Sobotta Atlas of Anatomy*, 17th edition).

The physical form of the patient is therefore not a neutral container for a diagnosis: height, weight and the internal proportions of an anatomy are individually variable enough to act as identifiers, which is why "we removed the name column" is not by itself a sufficient anonymisation argument. (Source: `20260917_lecture06_ethics.pdf`, slides 14–16)

The exam makes this operational by asking to rank four kinds of image from easiest to hardest to anonymise: an iris photograph, a biopsy scan, a head MRI including the brain, and a whole-body CT. The ranking follows how much identifying content the acquisition covers and whether it can be removed without destroying the image:

1. **Biopsy scan (easiest).** A microscopic tissue view with no face, body geometry or biometric signature; the residual risk lies in the metadata.
2. **Head MRI including the brain.** It may contain the face, but defacing removes it (as in the lecture's face-removal example), leaving brain anatomy.
3. **Whole-body CT.** The field of view covers the entire body, face included, and the 3D volume allows re-identification from the whole body habitus.
4. **Iris photograph (hardest).** It cannot really be anonymised: the iris *is* the identifier, so removing the identifying content removes the data.

(Source: `exams/exam2025.pdf`, §7a, together with `20260917_lecture06_ethics.pdf`, slides 11–15)

7.1.8 Licensing and public challenges

Data come with a licence, and the lecture's instruction is simply to *check the licence carefully* before re-use, with the Creative Commons family as the canonical example: CC BY 4.0 (attribution only) versus CC BY-NC 4.0 (attribution, non-commercial), a difference that decides whether a dataset may be used in an industrial collaboration at all. The slide points to `https://grand-challenge.org/challenges/` as the platform on which the medical-imaging challenges are hosted. (Source: `20260917_lecture06_ethics.pdf`, slide 17)

Public challenges are worth participating in for two main reasons. First, they supply a *common benchmark with a held-out test set on which every participant is evaluated by the organisers*, so results obtained by different groups become comparable instead of being reported on private splits with private metrics. Second, they force the *public release of the data, the annotations and the evaluation protocol under an explicit licence*, which is exactly what the reproducibility checklist of this chapter asks for (public datasets, available code, documented material). (Source: `exams/exam2025.pdf`, §7f, which poses the question; the two effects are the answer the question calls for — the benchmark/test-set point is *No source in this repo.* The deck's own pointers are the licensing slide and the challenge platform, `20260917_lecture06_ethics.pdf`, slides 17 and 27.)

7.1.9 Collaboration and data transfer agreements

Working with others on clinical data has two constraints. If the collaboration is not within the same group, a **Data Transfer Agreement** (DTA) is often required before any data move, and — independently of the paperwork — *not everyone in the project has access to the data*: access is granted per person, not per project. The second constraint is that collaborators have *different expertise and focus*, so the data are seen through different questions; both facts have to be designed into the project rather than discovered halfway through. (Source: `20260917_lecture06_ethics.pdf`, slide 18)

7.1.10 Code storage

For the code, several options are listed: use the facilities of one's company, typically a *private GitLab hosted on the facility*; keep a *local git repository*, somewhat backed up; or use a *public or private GitHub*, in which case the absolute rule attached to the option is *without any patient data saved there.* Reproducibility (below) pushes towards public hosting of the code, while the GDPR obligations of the previous sections set the boundary: code may be public, data may not. (Source: `20260917_lecture06_ethics.pdf`, slide 19)

7.1.11 Data breaches and electronic hygiene

On security, the lecture's guiding maxim is that *the best data is the data you do not have*: the primary measure is to *limit the scope of what is collected*, because data that were never acquired cannot leak. When a breach or leak does occur, the instruction is procedural rather than technical: the *institution* reports it as soon as possible. (Source: `20260917_lecture06_ethics.pdf`, slide 20)

Electronic hygiene follows from the same attitude, and its rules apply always, but especially when handling sensitive data. Passphrases should be strong for the few one needs to remember (the slide points to a readable-passphrase generator) and delegated to a password manager for everything else; a *second factor* should be used, physical tokens being the recommended form; drives should be *encrypted*; and one should never plug a "found" USB key into one's computer. (Source: `20260917_lecture06_ethics.pdf`, slide 21)

7.2 Reproducibility

7.2.1 The reproducibility checklist for papers

For a paper, the lecture adopts the checklist of `midl.io/reproducibility`, which is a list of yes/no questions a reviewer can ask:

1. Is the reproducibility of the work addressed?
2. Is the code publicly available?
3. Are public datasets used?
4. Are the required package dependencies listed?
5. Is the code for model training available?
6. Is the code for model evaluation available?
7. Is the presented trained model available?
8. Is there documentation for the available material?
9. Is there licensing for the available material?

The list separates three distinct things that are often conflated: the *method* (training and evaluation code, dependencies), the *data* (public datasets), and the *artefact* (the trained model, its documentation and its licence). A paper can score on some and fail on others, which is precisely what the private-data question at the end of this section is about. (Source: `20260917_lecture06_ethics.pdf`, slide 27)

7.2.2 Four meanings of "reproducible"

The lecture then asks what "reproducible" actually means, and answers with four candidate criteria of decreasing strength:

- whether the trained network parameters are *bit-wise identical*;
- whether one can *train the same model* (same code, same data) *on a different machine*;
- whether one can do so *with slightly different package versions*;
- whether the results fall *within a defined confidence interval* — and if so, which one.

The first criterion is exact reproducibility of the artefact, the second and third are reproducibility of the *procedure* under a change of environment, and the fourth is statistical reproducibility: a claim, plus an interval, rather than a point value. This is also the shape of the answer expected by the exam when it asks to define one meaning of reproducibility and to give a way to improve it: for example, taking "train the same model (code and data) on a different machine" as the definition, the way to improve it is to actually remove the uncontrolled sources of variation described below — set the seeds, use deterministic implementations of the operations, and pin the environment (package versions), while publishing code, data and the trained model. (Source: `20260917_lecture06_ethics.pdf`, slide 28; `exams/exam2025.pdf`, §7e)

7.2.3 The numerous sources of randomness

The reason bit-wise identical parameters are hard to obtain is that the training procedure itself is stochastic: the course trains networks with stochastic (i.e. random) gradient descent. The lecture lists the concrete sources:

- network weights are *initialised randomly*;
- *batch sampling* is random;

- the default *floating-point operations on a GPU are non-deterministic*;
- the *hardware architecture* might behave slightly differently;
- *kernel versions* might introduce some variation.

Two further caveats are stated explicitly: this list does *not* cover software, compiler and hardware *bugs*, which are a separate class of irreproducibility, and it is the reason why "same code, same data, different machine" is not expected to give the same weights by default. (Source: `20260917_lecture06_ethics.pdf`, slide 29)

7.2.4 True randomness versus pseudo-randomness

Randomness on a computer is itself worth a remark. *True randomness is very difficult to obtain* on a computer, and it is only truly required for *cryptography*; the slide's example is a cryptographic service that derives entropy from a wall of lava lamps. The alternative used everywhere else is *pseudo*-randomness: a sequence that appears random but is in fact generated by a deterministic process from a single *seed*. Two consequences follow. First, the "random" numbers of an experiment are reproducible if the seed is known, which is what the next section exploits. Second, whatever entropy the machine actually has can be inspected directly — on Linux, `cat /proc/sys/kernel/random/entropy_avail`. The comic reproduced on the slide (xkcd 221) makes the point in one line, a "random number" function that simply returns `4` and calls it guaranteed to be random. (Source: `20260917_lecture06_ethics.pdf`, slides 30–31)

7.2.5 Eliminating some sources of (pseudo-)randomness

Because pseudo-randomness is deterministic given the seed, one can remove part of the variance by setting the seed of every library that generates randomness. The lecture's snippet is exactly that:

```
import random
random.seed(0)
import numpy as np
np.random.seed(0)
import torch
torch.manual_seed(0)
```

In addition to seeding, one can request *deterministic algorithms* from the framework (PyTorch exposes this as `torch.use_deterministic_algorithms`), which removes the non-determinism of the GPU operations listed above at the price of speed. The section closes with the methodological warning that accompanies the whole discussion: *"Are you really testing a model, a seed, an 'artifact'?"* — if a reported improvement can be reversed by changing the seed, then what was measured was the seed and not the method, and a single-seed comparison cannot tell the two apart. (Source: `20260917_lecture06_ethics.pdf`, slide 32)

7.2.6 Deterministic configuration: the nnU-Net pipeline

Randomness belongs to the *training* of a network, not necessarily to the *choice of the configuration*. The nnU-Net pipeline — its self-configuring architecture and stages are described in §2.4 and cross-referenced rather than repeated here — makes that choice by a fixed set of rules rather than by a search. There is therefore no random or evolutionary search over configurations: given the same input dataset, the same rules produce the same configuration, so running the pipeline on a different machine with a different GPU cannot find a different "optimal" configuration. The nnU-Net paper confirms why the GPU does not matter: the rules act on a *predefined* GPU memory budget and *estimate* memory use from feature map sizes, so "no GPU is required to run the adaptation process" (§2.4.1); only changing that budget would change the configuration. What a different machine *can* change is the resulting network weights, through exactly the training-side sources of randomness

listed above (weight initialisation, batch sampling, non-deterministic GPU operations, hardware and kernel differences) — unless those are neutralised by seeding and deterministic algorithms. (Source: `20260917_lecture06_ethics.pdf`, slides 29 and 32; `exams/exam2025.pdf`, §7c)

The quiz asks for at least three statistics used for fingerprinting the data in the nnU-Net pipeline — the descriptions of the dataset that the configurator reads before deciding anything. The lecturer's model answer refers to the paper and lists *spacings*, *modality*, *shape* and *intensity distribution*; the full list (image size before and after cropping, spacing, modalities, number of classes, number of cases, foreground intensity mean, s.d. and 0.5/99.5 percentiles) is in §2.4.1. (Source: `quiz2024_annotated.pdf`, Q10; `nnunet paper.pdf`, Methods)

7.2.7 Private data and public code

Can research be called reproducible when the code is public but part of the validation data is private to a hospital? The answer is that it is only *partially* reproducible.

- **Satisfied:** the method side of the checklist (training and evaluation code, dependencies, documentation, licence). What stays verifiable is the implementation and whatever else the authors release, such as the trained model.
- **Not satisfied:** the data side. The meaning "train the same model (code and data) on a different machine" cannot be executed by an outsider, who has the code but not the data and so cannot verify the reported numbers.

Making the private cohort reusable is a data-management problem (consent, DTA and anonymisation, as in the first part of this chapter). When the data cannot be shared even for training, the fallback in this course is federated learning, see §9. (Source: `exams/quiz2024.pdf`, Q9; `20260917_lecture06_ethics.pdf`, slides 27–28, 11 and 18)

7.3 Inference, deployment and responsibility

Engineering culture. "Move fast and break things" (a discontinued Facebook motto) does not fit a clinical application, where what breaks is patients. Other engineering fields hold practitioners responsible for what they build (the Canadian engineers' *Iron Ring*); software has largely escaped this, with the *Therac-25* radiotherapy machine, whose software defects killed patients, as the counter-example. (Source: `20260917_lecture06_ethics.pdf`, slides 34–35)

AI and radiologists. Hinton (2016: "people should stop training radiologists now") and Ng (2017: pneumonia from chest X-rays "better than radiologists") were wrong predictions. The current, explicitly *unproved*, consensus is that radiologists using AI will do better than those without, and that AI will not replace them: it can accelerate some tasks, reduce the need for a second opinion, and make grading more consistent — properties of the clinician-plus-system pair, not of the model alone. (Source: `20260917_lecture06_ethics.pdf`, slides 36–37)

Who is responsible? The candidates are those collecting the data, the engineers designing and training the model, the company selling it, the regulator approving it, and the user — "a tough legal nut" left open: responsibility is spread along the whole pipeline. (Source: `20260917_lecture06_ethics.pdf`, slide 38)

Lessons from aerospace. Automation can cause a loss of intuition and understanding of the system (Air France 447), and confirmation bias must be mitigated (Air Transat 236): a system that is usually right trains its users to stop checking it. (Source: `20260917_lecture06_ethics.pdf`, slide 39)

Fairness, robustness, interpretability. These are the further requirements a medical system must meet. Robustness covers generalising across sites: an ultrasound model will generalise worse than a CT model because ultrasound is *operator-dependent* (quiz Q8, §1). The lecture closes by recommending

two MIDL keynotes (Lekadir 2024, trustworthy AI; Rusu 2025, multimodal early cancer detection in low-resource settings). (Source: `20260917_lecture06_ethics.pdf`, slides 40–42; `quiz2024_annotated.pdf`, Q8)

8 Image Reconstruction

So far the images were given. This chapter starts one step earlier, at the point where the image is *produced* from raw measurements, and asks how a neural network can help there. The lecture covers learned MRI reconstruction from under-sampled measurements, then a family of related "generative imaging" tasks that share the same idea: denoising, super-resolution, modality translation and anomaly detection. The lecture's last part (foundation models, domain shift) is in Chapter 9. (Source: `20260929_lecture08_reconstruction.pdf`, slides 2–3 and 38)

8.1 Motivation: MRI acquisition and under-sampling

The lecture recalls the MRI measurement as a five-step process, described physically in §1: (1) align all hydrogen atoms with the magnet; (2) emit a radiofrequency wave; (3) measure how quickly the atoms go back into alignment; (4) repeat for several frequencies; (5) reconstruct the image. The raw data collected are the *k-space*, the spectral (frequency) representation of the image, so reconstruction means going from k-space back to an image. (Source: `20260929_lecture08_reconstruction.pdf`, slide 6)

The idea for saving time is to measure only part of k-space, i.e. to *under-sample* it. If the sampling is naive, the result is visibly wrong: with an under-sampling factor of 2 the reconstructed image shows the anatomy repeated and overlapping (Figure 23). The reconstruction problem is therefore to recover a clean image from fewer measurements than a direct inverse transform needs. (Source: `20260929_lecture08_reconstruction.pdf`, slide 7; an interactive demo is linked on the slide)

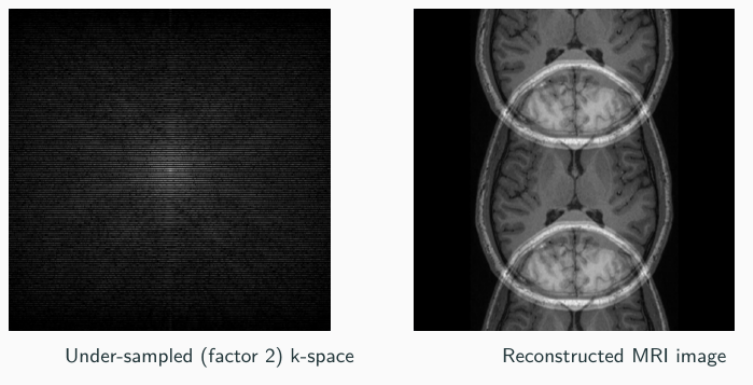


Figure 23: Under-sampled (factor 2) k-space (left) and the MRI image reconstructed directly from it (right): the anatomy appears repeated and overlapping. Source: `20260929_lecture08_reconstruction.pdf`, slide 7.

The lecture links this to JPEG image compression, which rests on a similar idea: remove the high frequencies to "compress" the image (the slide shows a photograph and its frequency spectrum before and after removing high frequencies). (Source: `20260929_lecture08_reconstruction.pdf`, slide 8)

8.2 Compressed sensing

Reconstruction from under-sampled data is an *ill-posed* problem: many different images are compatible with the few measurements taken. The classical answer, *compressed sensing* (the slide points to a commercial implementation from Siemens Healthineers), uses *iterative algorithms* to pick one plausible solution. The slide reproduces a vendor diagram of such an iterative pipeline. The rest of the chapter replaces that iteration by a learned network. (Source: `20260929_lecture08_reconstruction.pdf`, slide 9)

8.3 Learning MRI reconstruction from sub-sampled k-space

Notation. Let Ω be the image space and Ξ the spectral domain. The k-space information is $\mu \in \mathbb{C}^{\Xi}$ (complex-valued) and an image is $x \in \mathbb{R}^{\Omega}$. The lecture writes $\mathcal{F} : \Xi \to \Omega$ for the Fourier transform and $\mathcal{F}^{-1} : \Omega \to \Xi$ for its inverse, exactly as printed on the slide, and defines from a dataset of *fully sampled* images $\mathcal{D} = \{x_n\}_{n=1}^{N}$ the corresponding k-space data

$$\mu_n = \mathcal{F}^{-1}(x_n). \tag{74}$$

(Source: `20260929_lecture08_reconstruction.pdf`, slide 10)

Training objective. Since fully sampled images are available, one can *simulate* the fast acquisition: an artificially sub-sampled k-space $\tilde{\mu}_n$ is produced from μ_n, and a network $\mathcal{N}_\theta$ with parameters θ maps it to a reconstructed image, $\mathcal{N}_\theta : \tilde{\mu}_n \mapsto \tilde{x}_{\theta,n}$. The original fully sampled image x_n is then the target, and training solves

$$\min_\theta \mathcal{L}_{\text{rec}}(\tilde{x}_\theta, x), \qquad \text{for instance} \qquad \mathcal{L}_{\text{rec}}(\tilde{x}_\theta, x) \propto \sum_{i \in \Omega} \left(\tilde{x}_\theta^{(i)} - x^{(i)} \right)^2, \tag{75}$$

i.e. a voxelwise squared difference between the network's reconstruction and the fully sampled image. This is supervised training in the same sense as segmentation (Chapter 3), except that the target is an image and the loss is a regression loss rather than cross-entropy or Dice. (Source: `20260929_lecture08_reconstruction.pdf`, slide 11)

Adding self-supervision. The lecture presents VORTEX (Desai et al., MIDL 2022, best paper award), which adds a *consistency* term computed on augmented data; its diagram is shown in Figure 8 (already discussed under augmentation in Chapter 2). Take an augmentation strategy $\mathcal{A} : \tilde{\mu} \mapsto \tilde{\tilde{\mu}}$ that perturbs the sub-sampled k-space, and let the network reconstruct the perturbed input, $\tilde{\tilde{x}}_\theta := \mathcal{N}_\theta(\tilde{\tilde{\mu}})$. The two reconstructions of the same underlying data should agree, which gives the consistency loss

$$\mathcal{L}_{\text{cons.}}(\tilde{x}_\theta, \tilde{\tilde{x}}_\theta) \propto \sum_{i \in \Omega} \left(\tilde{x}_\theta^{(i)} - \tilde{\tilde{x}}_\theta^{(i)} \right)^2, \tag{76}$$

and the full objective adds it to the reconstruction loss with a weight $\lambda \in \mathbb{R}$,

$$\min_\theta \; \mathcal{L}_{\text{rec}} + \lambda \, \mathcal{L}_{\text{cons.}}. \tag{77}$$

Note that (76) compares two network outputs with each other, so it needs no fully sampled reference image; this is what makes it a self-supervision term, and it is the same "sum of a data loss and a weighted extra term" structure as the mixed loss (30) and the regularised registration objective (48). (Source: `20260929_lecture08_reconstruction.pdf`, slides 12–13)

8.4 Generative imaging: denoising, super-resolution and translation

The lecture groups several further tasks under the name "generative imaging". Each is illustrated by a recent MIDL or journal paper, and the deck shows an example image rather than a derivation, so these notes list the task and its input/output. (Source: `20260929_lecture08_reconstruction.pdf`, slide 14)

- **Denoising.** Map a noisy image to a clean one. The slide compares a noisy brain MRI with the outputs of several networks (UNet, RED-CNN, HAIR, QICNN) and a zoomed region in which the noise texture is visibly reduced (Dutta, MIDL 2026). *(slide 15)*
- **Super-resolution.** Map a low-resolution (LR) image to a high-resolution (HR) one; the example is blind super-resolution of 2D MRI (Poudel et al., MIDL 2026, the SRMRI dataset and benchmarks). *(slide 16)*

- **Modality generation (image translation).** Generate an image of one modality from another; the slide compares a source image, several translation methods (CycleGAN, U-GAT-IT, SynDiff, UNIT, EGSDE, I2I-Galip) and the target, with an error map per method (Korkmaz and Patel, MIDL 2026). *(slide 17)*
- **Denoising for anomaly detection.** A diffusion model is used as a learned denoiser (Bercea et al., *Melba*, 2025). A disease image x_P is perturbed with noise, and the model, guided by the supervision scheme, denoises it into a *pseudo-healthy* reconstruction $\hat{x}_H$. The *anomaly map* is the voxelwise difference $a = |x_P - \hat{x}_H|$, which highlights the pathological changes (Figure 24). *(slide 18)*

(Source: `20260929_lecture08_reconstruction.pdf`, slides 15–18)

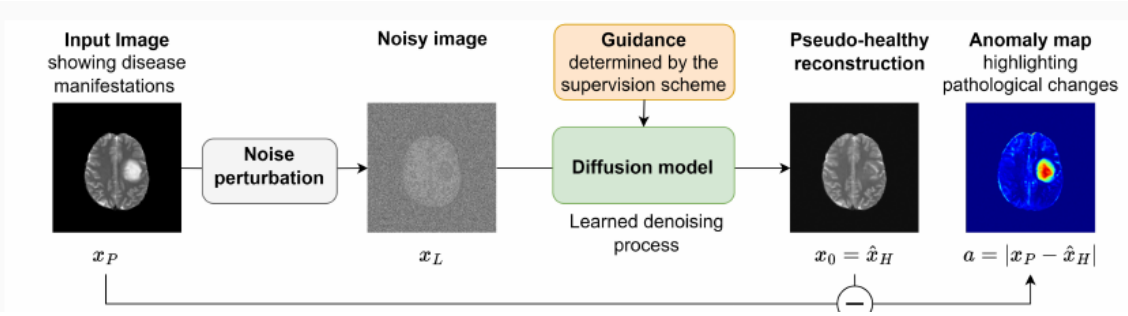


Figure 24: Denoising for anomaly detection: a noise-perturbed disease image is denoised by a diffusion model into a pseudo-healthy reconstruction, and the difference with the input is the anomaly map. Source: `20260929_lecture08_reconstruction.pdf`, slide 18 (Bercea et al., *Melba*, 2025).

Similarities and limits. The lecture's common thread is that all these tasks apply the same idea: "learn" to generate, or recover, the missing information. Its closing questions are the caveat: what are we actually measuring, and do we really have the information? Different modalities exist for a reason, so a network that fills in missing content produces plausible content, not necessarily measured content. (Source: `20260929_lecture08_reconstruction.pdf`, slide 19)

8.5 What the deck does not cover

The 2025 exam (§4) asks to choose between four deep-learning reconstruction approaches — direct, two-step/iterative, raw-to-task and conventional — for three tasks (A: complete, high-SNR measurements and a strong physics model; B: sparse measurements and limited prior knowledge; C: limited, noisy measurements and moderate prior knowledge) and to discuss computational constraints. The lecture deck in this repository does not name or define these four approaches, so the choice for each task is *No source in this repo (deck not provided).* What the deck does give as background for such a discussion is: compressed sensing as the iterative baseline for sparse (under-sampled) measurements (§8.1), the learned reconstruction network trained on simulated under-sampling (75), and the limit stated above, that missing information cannot be recovered when it was never measured. (Source: `exams/exam2025.pdf`, §4; `20260929_lecture08_reconstruction.pdf`, slides 9–11, 19)

9 Domain Adaptation, Transfer Learning and Federated Learning

A model is only as good as the population it was trained on, and in medical imaging the training population is almost never the deployment population. A network trained at one hospital is applied at another with different scanners, settings and patients; a network trained on one modality is asked to work on another; and the data needed to fix such a model may not be legally shareable at all. This chapter covers two families of answers: *adapting* a trained model to a new domain (transfer learning and domain adaptation), and *training a shared model without moving the data* (federated

learning), plus the privacy notion behind the latter and its price. Once a model is trained it becomes an artefact that can leave the hospital, so the questions of which data it saw, what it leaks and where it still works belong together with Chapter 7. (Source: `20260917_lecture06_ethics.pdf`, slides 23–25; `exams/exam2025.pdf`, §3)

Note on sources. The ethics deck covers federated learning, differential privacy and the "no free lunch" slide. The reconstruction/foundation-model deck (`20260929_lecture08_reconstruction.pdf`, slides 21–36) covers domain shift and foundation models such as SAM. The repository still has no deck on transfer-learning recipes or aggregation rules, so where this chapter explains the standard answer to an exam or quiz scenario, it is marked *No source in this repo (deck not provided).*

9.1 Transfer Learning and Domain Adaptation

Domain shift, as defined in the lecture. A training set is collected on a domain D_1, $\mathcal{D} = \{x_n\}_{n=1}^{N}$ with $x_n \sim D_1$ for all n, but at inference the data come from a second domain D_2, $x_m \sim D_2$. This is called *domain shift*, or sometimes *data drift*, and it is one of the main causes of poor generalisation. The lecture lists the typical sources: a different scanner, a different scanner vendor, a different scanning protocol, an image that is operator dependent, and different populations. (Source: `20260929_lecture08_reconstruction.pdf`, slides 27–28) The slide on *domain adaptation from source to target domains* shows the effect on segmentation: pairs of spine images (Water vs In-Phase MRI), prostate MRI from different sites, and cardiac MRI vs CT, where in the target domain the prediction confidence and accuracy drop (Bateson, Kervadec et al., *Medical Image Analysis* 2022, on source-free domain adaptation). (Source: `20260929_lecture08_reconstruction.pdf`, slide 29)

The problem in the exam: hospital A to hospital B. The 2025 exam describes a ResNet-50 trained on chest X-rays from Hospital A to separate pneumonia from normal. Tested on Hospital B, which uses a different scanner and contrast settings, its accuracy falls from 92% to 68%. The cause is a *domain shift*: the *source domain* (where the model was fitted) differs from the *target domain* (where it is used). Scanner and contrast settings change the intensity statistics, the noise and the texture of the same anatomy, so the features the network learned no longer line up with the new images. This is not an optimiser bug, and training longer on Hospital A data cannot fix it, because extra epochs only tie the model more tightly to the source domain. (Source: `exams/exam2025.pdf`, §3a poses the scenario; the explanation is *No source in this repo (deck not provided).*)

The two remedies. Both reuse the trained network instead of discarding it.

- **Transfer learning.** Keep the trained network as an initialisation. Its early layers already encode generic features (edges, blobs, textures), so one keeps the backbone, replaces the classification head if the labels change, and *fine-tunes* on target data. With little target data, the earliest layers are frozen so that only the layers closest to the task adapt.
- **Domain adaptation.** Attack the shift itself. With labelled target data this is fine-tuning; with only *unlabelled* target images one can (i) re-standardise the target images to the source intensity statistics, (ii) align the feature distributions of the two domains, or (iii) train adversarially, so that a domain classifier cannot tell source features from target features.

The exam expects two of these, typically fine-tuning on target data plus an unsupervised adaptation or harmonisation step; in practice they are combined because labelled data from the new hospital is the limiting factor. *No source in this repo (deck not provided); `exams/exam2025.pdf`, §3a gives only the question.*

Change of modality. The 2024 quiz asks about a model trained on brain MRI that must now find abnormalities in lung CT. Again one does not retrain from scratch: keep the pretrained network as a feature extractor, make the input and output layers compatible with the new modality and labels, and fine-tune with the generic early layers retained. Since MRI to CT changes the physical contrast mechanism and not merely the scanner, this is a stronger shift than changing hospital, so

transfer usually needs to be combined with intensity renormalisation and adaptation on the target data. *No source in this repo (deck not provided);* ***exams/quiz2024.pdf****, Q4 gives only the scenario.*

9.2 Federated Learning

Idea. Federated learning addresses the case where the data cannot be shared *even for training*: the legal and ethical constraints of Chapter 7 survive every de-identification attempt. So instead of moving the data to the model, the model moves to the data. A central *server* coordinates the training of one *global* model, while each participating site trains a *local* model on its own private data and never exports that data. (Source: `20260917_lecture06_ethics.pdf`, slide 23)

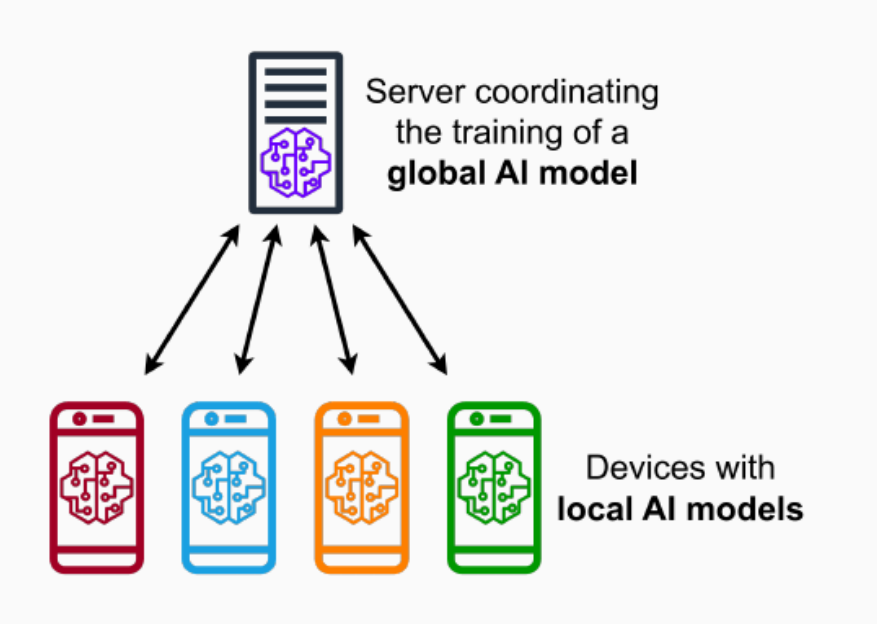


Figure 25: The federated-learning scheme: a central server coordinates the training of a global AI model while each client holds only a local model trained on locally held data; only parameters are exchanged. Source: `20260917_lecture06_ethics.pdf`, slide 23.

One round, step by step. The deck shows the picture but not the exchange itself, which is *No source in this repo (deck not provided).* The usual round is:

1. the server sends the current global parameters to every client;
2. each client trains locally for a while on its own data;
3. each client sends its updated parameters (never images) back;
4. the server aggregates them into a new global model, and the next round starts.

Exam case 1: five hospitals, disappointing accuracy. Five hospitals train a lung-nodule detector on CT, sending only weights to a central server. After several rounds the global accuracy is lower than expected *even though each hospital's model performs well locally.* The reason is that the sites have different scanners and patient populations, so the local updates come from differently distributed data and their aggregate need not be good for any of them. Good local scores therefore say nothing about the global model, which has to be validated on a set that reflects the target distribution (this last implication is *No source in this repo*). (Source: `exams/exam2025.pdf`, §3b)

Exam case 2: 1M, 1K and 1K scans. One hospital has 1M short-axis ultrasound heart scans, hospitals 2 and 3 have 1K each, and nothing may be shared. Federated learning fits: all three train

locally and exchange only parameters, so the small hospitals benefit from a dataset a thousand times larger than their own and the large one gains the others' diversity. Two design choices decide whether it works: the *aggregation rule* (a plain average would give a 1K site the same weight as the 1M site, so contributions can be weighted by data size), and the *heterogeneity* between sites, which can leave the global model worse than a locally trained one at some site, as in case 1. *No source in this repo (deck not provided)*: the deck introduces the server/client scheme but not aggregation or weighting; `exams/quiz2024.pdf`, Q1 gives only the scenario.

9.3 Privacy-preserving Learning: Differential Privacy

Hiding the raw files does not answer the question: how much can the *output* of an analysis reveal about the individuals in its input? The deck's answer is **differential privacy**. Take two databases D_1 and D_2 that differ in exactly one person's data (in the picture, D_2 is D_1 plus Joe). Run the same analysis M on both, giving answer A from D_1 and B from D_2. The analysis satisfies differential privacy if, for all such pairs, A and B are *indistinguishable*. In words: whether or not any one person's record is present, what an observer sees is essentially the same, so nobody can be identified, or even shown to have taken part, from the published statistic or trained model. (Source: `20260917_lecture06_ethics.pdf`, slide 24)

The slide gives only this definition and the keyword. How an analysis is made to satisfy it (deliberately adding noise) and the idea of a *privacy budget* ε are not in the deck: *No source in this repo (deck not provided).* The practical consequence for medical data is still direct: the criterion constrains what may be released from a hospital dataset, and it is what would make parameters or predictions leaving a federated site safe to release even though they were computed from private scans. (Source: `20260917_lecture06_ethics.pdf`, slide 24)

9.4 Drawbacks: No Free Lunch

The deck's drawbacks slide is titled *No free lunch*: both federated learning and differential privacy lead to **decreased performance** and a **higher computational cost**. They pay in different ways. Federated learning never pools the data, so it trains on updates from small, heterogeneous local sets; every round also needs local training and parameter exchange at every client, so time and communication grow with the number of rounds and participants. Differential privacy limits how much any single record can influence the output, which costs information. The lower accuracy seen in exam case 1 (good local models, weaker aggregate) is exactly this general statement in action: not sharing the data gives a model that is harder to train, more expensive to run and, all else equal, less accurate than the centralised alternative. (Source: `20260917_lecture06_ethics.pdf`, slide 25)

9.5 Foundation Models: SAM, MedSAM and Beyond

What the lecture means by "foundation model". Any general model trained on a "big enough" dataset (the lecture calls it a popular word to throw in). It distinguishes four independent axes: *promptable or not*; *general or medical*; *fine-tuned or not*; *supervised or self-supervised.* (Source: `20260929_lecture08_reconstruction.pdf`, slide 21)

SAM. The Segment Anything Model (Kirillov et al., ICCV 2023) is an *image encoder* that produces an image embedding, plus a *prompt encoder* taking points, boxes, text or a mask, plus a *mask decoder* that outputs several candidate masks with scores. It was developed and trained on 2D natural images; SAM2 and SAM3 are more powerful but also more costly to run. This is the property the 2025 exam (§3c) asks about: SAM does prompt-based segmentation, so a few clicks or a rough box tell it *what* to segment, with no retraining per case or structure. The other exam options are wrong for the same reason: SAM does not predict pixel intensities instead of masks, does not retrain itself at each click, and does not need a fully annotated training set from the same hospital. (Source: `20260929_lecture08_reconstruction.pdf`, slide 22; `exams/exam2025.pdf`, §3c)

Fine-tuning SAM for medical images. A timeline (from Magg, Kervadec and Sánchez, 2024) shows the wave of medical adaptations after SAM's release in April 2023 (Med-SAM, SAM-Med2D, SAM-Med3D) and after SAM2's release (Med-SAM2). A representative example is MedSAM (Ma et al., *Nature Communications* 2024), trained on a large collection of images spanning many modalities and organs and evaluated on external validation tasks against SAM, U-Net and DeepLabV3+ (specialist models). (Source: `20260929_lecture08_reconstruction.pdf`, slides 23–25) Because SAM and its derivatives are limited to 2D, 3D volumes such as MRI are handled with the *video mode* of SAM2 and later models, treating the slices as frames. (Source: `20260929_lecture08_reconstruction.pdf`, slide 26)

In-domain performance versus generalisation. The slide titled "Trade-off between in-domain performances and good generalization" shows 3D bone segmentations (shoulder, lower-leg and hip bones) on a private Amsterdam UMC CT dataset from Magg et al. (2024, zero-shot capability of SAM-family models for bone segmentation in CT). For two settings, marked by a blue and a green square, it shows the worst, median and best cases by Dice: the blue setting yields fragmented, incomplete bones even at the median, the green one clean bones. *The slide carries no legend saying which setting is SAM and which is MedSAM*, but the slide's title, and the course flashcard stating that "there is a trade-off between in-domain performances *after fine-tuning*, and generalisation ability", both say that fine-tuning buys in-domain accuracy at the cost of generalisation. On a *private* dataset, which is out-of-domain for every model, this points to the original **SAM** as the better performer (the clean, green row) and fine-tuned MedSAM as the fragmented one — the expected answer to exam §5g, inferred from the title and the flashcard since the figure itself is unlabelled (Source: `UvA__AI4MIv0.1.apkg`). (Source: `20260929_lecture08_reconstruction.pdf`, slide 30; `exams/exam2025.pdf`, §5g)

nnInteractive. From the team behind nnU-Net (§2.4), nnInteractive is an interactive 3D segmentation tool. Its slide shows diverse interactions (point, box, lasso, scribble), two prompt types (positive and negative), axis-independent 2D prompting followed by interactive refinement, a precise 3D mask as result, and open-set, modality-agnostic use. (Source: `20260929_lecture08_reconstruction.pdf`, slide 31)

Stability against user variability. A prompt-based model is only as reliable as its answer to slightly different prompts. Magg et al. (2026) recorded where different annotators place a centre point or a bounding box for the wrist, lower leg, shoulder and hip, and the spread differs by anatomical region. The lecture then shows, for the same reference mask and prompt, how the segmentation varies for slightly different human prompts: in the wrist example the SAM 2.1T segmentation visibly changes with the prompt, while the nnInteractive segmentation appears to stay close to the reference (a visual comparison on the slides, not a reported number). (Source: `20260929_lecture08_reconstruction.pdf`, slides 32–34)

Datasets and embedding models. TotalSegmentator is both a dataset and a trained model, covering skeleton, gastrointestinal tract, cardiovascular system, other organs and muscles, with 1939 CTs and 1296 MRIs available. The model is released under the Apache licence and the base datasets under CC-BY, with commercial licences covering more anatomies (Source: `UvA__AI4MIv0.1.apkg`). *(slide 35)* Foundation models can also produce *embeddings*: Virchow (Vorontsov et al., 2023) is a digital-pathology model trained on 1.5 million H&E slides with the DINOv2 algorithm, that embeds 224×224 tissue tiles into a representation adaptable to diverse tasks. *(slide 36)* (Source: `20260929_lecture08_reconstruction.pdf`, slides 35–36)

Take-away. The lecture's summary pairs foundation models with a warning: domain shift and domain adaptation are not to be under-estimated. (Source: `20260929_lecture08_reconstruction.pdf`, slide 38)

10 Classification and Explainability

The classification lecture is deliberately short and is organised around three questions rather than around an architecture: what the clinical question actually is, which label one should learn from when experts disagree, and how to classify images that are far too large to fit in a network. The explainability methods asked about in the 2025 exam (LIME, SHAP) are *not* in this deck: *No source in this repo* (see the exam appendix, §A). (Source: `20261001_classification.pdf`, slides 3, 11, 21, 26)

10.1 Classification among the imaging tasks

The deck positions classification against the two other recognition tasks on the same cardiac MRI slice: *classification* assigns a single label to the whole image ("left ventricle"), *object detection* returns a bounding box around the structure, and *image segmentation* labels every pixel (the voxel-wise classification of §2). The output therefore becomes progressively richer: one label per image, one box per object, one label per voxel. (Source: `20261001_classification.pdf`, slide 6)

The workhorse is again the convolutional neural network, going back to LeCun et al.'s 1989 network for handwritten zip-code recognition. The point the deck stresses is that in such a classification CNN both the *input* and the *output* are of fixed size — in contrast with the fully convolutional networks used for segmentation, whose output size follows the input (§2). Many newer "state-of-the-art" architectures exist, but the lecture does not go into them. (Source: `20261001_classification.pdf`, slide 7)

10.2 What is the question, actually?

Taking histopathology as the concrete example, the lecture argues that the hard part of a classification problem is formulating the question. "Will the patient survive?" is a binary question; "for how long?" can be posed as binary (above or below a threshold), as a multi-class problem over bins of survival time, or as a regression. A fourth, easily forgotten question sits above all of these: *is the label reliable?* The lecturer's motto, "everything in medical imaging is a maybe", sets up the first case study. (Source: `20261001_classification.pdf`, slides 8–10)

10.3 Panels, consensus, agreements and disagreements

Case study: Barrett's esophagus. Biopsies of Barrett's esophagus (whole histopathology slides) are graded into four possible labels: no dysplasia (NDBE), low-grade dysplasia (LGD), high-grade dysplasia (HGD), and indefinite (IND). The grade is not an objective measurement but an expert opinion, and difficult cases are therefore referred to a panel. (Source: `20261001_classification.pdf`, slide 12)

The Dutch Esophageal Pathology Panel (DEPP). The DEPP reviews difficult cases sent from the whole country. It is a panel of experts whose composition evolves over time: the deck shows the participation matrix of the 20 experts over 10 years, in which no single pathologist is present at every meeting. A panel works as follows: multiple pathologists meet at a fixed time; if more than 75% of the people in the room agree, that label becomes the *consensus label*; if not, they discuss until a consensus is reached. (Source: `20261001_classification.pdf`, slides 13–14)

Do experts agree? The pairwise agreement matrix between the pathologists (slide 15) shows that they do not fully agree with each other, which raises the central question for training: *from which expert should you get your labels?* The consensus label is one option, but it hides the disagreement and depends on who happened to be in the room. (Source: `20261001_classification.pdf`, slide 15)

One network per rater. The idea pursued in the case study is to train one network per rater, each on that rater's own labels. Most, but not all, of these per-rater networks converged (training curves on slide 16). Comparing each network's performance against its own rater's labels (intra-rater F1-score) with its performance against the consensus label (consensus F1-score) reveals that the

two are correlated (Figure 26): networks that reproduce their own rater well also agree well with the consensus, and networks trained on more samples tend to sit higher on both axes. (Source: `20261001_classification.pdf`, slides 16–17)

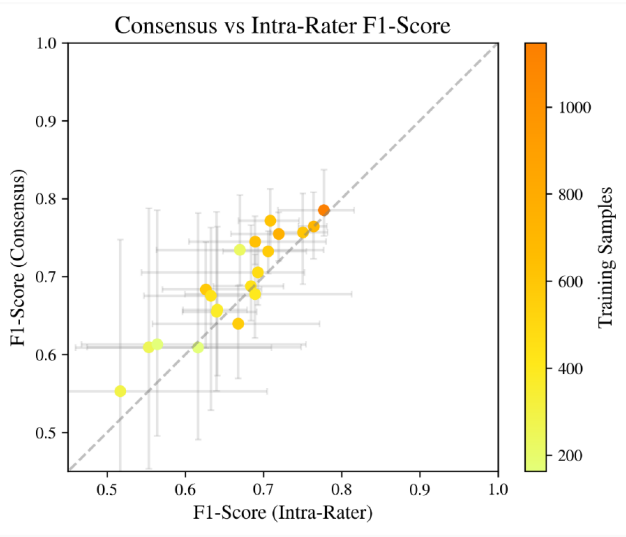


Figure 26: Consensus F1-score versus intra-rater F1-score for the per-rater networks, coloured by the number of training samples; the dashed line is the identity. Source: `20261001_classification.pdf`, slide 17.

A virtual panel. The per-rater networks can then be combined into a *virtual panel*, mimicking the real DEPP: each network "votes" as its rater would. Compared with a single network trained on the consensus labels, the virtual panel is (i) *interpretable*, since one can see which virtual raters agree or disagree on a case, and (ii) *better calibrated*: the expected calibration error (defined in §5.7, (61)) drops from 0.183 for the single consensus network to 0.050 for the (non-selective) virtual panel, whose reliability diagram follows the diagonal far more closely (Figure 27). The deck summarises the gain as better performance *on this specific application* — the caveat is explicit — together with a free estimation of *uncertainty*, obtained from the spread of the panel's votes. (Source: `20261001_classification.pdf`, slide 18)

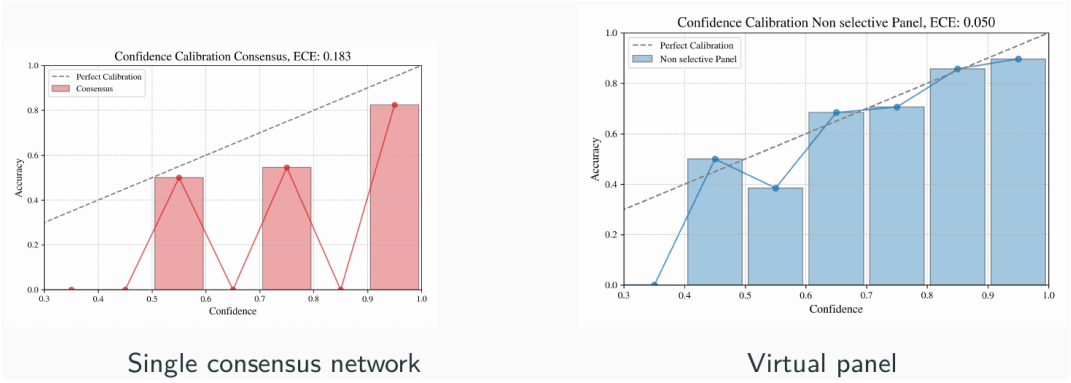


Figure 27: Reliability diagrams of a single network trained on consensus labels (left, ECE 0.183) and of the virtual panel of per-rater networks (right, ECE 0.050). Source: `20261001_classification.pdf`, slide 18.

Take-away. There are ways to leverage possibly conflicting labels rather than collapsing them; assessing uncertainty is as important as the prediction itself; and this is a whole research field in

itself, in which confidence, uncertainty and interpretability are all intertwined (the deck points to the UNSURE workshop at MICCAI and its MELBA special issues as the venue). The "Computer says no" sketch closes the part as a reminder of what an unexplained, overconfident automated decision looks like to a patient. The same message — labels are medical opinions subject to inter- and intra-rater variability — appears for segmentation metrics in §5. (Source: `20261001_classification.pdf`, slides 19–20)

10.4 Multiple-instance learning and very large images

The problem. Some images, such as whole-slide images (WSI) in histopathology, are too big to be processed at once, and come with a *single label for the whole image*. The information that determines the label is *somewhere* in the image, but one does not know where; and one cannot simply downsample the image, because the diagnostic detail would be lost. (Source: `20261001_classification.pdf`, slide 22)

Multiple-instance learning (MIL). Until recently the standard answer was MIL, built on the idea of *bags* of instances. A *positive bag* is one in which at least one instance has a positive label; a *negative bag* is one in which no instance has a positive label. A sub-patched WSI is exactly such a bag of patches: the slide-level label is known, the patch-level labels are not. (Source: `20261001_classification.pdf`, slide 23)

Foundation models to the rescue. The current approach ("FOMO to the rescue", Figure 28) is a three-step pipeline: split the slide into many tissue tiles, filtering out the background; compute an embedding of every tile with a pathology foundation model such as Virchow (a million-slide digital pathology foundation model); and learn the slide-level task from those embeddings with an *aggregator*, whose architecture varies. In the Virchow paper the same aggregated embeddings are adapted to pan-cancer detection, pan-cancer subtyping and digital biomarker prediction. The use of Virchow embeddings for downstream tasks is also listed among the foundation models of §9.5. (Source: `20261001_classification.pdf`, slide 24)

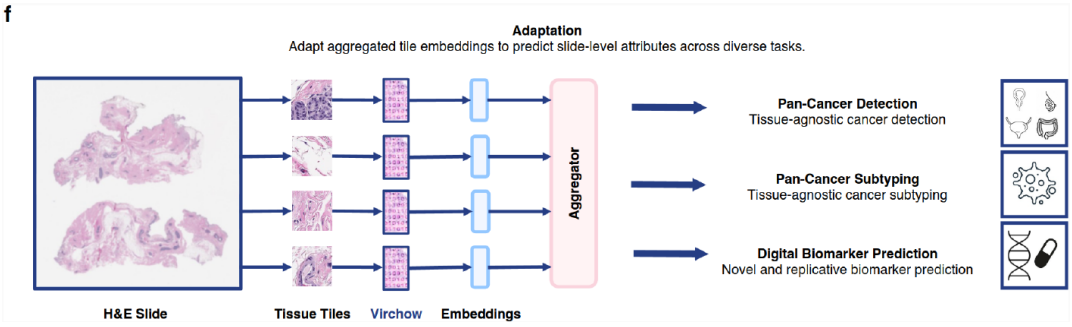


Figure 28: Slide-level classification with a pathology foundation model: the H&E slide is tiled, each tile is embedded by Virchow, and an aggregator maps the set of embeddings to slide-level predictions. Source: `20261001_classification.pdf`, slide 24.

The aggregator. The tile embeddings can be combined with attention, self-attention or other mechanisms, and the right choice depends on the clinical scenario. In the first scenario one only has to find the "smoking gun": a single positive patch suffices for the diagnosis, which is the easier case and is exactly the MIL positive-bag assumption. In the second, the *combination* and the *spatial relationship* of patches make the diagnosis, which is more difficult because the aggregator must model interactions between tiles. The choice is very application dependent. (Source: `20261001_classification.pdf`, slide 25)

10.5 Quantification versus decision

The final part returns to the formulation of the question. Discrete categories may be subjective and ambiguous (as the Barrett grades showed), and the goal is to help clinicians, not to replace them.

Quantification — providing better *biomarkers* on which the clinician bases the decision — is therefore appealing, and it also helps interpretability. Two examples illustrate it. For Barrett's esophagus, Botros et al. (Modern Pathology, 2024) output a *probability map* over the slide, highlighting where and how strongly each dysplasia grade is predicted instead of returning a single grade. For knee implants, the AtMoves project (presented in a guest talk) quantifies implant displacement to detect loosening, rather than issuing a binary "loose / not loose" verdict. (Source: `20261001_classification.pdf`, slides 27–29)

Take-aways. Classification in medical imaging faces three recurring difficulties: limited datasets, (very) big images, and ambiguous labels. Before building anything, one should take the time to think about which question is really worth asking. (Source: `20261001_classification.pdf`, slide 30)

A Appendix: Past Exam Questions with Model Answers

This appendix goes through every question of the two past papers in the repository — the 2025 exam (`exams/exam2025.pdf`, 22 October 2025, two hours, one A4 cheat sheet allowed) and the 2024 quiz (`exams/quiz2024.pdf`) — and gives for each a concise model answer followed by a pointer (→) to the section that develops it. For the quiz, the lecturer's handwritten answers in `quiz2024_annotated.pdf` are reproduced and marked **Official**; every other answer is derived from the notes. Questions whose topic is not covered by any source in this repository are marked *No source in this repo.* The exam's instructions stress that for open questions clarity and concision are graded, not the quality of the prose. (Source: `exams/exam2025.pdf`, p. 1; `quiz2024_annotated.pdf`)

A.1 2025 exam

Modality

1a Operator-dependent modality. Ultrasound (a). → §1, Table 1.

1b Relatively slow, may need sedation in young children. MRI (b). → §1, Table 1.

1c Two applications of CT. The notes give CT as quick, anatomical, 3D imaging that can be viewed and rotated in any plane; the two uses that follow are anatomical diagnosis (e.g. thoracic CT for lung, heart, trachea and esophagus) and 3D localisation/planning from the reconstructed volume. The exact pair expected is not stated in the slides. → §1.

1d Esophagus in thoracic CT (SegTHOR): one challenge, one strategy. Challenge: the esophagus is small and thin relative to the lungs and heart (class imbalance) and has low contrast against the surrounding soft tissue, so overlap losses and metrics are dominated by large organs. Strategies from the notes: a loss suited to unbalanced segmentation (boundary loss (13), or compound losses such as Dice+CE/Dice+TopK, §3.8); foreground oversampling of patches as in nnU-Net (§2.4.1); post-processing such as connected-component filtering. → §2, §3.

1e Back-projection for A, B, C, D. $A = 2$, $B = 5$, $C = 4$, $D = 3$; full derivation in §1 (worked example after (2)).

Classification

The classification lecture of this course (§10) covers multi-rater labels, multiple-instance learning and quantification, but *not* the specific content of these questions.

2a Two solutions for a large number of classes. *No source in this repo.* Nearest material: rethinking the question as binary / binned / regression (§10.2).

2b Implants or contrast agent classified as "cancer". *No source in this repo.* Nearest material: the take-away that the label and the question must be scrutinised, and quantification/probability maps that show *where* the evidence is (§10.5).

2c Unstable / anatomically irrelevant LIME explanations. *No source in this repo* (LIME is not in any deck).

2d SHAP: baseline 0.3, contributions $+0.2, +0.1, +0.1$. $0.3 + 0.2 + 0.1 + 0.1 = 0.7$ (e), since SHAP values add up from the baseline to the prediction. *No source in this repo* for SHAP itself.

Domain adaptation

3a Accuracy drops from 92% (Hospital A) to 68% (Hospital B). Domain shift: a different scanner (and vendor) and contrast settings change the image distribution, so the test domain no longer matches the training domain. Two remedies: fine-tune the pretrained model on (a few) labelled images from Hospital B (transfer learning), or apply domain adaptation from source to target domain (source-free if Hospital A's data cannot be shared). → §9.

3b Federated model worse than local ones. (b): the hospitals' data come from different scanners and populations (heterogeneous, non-identically distributed data), so the aggregated model struggles to fit all of them; federated learning in general pays a performance and compute price. → §9.2.

3c What lets SAM segment from a few clicks. (a): prompt-based segmentation — a prompt encoder takes points, boxes, text or masks. → §9.5.

Reconstruction

4 Choose among direct / two-step-iterative / raw-to-task / conventional DL for tasks A–C. *No source in this repo*: the four-approach taxonomy is not in the reconstruction deck. Background: learned reconstruction from under-sampled k-space and its consistency losses, §8, (77); classical back-projection, §1.

Segmentation

5a Conditions for a usable loss. (c) differentiable with respect to the *network output* and (e) real-valued (a scalar to minimise). It need not lie in $[0, 1]$, need not be positive, and need not be differentiable with respect to the annotation, which is constant. → §3.2.

5b Derivative of L_{CE}. Only the term of voxel i, class k depends on $s_\theta^{(i,k)}$, so $\partial L_{CE}/\partial s_\theta^{(i,k)} = -y^{(i,k)}/s_\theta^{(i,k)}$: $-1/s_\theta^{(i,k)}$ for the true class, 0 otherwise. → §3.3.1, (22).

5c When the CE gradient goes to infinity. When $y^{(i,k)} = 1$ and $s_\theta^{(i,k)} \to 0$: the network is confidently wrong on the true class, and $-1/s \to -\infty$ (hence the small ε in implementations, §2.12.1). → §3.3.1.

5d Two post-processing methods. (i) Connected-component analysis / non-maxima suppression: keep only the largest connected component; applicable when the target is known to be a single object (an organ), not for multiple lesions. (ii) DenseCRF: minimise an energy with unary (network) and pairwise (intensity and distance) terms to align the segmentation with image edges; applicable when boundaries follow intensity edges, less so with low contrast. → §2 (post-processing), (9).

5e `DiceLoss` producing `nan`. When a class is absent from both `y` and `s` in a sample, `denom` is 0 and $0/0$ = `nan`. Fix: `num = 2 * einsum(...) + eps` and `denom = einsum(...) + eps`, e.g. `eps = 1e-10`. → §3.4, (24); (58).

5f Predicted segmentation from s_θ. $\arg\max$ over the class axis at each pixel: pixel $(0,0)$: $(1,0,0) \to 0$; $(0,1)$: $(0.9,0.1,0) \to 0$; $(0,2)$: $(0.3,0.6,0.1) \to 1$; $(1,0)$: $(0.8,0.15,0.05) \to 0$; $(1,1)$: $(0.2,0.7,0.1) \to 1$; $(1,2)$: $(0.2,0.3,0.5) \to 2$. Hence $S = \left(\begin{smallmatrix} 0 & 0 & 1 \\ 0 & 1 & 2 \end{smallmatrix}\right)$. → §2.12.1.

5g SAM vs. MedSAM on a private CT dataset. SAM (a), by the in-domain versus generalisation trade-off of fine-tuning. → §9.5.

Metrics

6a Order the steps of a distance-based metric. (1) convert predicted probabilities to one-hot; (2) extract the *boundary*; (3) compute the EDT on both *boundaries*; (4) mask the distances with the *boundary* (of the other segmentation); (5) reduce the list of distances to one number (max, percentile, mean). "Upsample for improved precision" is superfluous. → §6.

6b Fill the HD blanks.

$$\mathrm{HD}(S, Y; k) = \max\Big(\max_{i \in \partial\Omega_S^{(k)}} \min_{j \in \partial\Omega_Y^{(k)}} d(i,j),\ \max_{i \in \partial\Omega_Y^{(k)}} \min_{j \in \partial\Omega_S^{(k)}} d(i,j) \Big).$$

→ (65).

6c Number of EDTs for Dice, HD and ASSD with K classes. $2K$: Dice needs none (it counts voxels); HD needs two EDTs per foreground class (one per boundary); ASSD reuses the same two distance maps. → §6.

6d Speeding up the computation. Compute the two EDTs per class once and share them across HD, HD95 and ASSD; skip classes that are absent from the prediction or the label (the distance is undefined there, assign the default value directly); run the EDT on GPU (cuCIM) or use DisTorch, which avoids the EDT altogether. → §6, §6.4, §6.6.

6e Two cases where $\partial\Omega_S = \Omega_S$. (i) The predicted object has no interior voxel: a thin structure (one or two voxels wide, a line, an isolated voxel), so every voxel touches the background and is kept as boundary; (ii) the prediction is empty, $\partial\emptyset = \emptyset$. → §6.6.

6f Shuffled but consistent slice order. Dice is unchanged (it counts overlapping voxels regardless of position); ASSD (and HD) change, because boundaries and Euclidean distances depend on the 3D arrangement of the slices. → §6.6.

Ethics and reproducibility

7a Rank by ease of anonymisation. Easiest: biopsy scan (tissue level, no outward features); then head MRI (identifiable face, but mitigated by skull stripping/cropping); then whole-body CT (body shape and proportions identify the person, cannot be cropped away); hardest: iris photo (a biometric identifier, functionally impossible to protect). → §7.1.

7b Retroactive consent, deceased patients. Consent may be waived when it cannot be asked (deceased patients, lost contact details) or "cannot reasonably be required"; the decision is made by the institution's ethics committee, not by the researcher. → §7.1.

7c Different machine/GPU, different nnU-Net configuration? No: the configuration is a deterministic function of the dataset fingerprint through fixed rules and a predefined memory budget; only the trained weights can differ (training randomness). → §7.2.6, §2.4.1.

7d Anonymisation vs. pseudo-anonymisation. Both replace identifiers by random IDs; in pseudo-anonymisation the person→ID mapping is kept somewhere (re-identification possible), in anonymisation it is deleted. → §7.1.

7e One meaning of reproducibility, one improvement. E.g. "re-running the same code on the same data on the same machine gives the same model"; improve it by fixing the seeds of every random generator and enabling deterministic algorithms (other meanings: different machine, different data, statistical reproducibility). → §7.2.

7f Two advantages of public challenges. A common benchmark with a held-out test set evaluated by the organisers (comparable results), and public data, annotations and protocols that others can reuse. → §7.1.

Registration

8a What the affine matrix encodes. (b) size of voxels, (d) position in the real world, (e) rotation. → §4.3, (45).

8b Rigid or non-rigid, scans years apart, ages 13–25. Lungs, heart: non-rigid (breathing, heartbeat). Kidneys: non-rigid. Skull, femur: rigid would suffice for adults, but here the patients grow between the scans, so at least affine (with scaling) and in general non-rigid. → §4.6, Table 6.

8c Learned vs. classical registration. Advantage: fast inference (one forward pass instead of an optimisation per pair). Disadvantage: correctness is not guaranteed at inference (the network may not reach the optimum for a new pair; depends on the training data). → §4.5.

8d Minimal changes to turn a segmentation U-Net into a registration network. Input: concatenate fixed and moving image (two channels); output: three channels (a displacement vector per voxel) with no softmax; add a differentiable spatial-transform layer that warps the moving image; train with a similarity loss plus a smoothness regulariser instead of CE/Dice. → §4.5, (50).

A.2 2024 quiz

Q1 One hospital with 1M ultrasound scans, two with 1K; no data sharing. Official: federated learning. → §9.2.

Q2 Variable image sizes, batch size 1, no sub-patching: must one resize for an FCN? Official: no need to resize, since an FCN's output matches its input in size. → §2.

Q3 Dice when label and prediction of class k are both empty. Official: $\mathrm{DSC} = \frac{2\cdot 0}{0+0} = \frac{0}{0}$, undefined; fix with $\frac{0+\varepsilon}{0+0+\varepsilon} = 1$, ε very small; an `if/else` is also acceptable; half a point is lost if the bias is not mentioned. The bias: absent structures count as perfect predictions, inflating the averaged score. → §5.3, Table 7, (58).

Q4 Brain-MRI model to lung CT without retraining from scratch. Official: domain adaptation. → §9.

Q5 One key difference between CE and Dice supervision. Official (either): the Dice gradient is much smaller in magnitude, $|\partial L_{DSC}/\partial s_\theta^{(i,k)}| \ll |\partial L_{CE}/\partial s_\theta^{(i,k)}|$; and the Dice gradient takes only two different values over the image (per class), whereas the CE gradient is voxel-specific. → §3.3.1, §3.5, §3.6, Figure 11.

Q6 Rigid or non-rigid, one week apart. Official: rigid: femur; either accepted: ribs; non-rigid: kidney, heart, lungs. → §4.6.

Q7 Training labels for learned registration (VoxelMorph). Official: trick question — no labels are needed, it uses unsupervised functions (image similarity + regulariser). Bonus point: mention annotated landmarks or segmentations as optional extra supervision. → §4.5, (51), (52).

Q8 Ultrasound or CT: which generalises worse to new sites? Official: ultrasound, as it is manually operated. → §1, §9.

Q9 Public code and paper, partly private validation data: reproducible? Official (graded generously): full points for noting that "reproducible" is ill-defined, that some results can be reproduced, and that the method can still be applied to other tasks. → §7.2.

Q10 Three nnU-Net fingerprint statistics. Official: see the paper — spacings, modality, shape, intensity distribution. → §2.4.1.

Q11 (Bonus) Why von Neumann told Shannon to call it "entropy". Official: "[. . .] no one really knows what it is, so in a debate you'll always have the advantage." → §3.3.

A.3 Coverage gaps

Three exam topics have no source in this repository: the classification/explainability questions §2a–§2c (large-class regimes, artefact-driven misclassification, LIME) and the SHAP definition behind §2d; and the four-approach reconstruction taxonomy of §4. The guest lectures (lectures 10–11) are not in the repository either, beyond what the exam (§5g) and the reconstruction deck say about them. (Source: `exams/exam2025.pdf`, §2, §4, §5g)